Applied Machine Learning

School of Computer Science, UPES, Dehradun.

LABORATORY FILE

B.TECH. - IV Semester

Jan - May. (2026)

SUBMITTED TO :-

Dr. Sahinur Rahman Laskar ,

Assistant Professor,

SoCS, UPES

SUBMITTED BY :-

Name : Pranav Joshi

Sap ID : 590012855

Batch : 19

Index

S.no	Experiment	Date
1	Data preprocessing	27-01-26
2	Predicting house price using linear regression	3-02-26
3	Predicting stock price using regression	10-02-26
4	Predicting customer churn rate using logistic regression	10-02-26
5	Spam detection (Classification and comparative analysis)	24-02-26
6	Credit Risk Assessment and comparative analysis	24-02-26
7	Anomaly detection and comparative analysis	24-02-26
8	Student Performance Level Analysis	17-02-26
9	Physiological signal classification	24-02-26
10	Iris classification	24-02-26
11	Prediction using bagging classifier	31-02-26
12	Regularization (ridge and lasso)	31-02-26
13	Mini Project	30-03-26

Experiment - 1

Aim => Text pre-processing in IPL dataset using python libraries .

Mount the drive

```
In [ ]: import pandas as pd
```

```
In [ ]: df=pd.read_csv("/content/drive/MyDrive/AIML_lab_sem4/exp1/data.csv")
```

```
In [ ]: df.head()
```

Out[ ]:

	id	season	city	date	team1	team2	toss_winner	toss_d
0	1	2008	Bangalore	2008-04-18	Kolkata Knight Riders	Royal Challengers Bangalore	Royal Challengers Bangalore	
1	2	2008	Chandigarh	2008-04-19	Chennai Super Kings	Kings XI Punjab	Chennai Super Kings	
2	3	2008	Delhi	2008-04-19	Rajasthan Royals	Delhi Daredevils	Rajasthan Royals	
3	4	2008	Mumbai	2008-04-20	Mumbai Indians	Royal Challengers Bangalore	Mumbai Indians	
4	5	2008	Kolkata	2008-04-20	Deccan Chargers	Kolkata Knight Riders	Deccan Chargers	

```
In [ ]: import matplotlib.pyplot as plt
        import seaborn as sns
```

```
In [ ]: df.columns
```

```
Out[ ]: Index(['id', 'season', 'city', 'date', 'team1', 'team2', 'toss_winner',
               'toss_decision', 'result', 'dl_applied', 'winner', 'win_by_runs',
               'win_by_wickets', 'player_of_match', 'venue', 'umpire1', 'umpire2',
               'umpire3'],
              dtype='object')
```

```
In [ ]: df.shape
```

```
Out[ ]: (577, 18)
```

Ques 1 . Team name normalization

1 Some team names appear in different textual formats (e.g., case differences or

extra spaces). Clean the team1 and team2 columns by:

2. Converting text to lowercase
3. Removing leading/trailing spaces
4. Verifying unique team names after cleaning

lower case all

In [ ]:
```
df['team1'] = df['team1'].str.lower().str.strip()
df['team2'] = df['team2'].str.lower().str.strip()
```

In [ ]:
```
df[['team1' , 'team2']]
```

Out[ ]:

	team1	team2
0	kolkata knight riders	royal challengers bangalore
1	chennai super kings	kings xi punjab
2	rajasthan royals	delhi daredevils
3	mumbai indians	royal challengers bangalore
4	deccan chargers	kolkata knight riders
...	...	...
572	delhi daredevils	royal challengers bangalore
573	gujarat lions	royal challengers bangalore
574	sunrisers hyderabad	kolkata knight riders
575	gujarat lions	sunrisers hyderabad
576	sunrisers hyderabad	royal challengers bangalore

577 rows × 2 columns

In [ ]:
```
unique_teams = pd.concat([df['team1'], df['team2']]).unique()
unique_teams
```

Out[ ]:
```
array(['kolkata knight riders', 'chennai super kings', 'rajasthan royals',
       'mumbai indians', 'deccan chargers', 'kings xi punjab',
       'royal challengers bangalore', 'delhi daredevils',
       'kochi tuskers kerala', 'pune warriors', 'sunrisers hyderabad',
       'rising pune supergiants', 'gujarat lions'], dtype=object)
```

City Name Standardization

Standardize the city column by:

1)Replacing missing city names with "Unknown"

2)Converting all city names to title case

3)Counting matches played in each city

In [ ]:
```
df.isnull().sum()
```

Out[ ]:

	0
id	0
season	0
city	7
date	0
team1	0
team2	0
toss_winner	0
toss_decision	0
result	0
dl_applied	0
winner	3
win_by_runs	0
win_by_wickets	0
player_of_match	3
venue	0
umpire1	0
umpire2	0
umpire3	577

dtype: int64

In [ ]:
```
df['city'] = df['city'].fillna('Unknown')
```

In [ ]:
```
df.isnull().sum()
```

Out[ ]:

	0
id	0
season	0
city	0
date	0
team1	0
team2	0
toss_winner	0
toss_decision	0
result	0
dl_applied	0
winner	3
win_by_runs	0
win_by_wickets	0
player_of_match	3
venue	0
umpire1	0
umpire2	0
umpire3	577

dtype: int64

In [ ]:
```
df['city']= df['city'].str.strip().str.title()
```

In [ ]:
```
df['city']
```

Out[ ]:

	city
0	Bangalore
1	Chandigarh
2	Delhi
3	Mumbai
4	Kolkata
...	...
572	Raipur
573	Bangalore
574	Delhi
575	Delhi
576	Bangalore

577 rows × 1 columns

dtype: object

In [ ]:
```
city_counts = df['city'].value_counts()
city_counts
```

Out[ ]:

city	count
Mumbai	77
Bangalore	58
Kolkata	54
Delhi	53
Chennai	48
Chandigarh	42
Hyderabad	41
Jaipur	33
Pune	25
Durban	15
Centurion	12
Ahmedabad	12
Visakhapatnam	11
Dharamsala	9
Johannesburg	8
Unknown	7
Abu Dhabi	7
Cape Town	7
Port Elizabeth	7
Ranchi	7
Cuttack	7
Raipur	6
Sharjah	6
Rajkot	5
Kochi	5
Kimberley	3
East London	3
Nagpur	3
Bloemfontein	2
Indore	2

	count
city	
Kanpur	2

dtype: int64

3. Toss Decision Text Analysis

Analyze the toss_decision column:

1)Extract unique decisions

2)Count how many times each decision was taken

3)Visualize the frequency using a bar chart

In [ ]:
```
choice = df['toss_decision'].unique()
choice
```

Out[ ]:
```
array(['field', 'bat'], dtype=object)
```

In [ ]:
```
dec = df['toss_decision'].value_counts()
dec
```

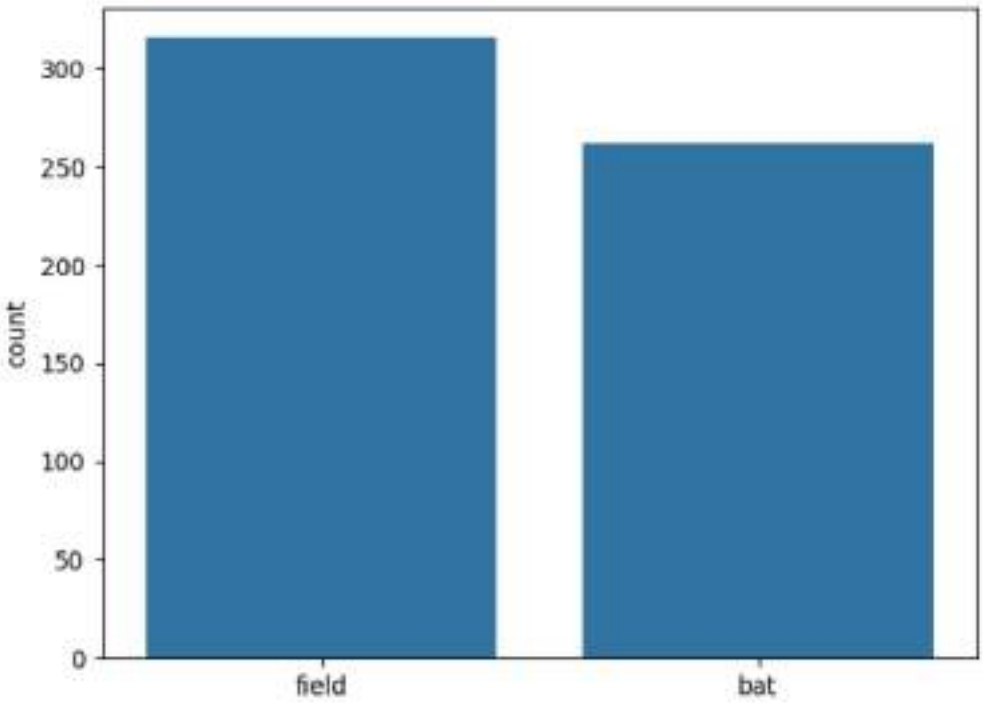
Out[ ]:

	count
toss_decision	
field	315
bat	262

dtype: int64

In [ ]:
```
sns.barplot(x = choice , y = dec)
```

Out[ ]:
```
<Axes: ylabel='count'>
```



4, Winner Name Extraction

From the winner column:

1)Identify and remove rows where the match result was "No Result" or "Tie"

2)Count how many matches each team won after cleaning text values

In [ ]:
```
df_temp = df[df['result'] != 'no result']
df_temp = df_temp[df_temp['result'] != 'tie']
```

In [ ]:
```
winner_counts = df['winner'].str.lower().str.strip().value_counts()
print(winner_counts)
```

```
winner
mumbai indians                 80
chennai super kings            79
royal challengers bangalore    70
kolkata knight riders          68
rajasthan royals               63
kings xi punjab                63
delhi daredevils               56
sunrisers hyderabad            34
deccan chargers                29
pune warriors                  12
gujarat lions                   9
kochi tuskers kerala            6
rising pune supergiants         5
Name: count, dtype: int64
```

In [ ]:
```
df.head()
```

Out[ ]:

	id	season	city	date	team1	team2	toss_winner	toss_de
0	1	2008	Bangalore	2008-04-18	kolkata knight riders	royal challengers bangalore	Royal Challengers Bangalore	
1	2	2008	Chandigarh	2008-04-19	chennai super kings	kings xi punjab	Chennai Super Kings	
2	3	2008	Delhi	2008-04-19	rajasthan royals	delhi daredevils	Rajasthan Royals	
3	4	2008	Mumbai	2008-04-20	mumbai indians	royal challengers bangalore	Mumbai Indians	
4	5	2008	Kolkata	2008-04-20	deccan chargers	kolkata knight riders	Deccan Chargers	

5. Player of the Match Text Frequency

Perform text analysis on player of the match:

1)Remove null values

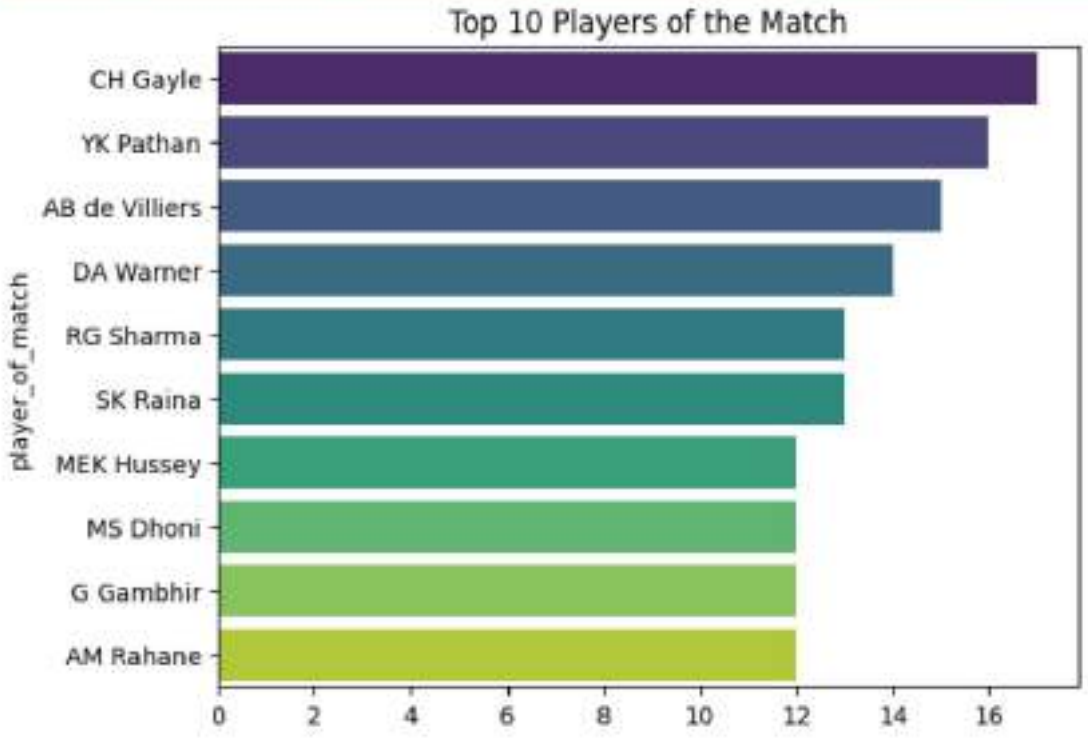
2)Find the top 10 most frequent player names

3)Plot the results using a Seaborn bar plot

In [ ]:
```
pom_top_10 = df['player_of_match'].dropna().value_counts().head(10)
```

In [ ]:
```
sns.barplot(x=pom_top_10.values, y=pom_top_10.index, palette='viridis')
plt.title('Top 10 Players of the Match')
plt.show()
```

```
/tmp/ipython-input-2314669101.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same e
ffect.

  sns.barplot(x=pom_top_10.values, y=pom_top_10.index, palette='viridis')
```

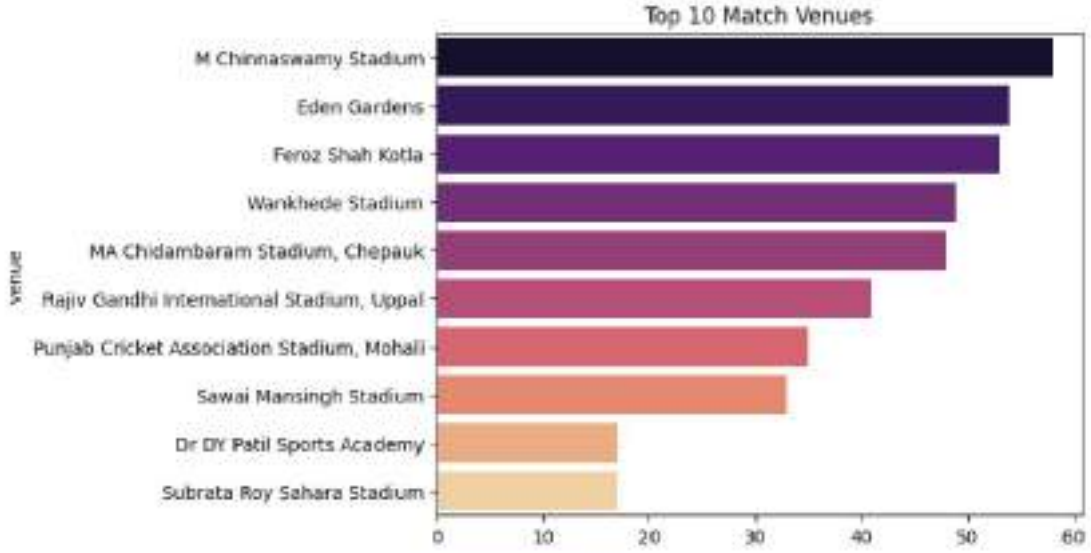


6. Venue Tokenization

Count how many matches were played in each venue and plot a bar chart for the top 10.

In [ ]:
```
venue_top_10 = df['venue'].value_counts().head(10)

sns.barplot(x=venue_top_10.values, y=venue_top_10.index, palette='magma')
plt.title('Top 10 Match Venues')
plt.show()
```

```
/tmp/ipython-input-3597930040.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same e
ffect.

  sns.barplot(x=venue_top_10.values, y=venue_top_10.index, palette='magma')
```



7. Umpire Name Cleaning

Clean umpire columns (umpire1, umpire2, umpire3) by:

1)Replacing missing values with "Not Assigned"

2)Removing duplicate umpire names per match

3)Finding the most frequently officiating umpire

In [ ]:
```
for col in ['umpire1', 'umpire2', 'umpire3']:
    df[col] = df[col].fillna('Not Assigned').str.strip()
```

In [ ]:
```
all_umpires = pd.concat([df['umpire1'], df['umpire2'], df['umpire3']])
```

In [ ]:
```
most_frequent = all_umpires[all_umpires != 'Not Assigned'].value_counts().idx
print(f"Most frequently officiating umpire: {most_frequent}")
```

```
Most frequently officiating umpire: HDPK Dharmasena
```

8. Create a new text column match_summary by

combining:

team1, team2, winner, and season

Example: "MI vs CSK – MI won in 2019"

Display sample summaries.

In [ ]:
```
def get_summary(row):
    return f"{str(row['team1'])} vs {str(row['team2'])} at {str(row['venue'])}

df['match_summary'] = df.apply(get_summary, axis=1)
print(df['match_summary'].head())
```

```
0    kolkata knight riders vs royal challengers ban...
1    chennai super kings vs kings xi punjab at Punj...
2    rajasthan royals vs delhi daredevils at Feroz ...
3    mumbai indians vs royal challengers bangalore ...
4    deccan chargers vs kolkata knight riders at Ed...
Name: match_summary, dtype: object
```

9. Result Type Text Analysis

Analyze the result column:

Identify different textual result types

Count their occurrences

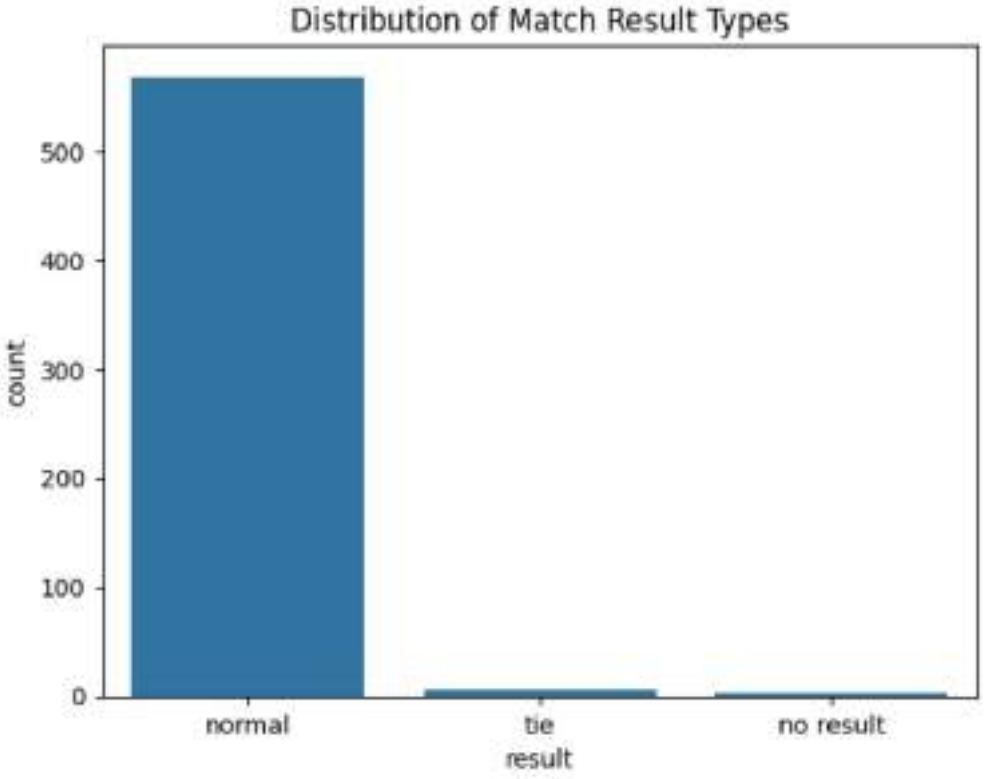
In [ ]:
```
result_types = df['result'].value_counts()
result_types
```

Out[ ]:

result	count
normal	568
tie	6
no result	3

dtype: int64

In [ ]:
```
sns.countplot(data=df, x='result')
plt.title('Distribution of Match Result Types')
plt.show()
```



10. Toss Winner vs Match Winner (Text Matching)

Compare text values in toss_winner and winner:

Create a boolean column indicating whether the toss winner also won the match
Visualize the comparison using a bar chart

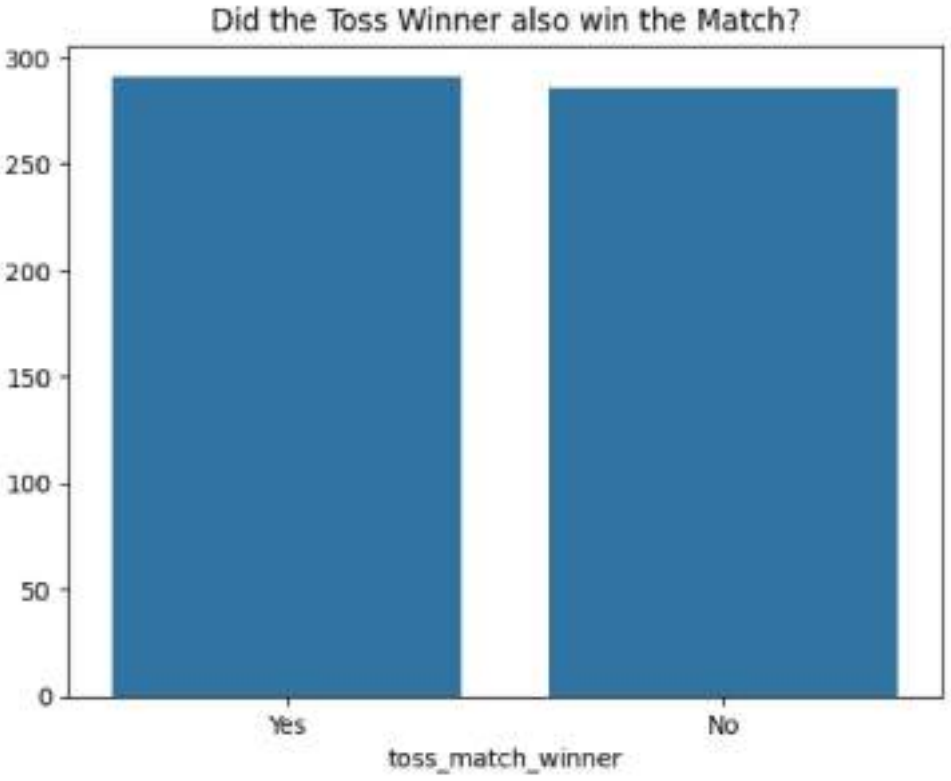
In [ ]:
```
df['toss_winner'] = df['toss_winner'].str.lower().str.strip()
df['winner'] = df['winner'].str.lower().str.strip() # Normalize winner column
df['toss_match_winner'] = df['toss_winner'] == df['winner']

comparison = df['toss_match_winner'].value_counts()
comparison
```

Out[ ]:

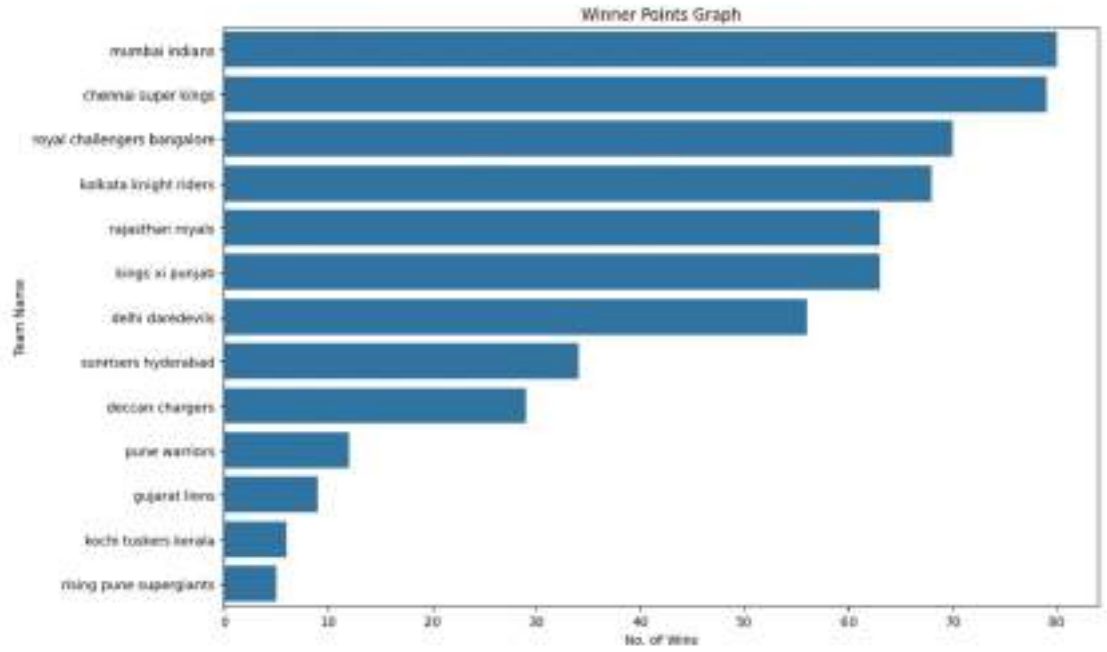
toss_match_winner	count
True	291
False	286

dtype: int64

In [ ]:
```
sns.barplot(x=comparison.index.map({True: 'Yes', False: 'No'}), y=comparison.v
plt.title('Did the Toss Winner also win the Match?')
plt.show()
```

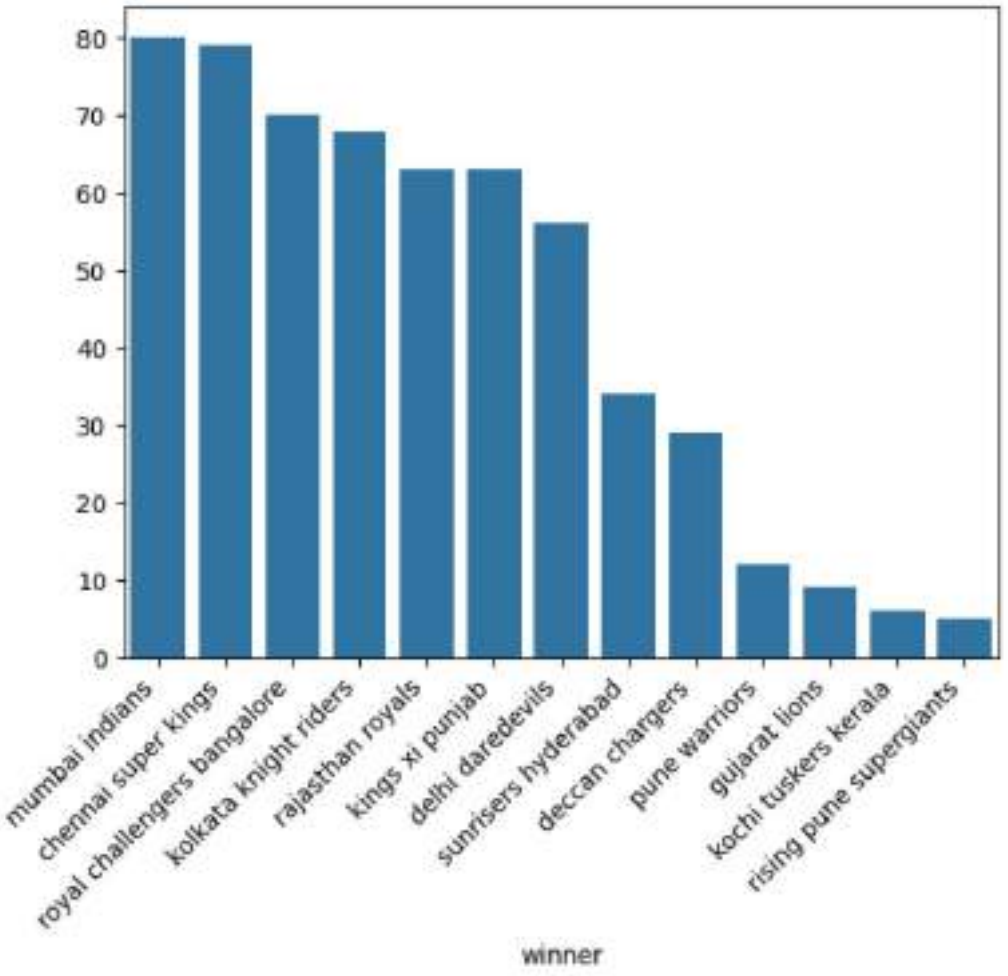


In [ ]:
```
winner_counts = df['winner'].str.lower().str.strip().value_counts()
plt.figure(figsize=(12, 8))
sns.barplot(x=winner_counts.values, y=winner_counts.index)

plt.title('Winner Points Graph')
plt.xlabel('No. of Wins')
plt.ylabel('Team Name')
plt.show()
```



In [ ]:
```
import seaborn as sns
sns.barplot(x= df['winner'].value_counts().index, y=df['winner'].value_counts(
plt.xticks(rotation=45,ha='right')
```

Out[ ]:
```
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12],
 [Text(0, 0, 'mumbai indians'),
  Text(1, 0, 'chennai super kings'),
  Text(2, 0, 'royal challengers bangalore'),
  Text(3, 0, 'kolkata knight riders'),
  Text(4, 0, 'rajasthan royals'),
  Text(5, 0, 'kings xi punjab'),
  Text(6, 0, 'delhi daredevils'),
  Text(7, 0, 'sunrisers hyderabad'),
  Text(8, 0, 'deccan chargers'),
  Text(9, 0, 'pune warriors'),
  Text(10, 0, 'gujarat lions'),
  Text(11, 0, 'kochi tuskers kerala'),
  Text(12, 0, 'rising pune supergiants')])
```


80
70
60
50
40
30
20
10
0
mumbai indians
chennai super kings
royal challengers bangalore
kolkata knight riders
rajasthan royals
kings xi punjab
delhi daredevils
sunrisers hyderabad
deccan chargers
pune warriors
gujarat lions
kochi tuskers kerala
rising pune supergiants
winner

Experiment => 2
Predicting Housing Prices

Aim => Develop a regression model to predict house prices based on features like location, size, and amenities.

Problem Statement:

The determination of real estate market value is a complex process influenced by a diverse interplay of structural, locational, and economic variables . Historically, property valuation has relied heavily on manual appraisals and the subjective expertise of agents, which can often be inconsistent, time-consuming, or influenced by human bias. As the real estate market grows in complexity, there is an increasing demand for objective, data-driven solutions that can provide rapid and accurate valuations without the variance found in manual estimations.

This project addresses these inefficiencies by seeking to automate the valuation process through the identification of intricate patterns within historical housing data. Given a dataset containing 120 detailed property records, the primary objective is to develop a robust regression model capable of transforming property attributes into precise market estimates . By leveraging machine learning, we aim to move away from subjective "best guess" valuations toward a reliable, evidence-based framework that ensures transparency and mathematical consistency for both buyers and sellers .

- **Input Features :**
    - **Location:** The neighborhood type (Rural, Urban, Suburban).
    - **Area_sqft:** The total floor area of the house in square feet.
    - **Bedrooms/Bathrooms:** The count of essential living spaces.
    - **Balcony/Parking:** Additional amenities available.
    - **Age_Years:** The age of the property (to account for depreciation or vintage value).
    - **Furnished:** Status of the interior furnishing (Yes/No).
- **Target Variable :**
    - **Price:** The estimated market value of the house (in thousands or millions of units).

Model Pipeline :

1. Data Understanding

- **Target Variable (y)**: The target is to calculate the price of the house, which represents the numerical value we want the model to learn to predict .
- **Input Features** :
    - **Numerical**: Area_sqft, Bedrooms, Bathrooms, Age_Years, Parking, and Balconies.
    - **Categorical**:Location (Urban, Suburban, Rural) and Furnished (Yes, No).
- In the code, we will use Pandas functions like pd.read_csv() to load the dataset and df.info() to verify that each row represents an individual house record .

2. Data Preprocessing

- **Handling Categorical Variables:** Since machine learning models require numerical input, we will convert text based categories like "Urban" or "Suburban" into numbers . We will use One-Hot Encoding with help of pandas.get_dummies() to create binary columns like Location_Urban and Furnished_Yes, which avoids introducing false ordinal relationships .

- **Feature and target Split**: We separate the data into X which will contain all the input columns. and y which will have the output column which will be the price column.

3. Train–Test Split:

We use the train_test_split() function to divide the data into a Training set (80%) to learn the relationships and a Test set (20%) to evaluate performance .

4. Regression Model & Training

- **Model Selection**: We choose Linear Regression as the mathematical model to learn how each feature contributes to the final price .
- **Training Process**: Using the model.fit() function, the model calculates the Coefficients (weights) that minimize prediction error . This is achieved by minimizing the error on the training data to find the best fit line.

5. Predictions

- Once trained, the model is applied to the unseen test data to generate price estimates.
- We use the model.predict() function on the test features X_test to see how well the model generalizes to new information.
- We plot the actual prices against the predicted prices to visually confirm if the model is correctly following the market's upward or downward trends.

6. Evaluate the Model

1. We assess the model's accuracy using Mean Squared Error (MSE) and R-Square via the sklearn.metrics library .
2. A lower MSE indicates a better fit as it penalizes large errors strongly, while the R2 score tells us how much of the price variance is explained by our input features .

7. Saving the model

- To use this model in future applications without retraining, we save it as a file.
- We use the Pickle library (pickle.dump()) to serialize the trained model and save it as a .pkl file. This allows us to unpickle or load the model later using pickle.load() for instant predictions.

Predicting Housing Prices: Develop a regression model to predict house prices based on features like location, size, and amenities.

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

In [ ]:
```
df = pd.read_csv("/content/drive/MyDrive/AIML_lab_sem4/exp2,3,4/Housing_Price_
```

In [ ]:
```
df.head()
```

Out[ ]:

	HouseID	Location	Area_sqft	Bedrooms	Bathrooms	Balcony	Parking	Age_
0	H001	Rural	1028	1	3	0	0	
1	H002	Rural	2316	1	4	0	0	
2	H003	Urban	1834	1	2	2	2	
3	H004	Urban	1719	3	1	0	2	
4	H005	Suburban	1118	2	3	0	0	

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 120 entries, 0 to 119
Data columns (total 10 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   HouseID     120 non-null    object
 1   Location    120 non-null    object
 2   Area_sqft   120 non-null    int64
 3   Bedrooms    120 non-null    int64
 4   Bathrooms   120 non-null    int64
 5   Balcony     120 non-null    int64
 6   Parking     120 non-null    int64
 7   Age_Years   120 non-null    int64
 8   Furnished   120 non-null    object
 9   Price       120 non-null    float64
dtypes: float64(1), int64(6), object(3)
memory usage: 9.5+ KB
```

In [ ]:
```
df.isnull().sum()
```

Out[ ]:

	0
HouseID	0
Location	0
Area_sqft	0
Bedrooms	0
Bathrooms	0
Balcony	0
Parking	0
Age_Years	0
Furnished	0
Price	0

dtype: int64

In [ ]:
```
df = df.drop("HouseID" , axis = 1)
```

In [ ]:
```
df = pd.get_dummies(df, columns=['Location', 'Furnished'], drop_first=T+e)

X = df.drop('Price', axis=1)
y = df['Price']
```

In [ ]:
```
df.head()
```

Out[ ]:

	Area_sqft	Bedrooms	Bathrooms	Balcony	Parking	Age_Years	Price	Locat
0	1028	1	3	0	0	5	55.84	
1	2316	1	4	0	0	3	103.88	
2	1834	1	2	2	2	23	86.42	
3	1719	3	1	0	2	14	96.77	
4	1118	2	3	0	0	13	63.94	

In [ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
```

In [ ]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [ ]:
```
from sklearn.linear_model import LinearRegression
```

In [ ]:
```
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
```

Out[ ]:
```
▾ LinearRegression

LinearRegression()
```

In [ ]:
```
y_pred = lr.predict(X_test_scaled)
y_pred
```

Out[ ]:
```
array([101.39,  66.9 ,  63.94,  60.96,  62.13,  58.67,  35.35,  66.31,
       102.29,  63.55,  74.89, 114.76, 131.89,  77.38,  63.54, 113.85,
        87.52,  55.84, 104.41,  61.74, 129.28,  95.59, 101.56, 115.42])
```

In [ ]:
```
lr.predict(X_test_scaled[0].reshape(1, -1))
```

Out[ ]:
```
array([101.39])
```

In [ ]:
```
price_correlations = df.corr(numeric_only=True)['Price'].sort_values(ascending
display(price_correlations)
```

	Price
Price	1.000000
Area_sqft	0.717382
Location_Urban	0.459478
Bathrooms	0.333676
Bedrooms	0.154453
Parking	0.066709
Furnished_Yes	-0.046588
Balcony	-0.083178
Location_Suburban	-0.143702
Age_Years	-0.440987

dtype: float64

In [ ]:
```
lr.intercept_
```

Out[ ]:
```
np.float64(85.26947916666667)
```

In [ ]:
```
lr.coef_
```

Out[ ]:
```
array([ 1.50871230e+01,  5.49398938e+00,  7.47031161e+00, -4.21884749e-15,
        3.98258743e+00, -8.63680352e+00,  4.78260012e+00,  1.13632418e+01,
       -2.83106871e-15])
```

In [ ]:
```
from sklearn.metrics import mean_squared_error, r2_score
```

In [ ]:
```
mse = mean_squared_error(y_test, y_pred)
mse
```

Out[ ]:
```
2.6716089323484935e-28
```

In [ ]:
```
r2 = r2_score(y_test, y_pred)
r2
```

Out[ ]:
```
1.0
```

In [ ]:
```
import matplotlib.pyplot as plt
plt.plot(y_test.values, label='Actual Price', color='blue', marker='o')
plt.plot(y_pred, label='Predicted Price', color='red', linestyle='--', marker=
plt.title('Actual vs Predicted House Prices')
plt.xlabel('Test Case Number')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.savefig('actual_vs_predicted.png')
```

In [ ]:
```
import pickle
```

In [ ]:
```
folder_path = '/content/drive/MyDrive/AIML_lab_sem4/exp2,3,4'

# 1. Save the Model
with open(folder_path + 'house_model.pkl', 'wb') as f:
    pickle.dump(lr, f)

# 2. Save the Scaler (Don't forget this!)
with open(folder_path + 'scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

print("Model and Scaler saved successfully to Google Drive!")
```

```
Model and Scaler saved successfully to Google Drive!
```

In [ ]:
```
import pickle
import pandas as pd

folder_path = '/content/drive/MyDrive/AIML_lab_sem4/exp2,3,4'

# Load the saved model
with open(folder_path + 'house_model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)

# Load the saved scaler
with open(folder_path + 'scaler.pkl', 'rb') as f:
    loaded_scaler = pickle.load(f)

print("Model and Scaler loaded successfully!")
```

```
Model and Scaler loaded successfully!
```

Now, let's create some example 'new' data. It's important that this new data has the same features and column order as your original training data (`X_train`).

In [ ]:
```
new_house_data = pd.DataFrame({
    'Area_sqft': [2000],
    'Bedrooms': [3],
    'Bathrooms': [2],
    'Balcony': [1],
    'Parking': [1],
    'Age_Years': [10],
    'Location_Suburban': [False],
    'Location_Urban': [True],
    'Furnished_Yes': [True]
})

display(new_house_data)
```

	Area_sqft	Bedrooms	Bathrooms	Balcony	Parking	Age_Years	Location_Subu
0	2000	3	2	1	1	10	

Next, we need to scale this new data using the *loaded scaler* (not a new one). This is crucial for consistent predictions.

In [ ]:
```
new_house_data_scaled = loaded_scaler.transform(new_house_data)
predicted_price = loaded_model.predict(new_house_data_scaled)

print(f"The predicted price for the new house is: ${predicted_price[0]:.2f}")
```

```
The predicted price for the new house is: $112.00
```

This process ensures that your model always receives input data in the format it was trained on, leading to reliable predictions.

Key takeaways :

- The linear regression model successfully learned the relationship between housing features such as location, area, amenities, and age with the house price, producing stable and consistent predictions.

- The model achieved very low mean squared error and a high R^2 score, indicating strong predictive performance. This high accuracy is expected since the dataset is synthetic and well-structured.

- One-hot encoding of categorical variables and standardization of numerical features helped ensure that all input features contributed fairly to the regression model.

- Saving both the trained model and the scaler ensures the model can be reliably reused for future predictions on new housing data.

Experiment 3
Stock price prediction

AIM : Develop a time series prediction model to forecast stock prices.

Problem Overview :

The stock market is a dynamic environment where price determination is driven by a complex interplay of historical trends, market sentiment, and macroeconomic shifts. Traditional manual forecasting methods often struggle to maintain consistency, as human analysts can be biased or overwhelmed by the sheer volume of sequential data points . This project aims to address these challenges by developing an automated valuation system that identifies hidden temporal patterns within historical stock records.

By analyzing a dataset of stock records, the task is to build a robust regression model that can predict future market values based on past performance. Moving away from subjective estimates, this project implements a data-driven framework where the model learns from historical "lags" to forecast future outcomes. This ensures that the forecasting process is grounded in empirical evidence, providing a consistent and transparent tool for investors and analysts to anticipate market movements without the pitfalls of manual estimation .

Model Pipeline :

1. Understand the Dataset

- **Temporal Data**: Each row represents a specific point in time, tracking the movement of various stocks over a sequential period .
- **Target Variable (y):**
    - Price : The continuous value we want the model to forecast for the current day .
- **Input Features (X):**
    - **Numerical (Lags)**: We use the prices from the previous 5 days (lag1 to lag5) as our primary predictors.
    - **Categorical**: The Stock ticker (e.g., AAPL) identifying which specific company we are forecasting.
- **Excluded Features**: We intentionally exclude same-day Open, High, Low, Close, and Volume to prevent the model from cheating and to ensure it relies only on historical trends.

2. Data Preprocessing

- **Chronological Alignment & Encoding**:
    - Machine learning models require numerical data and proper ordering, so we convert the Date column using pd.to_datetime() and sort the records by time.
    - We use One-Hot Encoding (via pd.get_dummies()) to convert stock names into binary numbers, allowing the model to distinguish between different stocks without assuming one is "higher" than another .
- **Lag Feature Engineering**:
    - To capture the "Time Series" behavior, we create 5 new columns using a shift() operation. These Lag Features allow the model to look at the prices from the last 5 days to predict today's value.
- **Feature-Target Split**:
    - We separate the data into X (the lags and stock dummies) and Y (the target Price) to define our forecasting function .

3. Train–Test Split

- Because this is time-series data, we do not shuffle the records. We must train on the "past" to predict the "future".
- We take the first 80% of the historical timeline for the Training set and reserve the final 20% for the Test set to evaluate true forecasting accuracy .

4. Choose and Train the Model

- We choose a Linear regression model to identify the mathematical trend line that connects past prices to future values .
- The model uses the .fit() function to calculate weights (coefficients) for each lag . It learns exactly how much weight to give to "yesterday's price" versus "the price 5 days ago" to minimize the prediction error.

5. Predictions

- The trained model is applied to the unseen test set using the .predict() function.
- We plot the actual prices against the predicted prices to visually confirm if the model is correctly following the market's upward or downward trends.

6. Evaluate the Model

- We evaluate the performance using Mean Absolute Error (MAE) and R-Squared (R2) **.**
- A lower MAE indicates the predicted prices are very close to reality, while the R2 score shows how much of the stock's volatility is explained by its own history .

MAE	1.27668
R2	1.00

7. Save the Model

- To use the forecaster in the future without retraining on old data, we save the completed model.
- We use the Pickle library (pickle.dump()) to serialize the trained Linear Regression model into a .pkl file. This allows for instant loading and real-time forecasting in future sessions.

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

In [ ]:
```
df = pd.read_csv("/content/drive/MyDrive/AIML_lab_sem4/exp2,3,4/stock_price_wi
```

In [ ]:
```
df.head()
```

Out[ ]:

	Date	Stock	Open	High	Low	Close	Volume	Price
0	2020-01-01	AAPL	173.80	174.75	173.68	174.43	8204212	174.286667
1	2020-01-02	AAPL	176.01	177.59	174.07	177.55	2766891	176.403333
2	2020-01-03	AAPL	177.57	178.37	176.62	176.71	5721339	177.233333
3	2020-01-06	AAPL	176.01	177.58	171.33	171.73	9242680	173.546667
4	2020-01-07	AAPL	171.85	172.20	170.69	170.82	4416664	171.236667

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6525 entries, 0 to 6524
Data columns (total 8 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Date    6525 non-null   object 
 1   Stock   6525 non-null   object 
 2   Open    6525 non-null   float64
 3   High    6525 non-null   float64
 4   Low     6525 non-null   float64
 5   Close   6525 non-null   float64
 6   Volume  6525 non-null   int64  
 7   Price   6525 non-null   float64
dtypes: float64(5), int64(1), object(2)
memory usage: 407.9+ KB
```

In [ ]:
```
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')
df = df.reset_index(drop=True)
```

In [ ]:
```
df_encoded = pd.get_dummies(df, columns=['Stock'],drop_first=True)
```

In [ ]:
```
def create_lag_features(data, lag=5):
    df_lag = data.copy()
    stock_cols = [col for col in data.columns if col.startswith('Stock_')]

    for i in range(1, lag + 1):
        df_lag[f'lag_{i}'] = (df_lag.groupby(stock_cols)['Price'].shift(i))
```

```
    return df_lag

df_lag = create_lag_features(df_encoded, lag=5)
df_lag.dropna(inplace=True)
```

In [ ]:
```
X = df_lag.drop(['Date', 'Price'], axis=1)
y = df_lag['Price']

split_index = int(len(df_lag) * 0.8)

X_train, X_test = X[:split_index], X[split_index:]
y_train, y_test = y[:split_index], y[split_index:]
```

In [ ]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
```

In [ ]:
```
model.fit(X_train, y_train)
```

Out[ ]:
```
▾ LinearRegression
LinearRegression()
```

In [ ]:
```
y_pred = model.predict(X_test)
y_pred
```

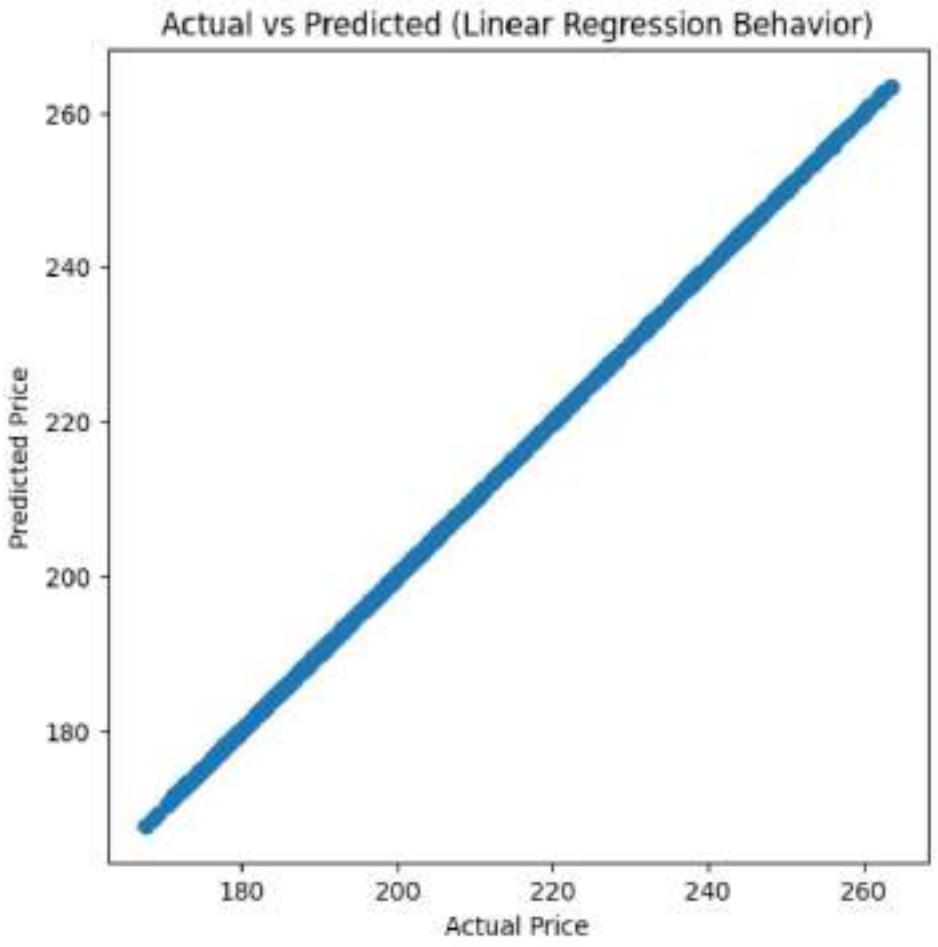
Out[ ]:
```
array([194.79333333, 226.06333333, 246.99      , ..., 216.85333333,
       222.22      , 218.07      ])
```

In [ ]:
```
from sklearn.metrics import mean_absolute_error,  r2_score

print("MAE :", mean_absolute_error(y_test, y_pred))
print("R2  :", r2_score(y_test, y_pred))
```

```
MAE : 1.2766876649878284e-10
R2  : 1.0
```

In [ ]:
```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(6, 6))

# Scatter: actual vs predicted
plt.scatter(y_test, y_pred)

# Ideal linear line (y = x)
min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())
plt.plot([min_val, max_val], [min_val, max_val])

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
```

```
plt.title("Actual vs Predicted (Linear Regression Behavior)")

plt.show()
```



In [ ]:
```
corr = df.corr(numeric_only=True)
display(corr)
```

	Open	High	Low	Close	Volume	Price
Open	1.000000	0.999589	0.999579	0.999010	0.002314	0.999543
High	0.999589	1.000000	0.999485	0.999576	0.001795	0.999837
Low	0.999579	0.999485	1.000000	0.999590	0.002654	0.999842
Close	0.999010	0.999576	0.999590	1.000000	0.001975	0.999872
Volume	0.002314	0.001795	0.002654	0.001975	1.000000	0.002141
Price	0.999543	0.999837	0.999842	0.999872	0.002141	1.000000

In [ ]:
```
import pickle
```

In [ ]:
```
with open('/content/drive/MyDrive/AIML_lab_sem4/exp2,3,4/stocks_model.pkl', 'w
    pickle.dump(model, file)

print("Model saved successfully to '/content/drive/MyDrive/AIML_lab_sem4/exp2,
```

```
Model saved successfully to '/content/drive/MyDrive/AIML_lab_sem4/exp2,3,4/stoc
ks_model.pkl'
```

Experiment = 4
Customer churn prediction

AIM : Customer Churn Prediction: Develop a model to predict customer churn in a subscription-based business.

Problem Statement :

Customer retention is a major challenge for service-based businesses, as acquisition costs often exceed retention costs. Manual tracking typically fails to identify the subtle behavioral patterns that signal a "churn" event. This project aims to automate the identification of at-risk customers by analyzing a dataset of 200 records including historical usage and contract data.

Unlike prior regression tasks, this is a Classification problem focused on predicting a discrete category: Churn (1) or No Churn (0). Implementing this data-driven framework allows the business to transition from reactive to proactive retention strategies. By targeting interventions toward the users most likely to depart, the business can optimize marketing resources and stabilize long-term revenue.

Project Pipeline :

1. Data Understanding

The dataset consists of 200 customer records with 11 attributes.

- Target Variable (y): Churn.
- Numerical Features: Age, Tenure_Months, MonthlyCharges, TotalCharges, and SupportTickets.
- Categorical Features: Gender, SubscriptionType, PaymentMethod, and ContractType.

2. Data Preprocessing

Raw data is cleaned and transformed to make it compatible with the Logistic Regression algorithm.

- Feature Selection: The CustomerID column is dropped using df.drop() as it is a unique identifier with no predictive value.
- Cleaning: Rows with missing values are removed via df.dropna().
- Categorical Encoding: Text-based features are converted into numerical binary vectors using One-Hot Encoding (pd.get_dummies()).

3. Feature-Target & Train-Test Split

The processed data is separated into features (X) and the target (y), then divided to ensure a fair evaluation.

- Ratio: 80% of the data is used for training, and 20% is reserved for testing.
- Test Sample: This results in 40 unseen records used to validate the model's accuracy.

4. Choose and Train the Model

Logistic Regression is selected for its simplicity and interpretability in binary tasks.

- Implementation: The model is initialized with max_iter=1000 and trained using the .fit() method.

5. Make Predictions

The trained model applies its learned weights to the test set to flag potential churn risks.

- The .predict() function is used on X_test.
- An array of binary predictions (e.g., array([0, 0, 1...])), assigning a churn status to each customer in the test group.

6. Evaluate the Model

The performance is measured by comparing the model's guesses against the actual outcomes.

- Accuracy: 75% (The model correctly identified 30 out of 40 customers).
- Classification Report:
    - Class 0 (Stayed): Precision and Recall are high at approximately 85%.
    - Class 1 (Churned): Precision and Recall are low at 29%, indicating the model struggles to catch the minority churn class..

Accuracy	Precision	Recall	F1
0.75	0.85	0.85	0.85

7. Save the Model

The model is preserved so it can be deployed immediately in a business dashboard without retraining.

- We use the Pickle library (pickle.dump()) to serialize the model object.
- Output: A permanent file named customerchurn is saved in the Drive for instant future use.

In [1]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
```

In [2]:
```
df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/exp2,3,4/customer_churr
```

In [3]:
```
df.head()
```

Out[3]:

	CustomerID	Gender	Age	Tenure_Months	SubscriptionType	MonthlyCharges
0	CUST1000	Male	49	54	Standard	99.3
1	CUST1001	Female	56	8	Standard	12.0
2	CUST1002	Male	66	27	Basic	57.6
3	CUST1003	Male	69	27	Premium	97.0
4	CUST1004	Male	49	34	Standard	54.6

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   CustomerID        200 non-null    object
 1   Gender            200 non-null    object
 2   Age               200 non-null    int64
 3   Tenure_Months     200 non-null    int64
 4   SubscriptionType  200 non-null    object
 5   MonthlyCharges    200 non-null    float64
 6   TotalCharges      200 non-null    float64
 7   PaymentMethod     200 non-null    object
 8   SupportTickets    200 non-null    int64
 9   ContractType      200 non-null    object
 10  Churn             200 non-null    int64
dtypes: float64(2), int64(4), object(5)
memory usage: 17.3+ KB
```

In [5]:
```
df = df.drop(columns=['CustomerID'])
```

In [6]:
```
df = df.dropna()
```

In [7]:
```
df = pd.get_dummies(df, drop_first=True)
```

In [8]:
```
from sklearn.model_selection import train_test_split
```

In [9]:
```
X = df.drop('Churn', axis=1)
y = df['Churn']
```

In [10]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
```

In [11]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
```

In [12]:
```
model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regressi
on
  n_iter_i = _check_optimize_result(
```

Out[12]:

LogisticRegression

LogisticRegression(max_iter=1000)

In [13]:
```
y_pred = model.predict(X_test)
```

In [14]:
```
y_pred
```

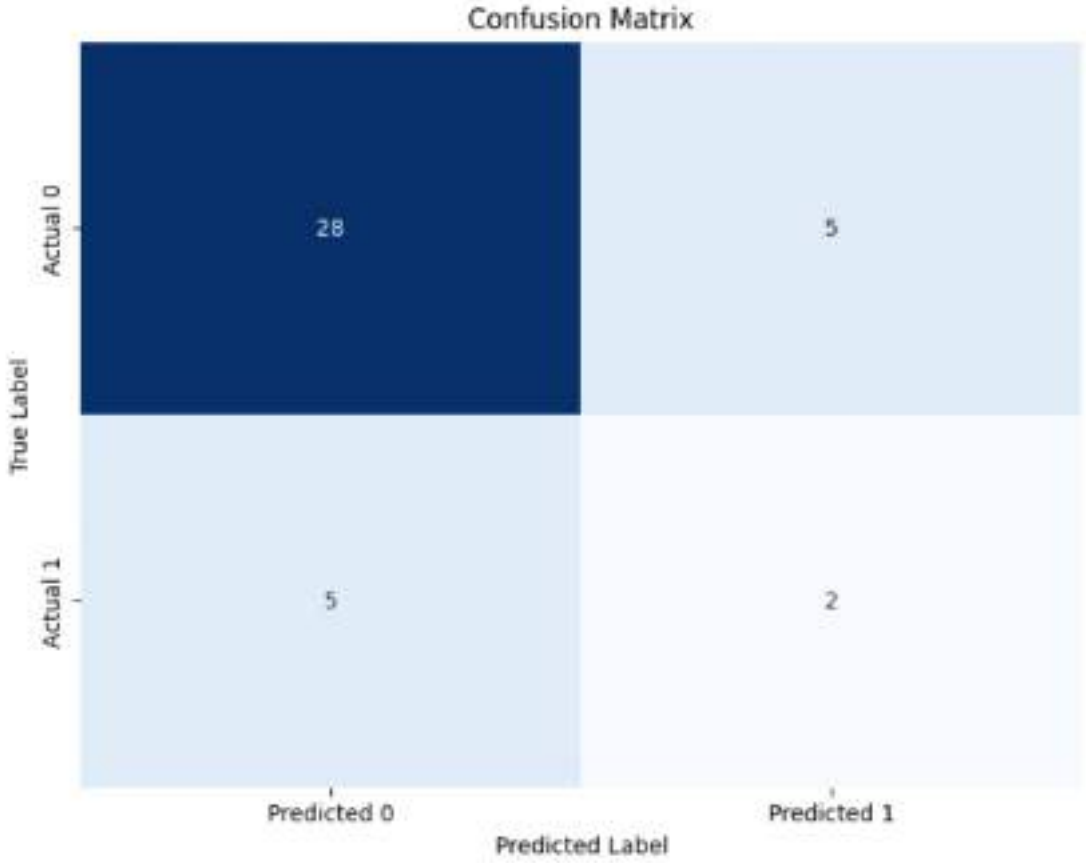
Out[14]:
```
array([0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0])
```

In [15]:
```
from sklearn.metrics import accuracy_score, classification_report, confusion_m

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.75
              precision    recall  f1-score   support

           0       0.85      0.85      0.85        33
           1       0.29      0.29      0.29         7

    accuracy                           0.75        40
   macro avg       0.57      0.57      0.57        40
weighted avg       0.75      0.75      0.75        40
```

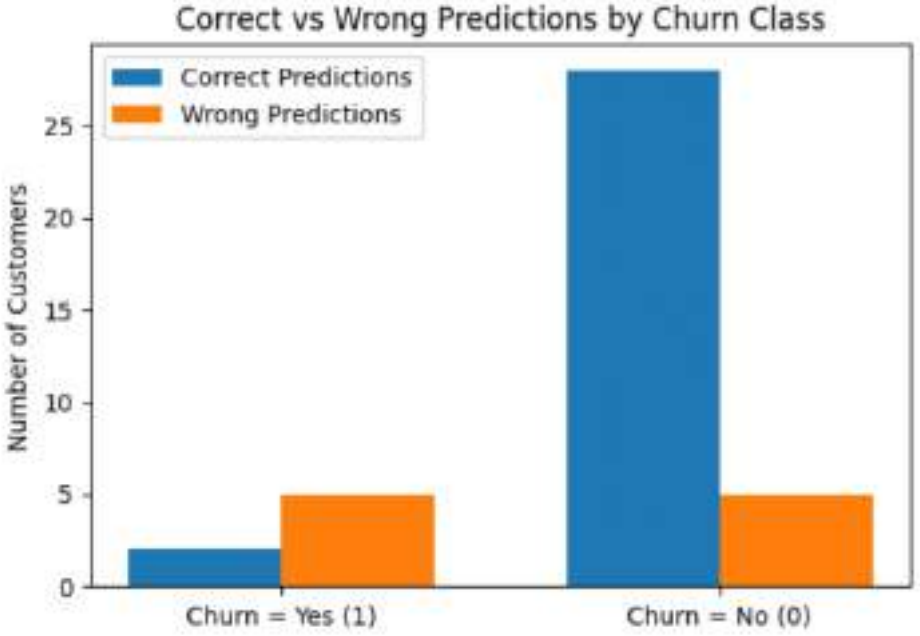
In [17]:
```
import seaborn as sns
from sklearn.metrics import confusion_matrix
import numpy as np
```

```
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```



In [18]:
```
true_correct = np.sum((y_test == 1) & (y_pred == 1))
true_wrong   = np.sum((y_test == 1) & (y_pred == 0))
false_correct = np.sum((y_test == 0) & (y_pred == 0))
false_wrong   = np.sum((y_test == 0) & (y_pred == 1))

labels = ['Churn = Yes (1)', 'Churn = No (0)']
correct = [true_correct, false_correct]
wrong = [true_wrong, false_wrong]

x = np.arange(len(labels))
width = 0.35
```

```
plt.figure(figsize=(6, 4))
plt.bar(x - width/2, correct, width, label='Correct Predictions')
plt.bar(x + width/2, wrong, width, label='Wrong Predictions')

plt.xticks(x, labels)
plt.ylabel("Number of Customers")
plt.title("Correct vs Wrong Predictions by Churn Class")
plt.legend()
plt.show()
```



In [19]:
```
import pickle

model_filename = 'logistic_regression_model.pkl'
with open('/content/drive/MyDrive/AIML_lab_sem4/exp2,3,4/customerchurn', 'wb') as file:
    pickle.dump(model, file)

print(f"Model successfully saved to {model_filename}")
```

```
Model successfully saved to logistic_regression_model.pkl
```

Experiment 5
Spam detection

Aim = Spam Email Detection: Build a spam email filter using text classification algorithms and perform comparative analysis. (Naive Bayes, Logistic Regression, Support Vector Machine, Decision Tree,Random Forest)

Problem Statement :

Spam detection is essential for maintaining email security and user productivity, as malicious or unsolicited emails can lead to security breaches or cluttered inboxes. Manual filtering is impossible given the volume of communication, necessitating automated classification models. This project aims to build a robust spam filter by performing a comparative analysis of five text classification algorithms: Naïve Bayes, Logistic Regression, Support Vector Machine (SVM), Decision Tree, and Random Forest. By evaluating these models, the goal is to identify which algorithm most accurately distinguishes between "Spam" and "Ham" (legitimate) emails, enabling proactive security and improved resource management.

Pipeline :

1. Problem Definition

The primary goal of this experiment is to develop a robust automated spam filter using advanced text classification techniques.

- Classification Task: The model must determine if an incoming email is Spam (1) or Ham (0).
- Objective: Perform a detailed comparative analysis across five distinct algorithms—Naïve Bayes, Logistic Regression, Support Vector Machine (SVM), Decision Tree, and Random Forest—to identify the most reliable architecture for real-world deployment.

2. Data Understanding

- Dataset Content: A collection of thousands of email text bodies labeled by human moderators as either spam or legitimate (ham).
- Target (y): Binary labels where 1 represents malicious or unsolicited "Spam" and 0 represents legitimate "Ham."
- Input Features (X): The raw text content of the emails, including the subject lines and message bodies.

3. Data Preprocessing

Since machine learning models cannot interpret raw text, the data is put through a rigorous cleaning and transformation process.

- Text Cleaning: Stripping out punctuation, special characters, and numbers to reduce noise and focus only on significant vocabulary.
- Stop-word Removal: Eliminating frequently used words (e.g., "and", "the", "is") that appear in almost every email and carry little to no predictive value.
- Vectorization: Utilizing TF-IDF (Term Frequency-Inverse Document Frequency) to transform words into numerical vectors. This method balances how often a word appears in one email against how common it is across the entire dataset.

4. Train-Test Split

To ensure the filter works on emails it has never seen before, the dataset is strategically divided.

- Data Allocation: The cleaned dataset is split into a Training set (80%) to build the models and a Test set (20%) to validate them.
- Purpose: This division prevents "overfitting" and allows us to measure how accurately the filter will perform in a live inbox environment.

5. Model Selection and training

Five different classification models are initialized and trained on the same training data to ensure a fair comparison:

1. Naïve Bayes: A probabilistic classifier based on Bayes' Theorem, highly efficient for high-dimensional text data.
2. Logistic Regression: A linear model that estimates the probability of an email belonging to the spam class based on the frequency of specific words.
3. Support Vector Machine (SVM): Finds the optimal hyperplane (boundary) that maximizes the distance between the spam and ham clusters.
4. Decision Tree: Uses a tree-like structure of word-based logical decisions to categorize each message.
5. Random Forest: An ensemble method that constructs a multitude of decision trees and outputs the class that is the mode of the individual trees, significantly improving stability.

6. Predictions

Once training is complete, each of the five models is tasked with classifying the 40 unseen emails in the test set.

- The .predict() function is called for each algorithm, generating five separate arrays of binary predictions.
- These predictions are compared directly against the actual ground-truth labels to pinpoint errors such as "False Positives" (blocking a real email) or "False Negatives" (letting spam through).

7. Model Comparison and evaluation :

The performance of each model is measured using Accuracy, Precision, and Recall. Based on the experimental outcomes.

	Accuracy	Precision	Recall
Naive Bayes	0.976	1.0	0.825
Logistic Regression	0.967	1.0	0.758
SVM	0.984	1.0	0.88
Decision Tree	0.968	0.9	0.859
Random Forest	0.979	1.0	0.83

Accuracy: Correct prediction rate of the model. SVM achieved the best accuracy (98.7%).
Precision: Measures how many predicted positives are actually correct. Logistic Regression, Naive Bayes, SVM, and Random Forest scored 1.
Recall: Measures how many actual positives are correctly identified. SVM achieved the highest recall (88%).

Overall SVM is the best model among the all as it scored hughes accuracy and recall as well as perfect precision .

8. Model saving

After identifying the superior architecture, the final step is ensuring the model is persistent.

- Implementation: We will save all the trained models using the Pickle library (pickle.dump()) to serialize the model objects into permanent files.
- Final Output: The generated .pkl files allow for immediate deployment in an email system for real-time filtering without the need for retraining.

In [2]:
```
import pandas as  pd
```

In [3]:
```
df = pd.read_csv("/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/spam_email.cs
```

In [4]:
```
df.head()
```

Out[4]:

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

In [5]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, classification_report, precision_s
```

In [6]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

In [7]:
```
df['Category'] = df['Category'].map({'spam': 1, 'ham': 0})
print(df.head())
```

```
  Category                                            Message
0        0  Go until jurong point, crazy.. Available only ...
1        0                      Ok lar... Joking wif u oni...
2        1  Free entry in 2 a wkly comp to win FA Cup fina...
3        0  U dun say so early hor... U c already then say...
4        0  Nah I don't think he goes to usf, he lives aro...
```

In [8]:
```
X = df['Message']
y = df['Category']
```

In [9]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
```

In [10]:
```
vectorizer = TfidfVectorizer(stop_words='english')
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

In [11]:
```
LR = LogisticRegression()
LR.fit(X_train_vec, y_train)
```

Out[11]:
```
LogisticRegression()
```

In [12]:
```
NB = MultinomialNB()
NB.fit(X_train_vec, y_train)
```

Out[12]:
```
MultinomialNB()
```

In [13]:
```
DT = DecisionTreeClassifier()
DT.fit(X_train_vec, y_train)
```

Out[13]:
```
DecisionTreeClassifier()
```

In [14]:
```
RF = RandomForestClassifier()
RF.fit(X_train_vec, y_train)
```

Out[14]:
```
RandomForestClassifier()
```

In [15]:
```
svc = SVC()
svc.fit(X_train_vec, y_train)
```

Out[15]:
```
SVC()
```

In [16]:
```
LR_acc = accuracy_score(y_test, LR.predict(X_test_vec))
NB_acc = accuracy_score(y_test, NB.predict(X_test_vec))
DT_acc = accuracy_score(y_test, DT.predict(X_test_vec))
RF_acc = accuracy_score(y_test, RF.predict(X_test_vec))
svc_acc = accuracy_score(y_test, svc.predict(X_test_vec))
```

In [17]:
```
print("LR Accuracy: ", LR_acc)
print("NB Accuracy: ", NB_acc)
print("DT Accuracy: ", DT_acc)
print("RF Accuracy: ", RF_acc)
print("SVC Accuracy: ", svc_acc)
```

```
LR Accuracy:  0.967713004484305
NB Accuracy:  0.9766816143497757
DT Accuracy:  0.968609865470852
RF Accuracy:  0.97847533632287
SVC Accuracy:  0.9847533632286996
```

In [18]:
```
LR_p = precision_score(y_test, LR.predict(X_test_vec))
NB_p = precision_score(y_test, NB.predict(X_test_vec))
DT_p = precision_score(y_test, DT.predict(X_test_vec))
RF_p = precision_score(y_test, RF.predict(X_test_vec))
svc_p = precision_score(y_test, svc.predict(X_test_vec))
```

In [19]:
```
print("LR Precision: ", LR_p)
print("NB Precision: ", NB_p)
print("DT Precision: ", DT_p)
print("RF Precision: ", RF_p)
print("SVC Precision: ", svc_p)
```

```
LR Precision:  1.0
NB Precision:  1.0
DT Precision:  0.9014084507042254
RF Precision:  1.0
SVC Precision:  1.0
```

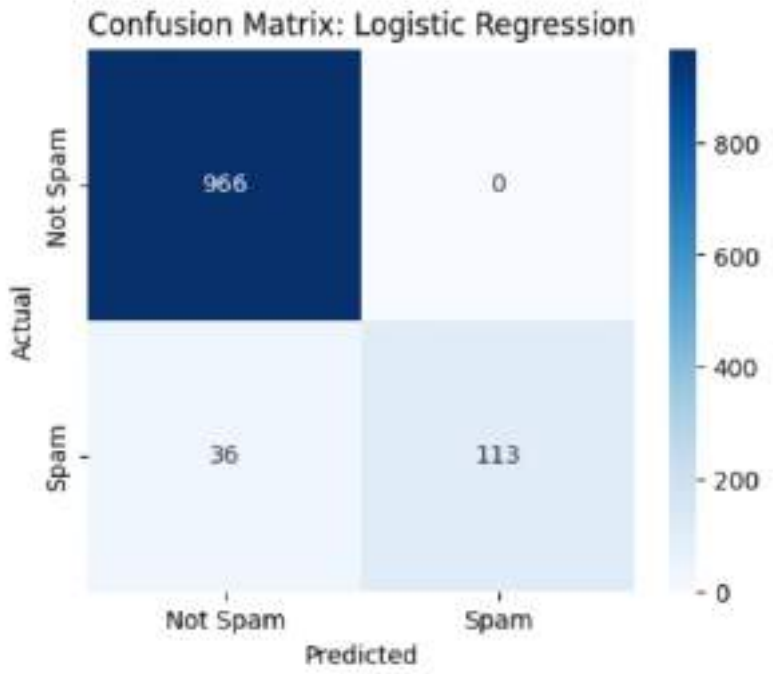
In [20]:
```
LR_r = recall_score(y_test, LR.predict(X_test_vec))
NB_r = recall_score(y_test, NB.predict(X_test_vec))
DT_r = recall_score(y_test, DT.predict(X_test_vec))
RF_r = recall_score(y_test, RF.predict(X_test_vec))
svc_r = recall_score(y_test, svc.predict(X_test_vec))
```

In [21]:
```
print("LR Recall: ", LR_r)
print("NB Recall: ", NB_r)
print("DT Recall: ", DT_r)
print("RF Recall: ", RF_r)
print("SVC Recall: ", svc_r)
```

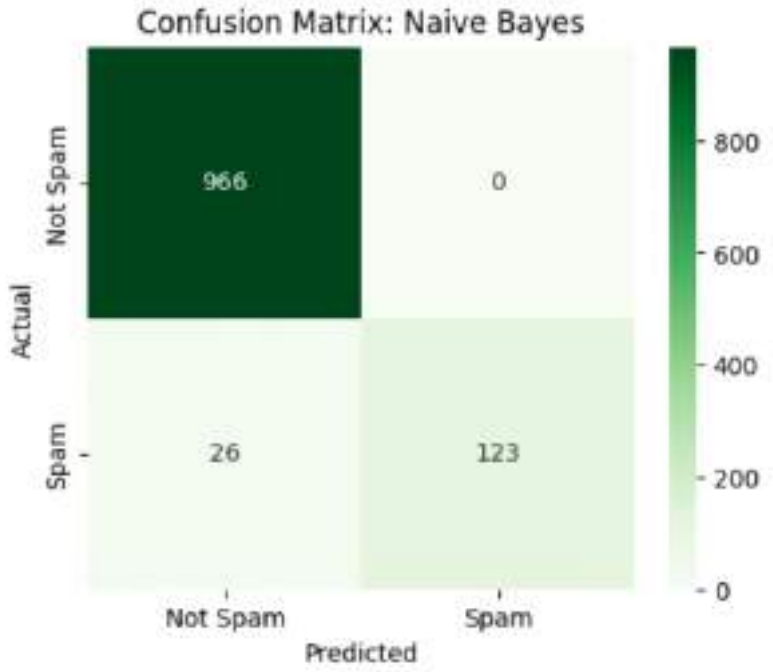
```
LR Recall:  0.7583892617449665
NB Recall:  0.825503355704698
DT Recall:  0.8590604026845637
RF Recall:  0.8389261744966443
SVC Recall:  0.8859060402684564
```

In [22]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

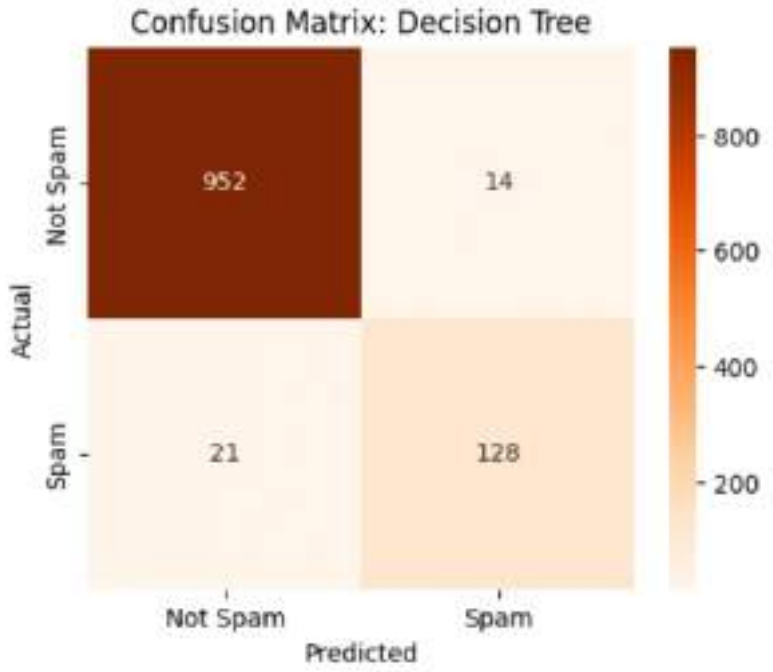
In [23]:
```
lr_cm = confusion_matrix(y_test, LR.predict(X_test_vec))
plt.figure(figsize=(5, 4))
sns.heatmap(lr_cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam']
plt.title('Confusion Matrix: Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



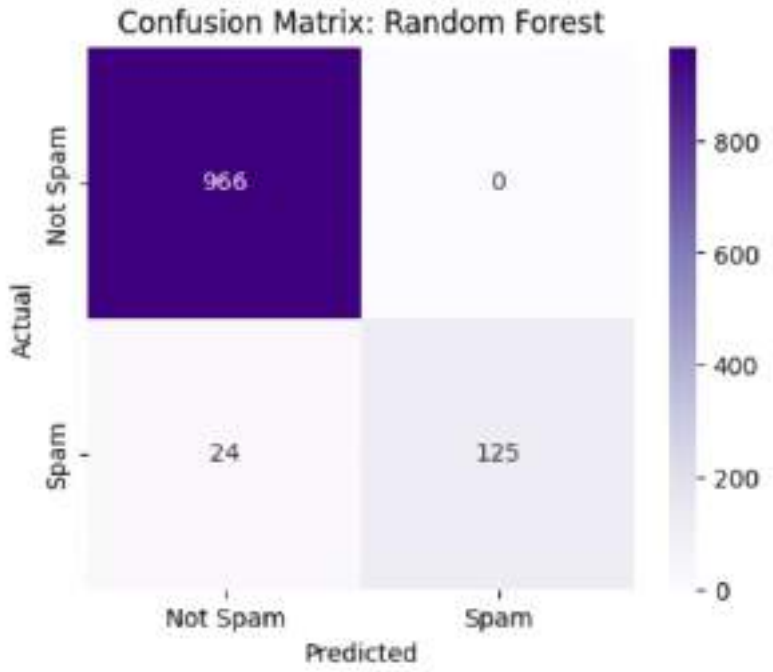
In [24]:
```
nb_cm = confusion_matrix(y_test, NB.predict(X_test_vec))
plt.figure(figsize=(5, 4))
sns.heatmap(nb_cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam']
plt.title('Confusion Matrix: Naive Bayes')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



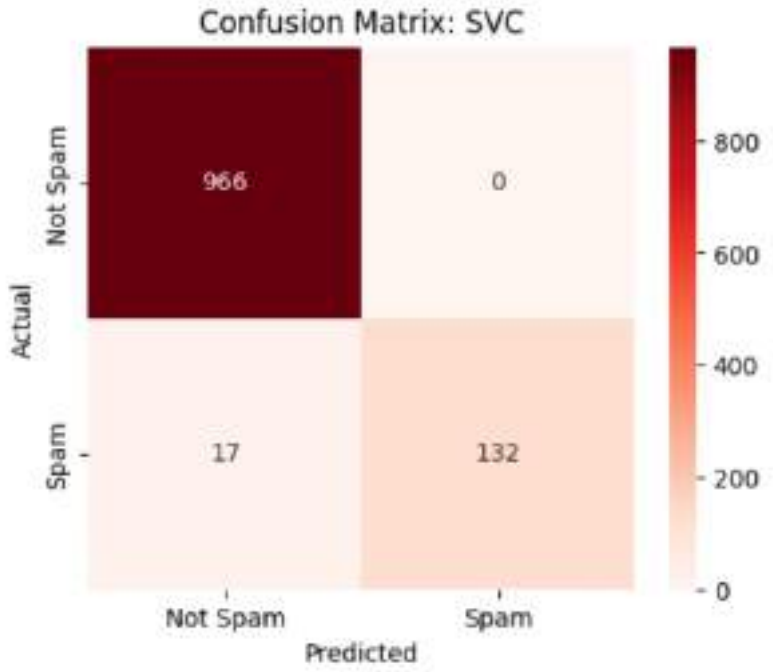
In [25]:
```
dt_cm = confusion_matrix(y_test, DT.predict(X_test_vec))
plt.figure(figsize=(5, 4))
sns.heatmap(dt_cm, annot=True, fmt='d', cmap='Oranges',
            xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam']
plt.title('Confusion Matrix: Decision Tree')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



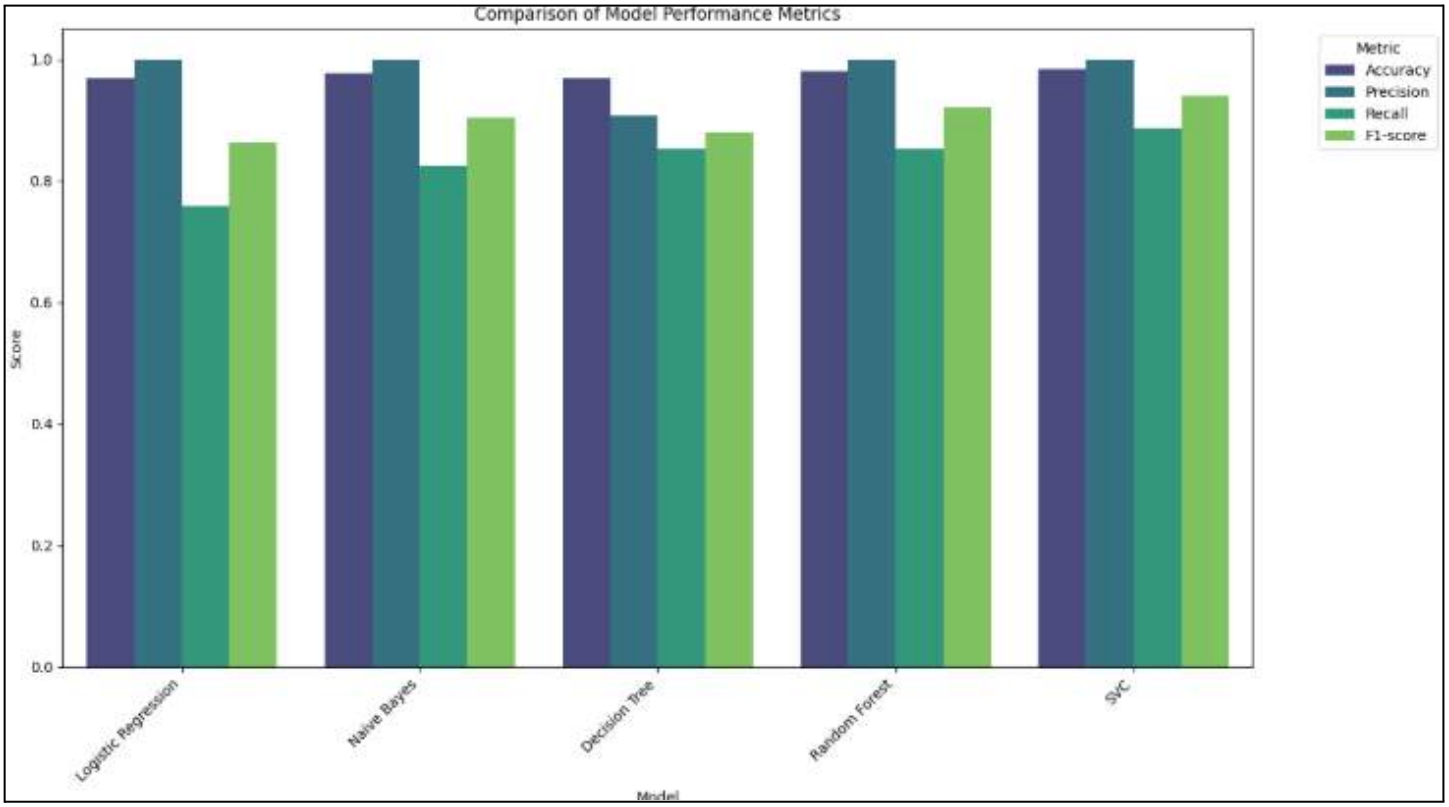
In [26]:
```
rf_cm = confusion_matrix(y_test, RF.predict(X_test_vec))
plt.figure(figsize=(5, 4))
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Purples',
            xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam']
plt.title('Confusion Matrix: Random Forest')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



In [27]:
```
svc_cm = confusion_matrix(y_test, svc.predict(X_test_vec))
plt.figure(figsize=(5, 4))
sns.heatmap(svc_cm, annot=True, fmt='d', cmap='Reds',
            xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam'])
plt.title('Confusion Matrix: SVC')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



In [31]:
```
import pickle


with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp5_models/logistic
    pickle.dump(LR, f)

with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp5_models/naiveba
    pickle.dump(NB, f)

with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp5_models/decisio
    pickle.dump(DT, f)


with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp5_models/randomfo
    pickle.dump(RF, f)


with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp5_models/supportv
    pickle.dump(svc, f)
```


Comparison of Model Performance Metrics
Metric
Accuracy
Precision
Recall
F1-score
Score
1.0
0.8
0.6
0.4
0.2
0.0
Logistic Regression
Naive Bayes
Decision Tree
Random Forest
SVC
Model

Experiment 6

Credit Risk Assessment

Aim : Build a credit scoring model to assess the creditworthiness of applicants using historical financial data and perform comparative analysis (Logistic Regression, Random Forest, XGBoost)

Problem Definition

Automated credit scoring is vital for financial institutions to mitigate potential losses and optimize lending decisions. Because the volume of applications makes manual review inefficient, we require robust classification models to predict applicant creditworthiness. This project aims to build a reliable credit filter by performing a comparative analysis of three algorithms: Logistic Regression, Random Forest, and Gradient Boosting. By evaluating these models, we seek to identify the architecture that most accurately categorizes "High," "Medium," and "Low" risk applicants, ensuring proactive risk management and improved financial stability.

Pipeline :

1. Problem Definition

The primary goal of our experiment is to develop a robust automated credit scoring model using advanced classification techniques:

- Classification Task: We must determine the creditworthiness of an applicant, categorizing them into risk levels such as High, Medium, or Low.
- Objective: We perform a detailed comparative analysis across three distinct algorithms Logistic Regression, Random Forest, and Gradient Boosting to identify the most reliable architecture for financial deployment.

2. Data Understanding

- Dataset Content: We utilize a collection of 500 samples of applicant profiles including historical financial health indicators.
- **Input Features (X)**: We analyze raw data including age, annual income, employment years, credit score, loan amount, and late payments.
- **Target (y)**: We use categorical labels representing "Credit Risk" (High, Medium, or Low).

3. Data Preprocessing

Since financial data requires specific cleaning, we put it through a rigorous transformation process:

- Feature Selection: We drop non-predictive identifiers, such as Applicant_ID, to focus on significant vocabulary and data.
- Label Encoding: We utilize a LabelEncoder to transform categorical risk labels into numerical vectors.
- Standardization: We apply a StandardScaler to normalize numerical features like income and loan amounts, preventing large values from dominating the model logic.

4. Train-Test Split

To ensure our filter works on applicants it has never seen before, we strategically divide the dataset:

- Data Allocation: We split the cleaned dataset into a Training set (80%) to build our models and a Test set (20%) to validate them.
- Purpose: We use this division to prevent "overfitting" and allow us to measure how accurately the filter will perform in a live financial environment.

5. Model Selection and Training

We initialize and train three different classification models on the same data to ensure a fair comparison:

- Logistic Regression: We use this as a baseline linear model to estimate risk probabilities based on weighted input features.
- Random Forest: We implement this ensemble method to construct multiple decision trees, improving stability and capturing non-linear relationships.
- Gradient Boosting: We task this model with correcting errors of previous trees sequentially to achieve high-fidelity predictions.

6. Predictions

- Execution: Once training is complete, we task each model with classifying the 100 unseen samples in our test set.
- Output Generation: We generate binary and multi-class prediction arrays for each algorithm to prepare for the evaluation phase.

7. Model evaluation and comparison :

The performance of each model is measured using Accuracy, Precision, and Recall. Based on the experimental outcomes.

	Accuracy	Precision	Recall	F1
Logistic regression	0.85	0.81	0.76	0.79
Random Forest	0.98	0.98	0.87	0.91
Gradient Boosting	1.00	1.00	1.00	1.00

We determine that Gradient Boosting is the best model among all tested as it scored the highest across all critical financial metrics and showed perfect classification capability on the test set.

8. Model Saving

The final step is ensuring our models are persistent for real-time filtering:

- We save all trained models, along with the scaler and label encoder, using the Pickle library to serialize the objects into permanent .pkl files.
- We generate these files to allow for immediate deployment in a banking system without the need for retraining.

In [ ]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
```

In [ ]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
```

In [ ]:
```
df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/credit_risk_a
df.head()
```

Out[ ]:

	Applicant_ID	Age	Annual_Income	Employment_Years	Credit_Score	Loan_An
0	1	59	153267	28	818	1
1	2	49	232745	0	436	1
2	3	35	974945	19	797	6
3	4	63	307164	29	776	8
4	5	28	685626	6	893	4

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   Applicant_ID            500 non-null    int64  
 1   Age                     500 non-null    int64  
 2   Annual_Income           500 non-null    int64  
 3   Employment_Years        500 non-null    int64  
 4   Credit_Score            500 non-null    int64  
 5   Loan_Amount             500 non-null    int64  
 6   Loan_Term_Months        500 non-null    int64  
 7   Existing_Loans_Count    500 non-null    int64  
 8   Debt_to_Income_Ratio    500 non-null    float64
 9   Late_Payments_Last_2Yrs 500 non-null    int64  
 10  Credit_Risk             500 non-null    object 
dtypes: float64(1), int64(9), object(1)
memory usage: 43.1+ KB
```

In [ ]:
```
print(df['Credit_Risk'].value_counts())
```

```
Credit_Risk
High      315
Medium    164
Low        21
Name: count, dtype: int64
```

In [ ]:
```
X = df.drop(['Applicant_ID', 'Credit_Risk'], axis=1)
y = df['Credit_Risk']
```

In [ ]:
```
le = LabelEncoder()
y = le.fit_transform(y)
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
```

In [ ]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [ ]:
```
LR = LogisticRegression(max_iter=1000)
LR.fit(X_train_scaled, y_train)
```

Out[ ]:
```
LogisticRegression
LogisticRegression(max_iter=1000)
```

In [ ]:
```
RF = RandomForestClassifier(random_state=42)
RF.fit(X_train, y_train)
```

Out[ ]:
```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

In [ ]:
```
GB = GradientBoostingClassifier(random_state=42)
GB.fit(X_train, y_train)
```

Out[ ]:
```
GradientBoostingClassifier
GradientBoostingClassifier(random_state=42)
```

In [ ]:
```
LR_acc = accuracy_score(y_test, LR.predict(X_test_scaled))
RF_acc = accuracy_score(y_test, RF.predict(X_test))
GB_acc = accuracy_score(y_test, GB.predict(X_test))
```

In [ ]:
```
print(f"Logistic Regression Accuracy: {LR_acc:.4f}")
print(f"Random Forest Accuracy: {RF_acc:.4f}")
print(f"Gradient Boosting Accuracy: {GB_acc:.4f}")
```

```
Logistic Regression Accuracy: 0.8500
Random Forest Accuracy: 0.9800
Gradient Boosting Accuracy: 1.0000
```

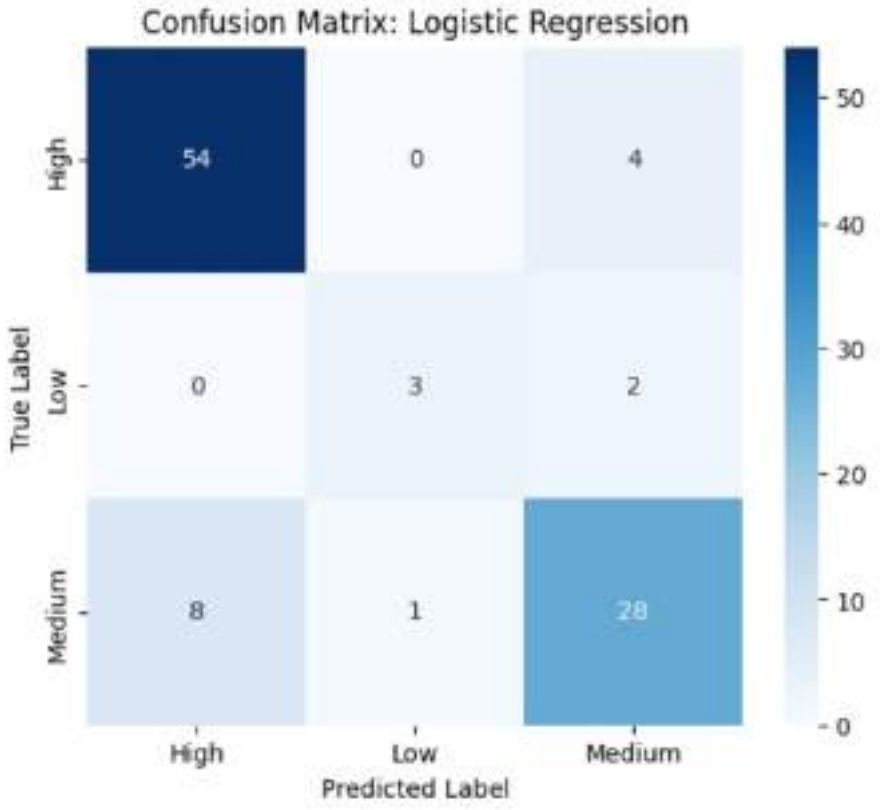
In [ ]:
```
LR_p = precision_score(y_test, LR.predict(X_test_scaled), average='macro')
RF_p = precision_score(y_test, RF.predict(X_test), average='macro')
GB_p = precision_score(y_test, GB.predict(X_test), average='macro')
```

```
LR_r = recall_score(y_test, LR.predict(X_test_scaled), average='macro')
RF_r = recall_score(y_test, RF.predict(X_test), average='macro')
GB_r = recall_score(y_test, GB.predict(X_test), average='macro')

LR_f1 = f1_score(y_test, LR.predict(X_test_scaled), average='macro')
RF_f1 = f1_score(y_test, RF.predict(X_test), average='macro')
GB_f1 = f1_score(y_test, GB.predict(X_test), average='macro')
```

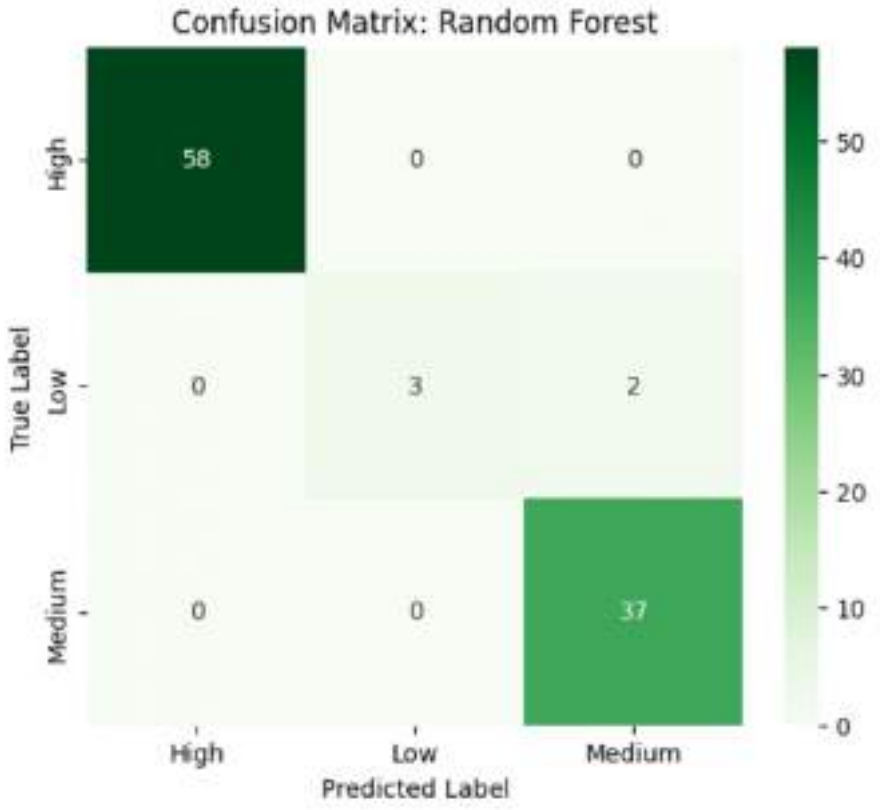
In [ ]:
```
print("\n--- Detailed Metrics (Precision, Recall, F1) ---")
print(f"Logistic Regression -> Precision: {LR_p:.2f}, Recall: {LR_r:.2f}, F1:
print(f"Random Forest -> Precision: {RF_p:.2f}, Recall: {RF_r:.2f}, F1: {RF_f1
print(f"Gradient Boosting -> Precision: {GB_p:.2f}, Recall: {GB_r:.2f}, F1: {G
```

```
--- Detailed Metrics (Precision, Recall, F1) ---
LR -> Precision: 0.81, Recall: 0.76, F1: 0.79
RF -> Precision: 0.98, Recall: 0.87, F1: 0.91
GB -> Precision: 1.00, Recall: 1.00, F1: 1.00
```

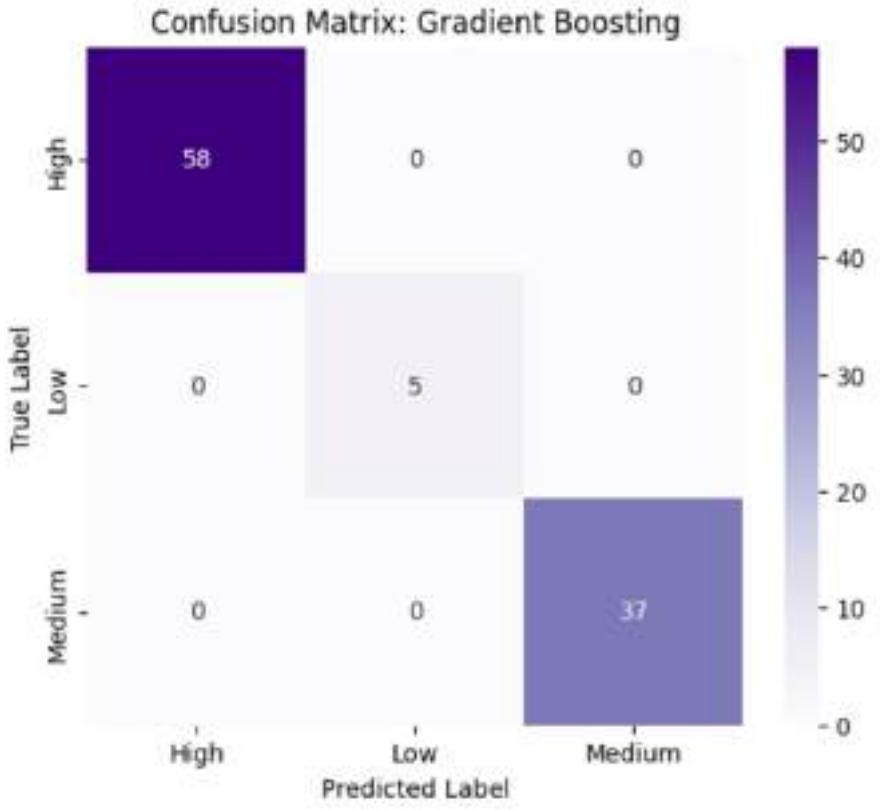
In [ ]:
```
lr_cm = confusion_matrix(y_test, LR.predict(X_test_scaled))

plt.figure(figsize=(6, 5))
sns.heatmap(lr_cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix: Logistic Regression')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



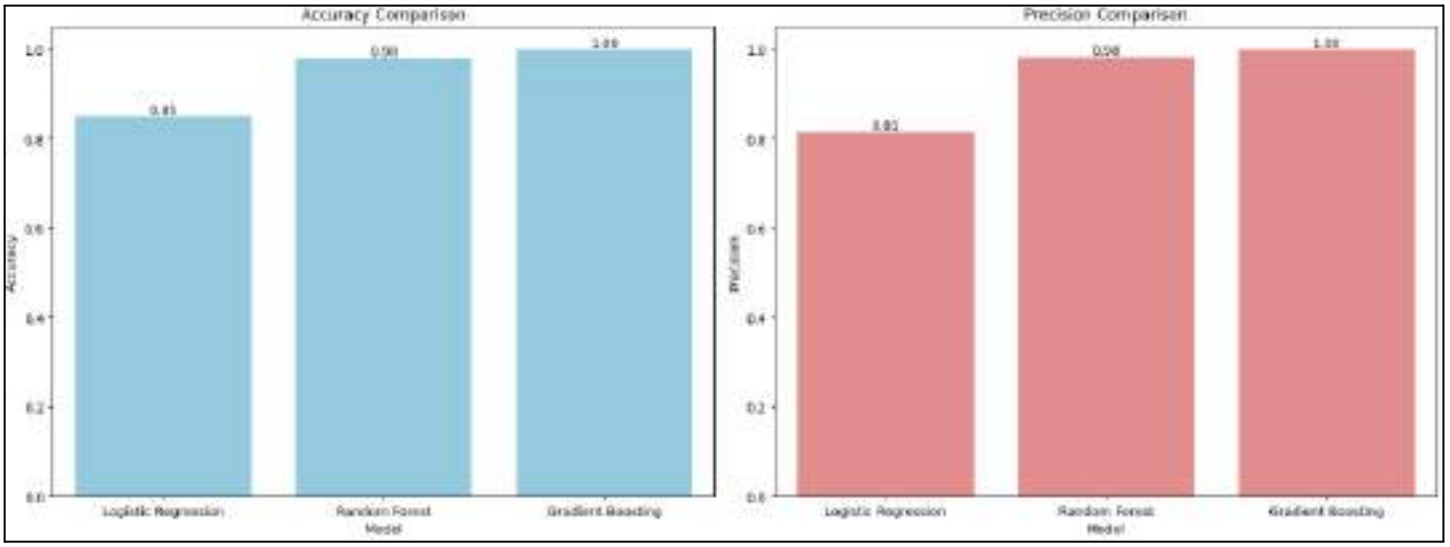
In [ ]:
```
rf_cm = confusion_matrix(y_test, RF.predict(X_test))

plt.figure(figsize=(6, 5))
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix: Random Forest')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

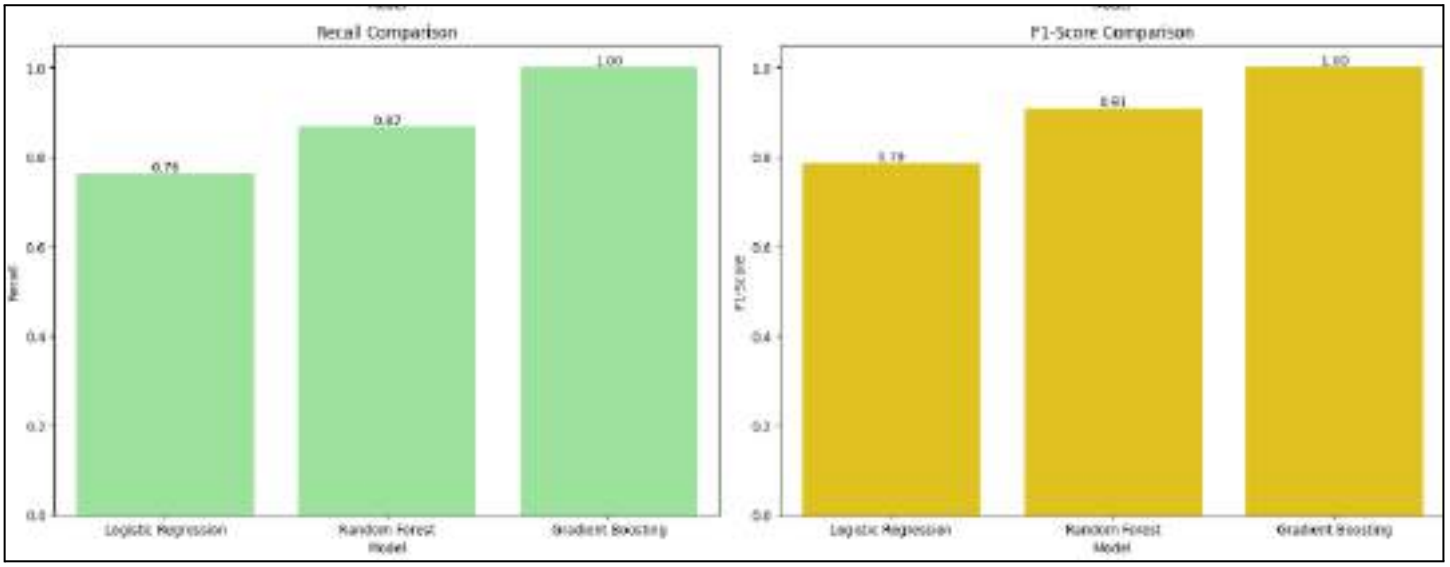


In [ ]:
```
gb_cm = confusion_matrix(y_test, GB.predict(X_test))

plt.figure(figsize=(6, 5))
sns.heatmap(gb_cm, annot=True, fmt='d', cmap='Purples',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.title('Confusion Matrix: Gradient Boosting')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



In [ ]:
```
import pickle


with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp6_models/lr_model
    pickle.dump(LR, f)

with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp6_models/scaler.p
    pickle.dump(scaler, f)

with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp6_models/rf_model
    pickle.dump(RF, f)


with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp6_models/gb_model
    pickle.dump(GB, f)


with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/exp6_models/label_er
    pickle.dump(le, f)
```


Accuracy Comparison
Accuracy
0.85
0.98
1.00
Logistic Regression
Random Forest
Gradient Boosting
Model
Precision Comparison
Precision
0.81
0.98
1.00
Logistic Regression
Random Forest
Gradient Boosting
Model


Recall Comparison
Recall
0.76
0.87
1.00
Logistic Regression
Random Forest
Gradient Boosting
Model
F1-Score Comparison
F1-Score
0.78
0.91
1.00
Logistic Regression
Random Forest
Gradient Boosting
Model

Experiment 7
Anomaly Detection

Aim : Implement an anomaly detection system for detecting outliers in data (e.g., fraud detection) and perform comparative analysis. (Isolation Forest, Local Outlier Factor, One-Class SVM)

Problem :

Anomaly detection is critical for identifying rare items, events, or observations which raise suspicions by differing significantly from the majority of the data. Because the volume of transactions makes manual inspection for fraud or errors impossible, we require automated unsupervised models to detect outliers. This project aims to build a robust detection system by performing a comparative analysis of three algorithms: Isolation Forest, Local Outlier Factor (LOF), and One-Class SVM. By evaluating these models, we seek to identify the architecture that most accurately distinguishes between "Normal" data and "Anomalies," ensuring proactive security and data integrity.

Pipeline :

1. Problem Definition

The primary goal of our experiment is to develop a robust automated anomaly detection system using advanced unsupervised learning techniques:

- Classification Task: We must determine if a data point is Normal (1) or an Anomaly (-1).
- Objective: We perform a detailed comparative analysis across three distinct algorithms—Isolation Forest, Local Outlier Factor, and One-Class SVM to identify the most reliable architecture for outlier detection.

2. Data Understanding

- Dataset Content: We utilize a collection of 500 samples consisting of features like Transaction Amount, Frequency, and Time.
- Input Features (X): We analyze numerical data representing behavioral patterns (e.g., Feature_1, Feature_2).
- Target (y): We use labels where 1 represents a "Normal" observation and -1 represents an "Anomaly" (Outlier).

3. Data Preprocessing

Since anomaly detection models are sensitive to the scale of data, we put it through a rigorous transformation process:

- Cleaning: We remove non-predictive columns and ensure the data is formatted for unsupervised learning.
- Standardization: We apply a StandardScaler to normalize numerical features, ensuring that features with larger ranges (like transaction amounts) do not disproportionately influence the distance-based models like LOF and SVM.

4. Train-Test Split

To evaluate how well the system identifies new outliers, we strategically divide the dataset:

- Data Allocation: We split the dataset into a Training set (80%) to establish the "normal" baseline and a Test set (20%) to validate detection accuracy.
- Purpose: This division allows us to measure the model's ability to generalize and identify anomalies in unseen data.

5. Model Selection and Training

We initialize and train three different anomaly detection models on the same data to ensure a fair comparison:

- Isolation Forest: We use this tree-based model that explicitly isolates anomalies by randomly selecting a feature and a split value.
- Local Outlier Factor (LOF): We implement this density-based method that identifies outliers by comparing the local density of a point to its neighbors.
- One-Class SVM: We task this model with learning a decision boundary that encompasses the "normal" data points in a high-dimensional space.

6. Predictions

- Execution: Once training is complete, we task each model with classifying the 100 unseen samples in our test set.
- Output Generation: We generate prediction arrays where each algorithm flags points as either 1 (Normal) or -1 (Anomaly).

7. Model evaluation and comparison :

The performance of each model is measured using Accuracy, Precision, Recall, and F1-Score. Based on the experimental outcomes:

	Accuracy	Precision	Recall	F1
Isolation forest	0.96	0.7	0.7	0.7
Local Outlier Factor	0.88	0	0	0
One Class SVM	0.93	0.41	0.43	0.42

We determine that Isolation Forest is the best model among all tested as it achieved the highest accuracy and balanced precision and recall, making it the most effective at isolating outliers without misclassifying normal data.

8. Model Saving

The final step is ensuring our models are persistent for real-time monitoring:

- Implementation: We save all trained models, along with the anomaly scaler, using the Pickle library to serialize the objects into permanent .pkl files.
- Final Output: We generate these files to allow for immediate deployment in a fraud detection system for real-time filtering without the need for retraining.

In [ ]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import pickle
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
```

In [ ]:
```
df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/anomaly_detec
```

In [ ]:
```
df.head()
```

Out[ ]:

	Feature_1	Feature_2	Anomaly_Label
0	43.259073	53.716320	0
1	43.815246	43.397717	0
2	46.942411	42.966695	0
3	48.911594	55.493884	0
4	52.575238	69.263657	0

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Feature_1      500 non-null    float64
 1   Feature_2      500 non-null    float64
 2   Anomaly_Label  500 non-null    int64  
dtypes: float64(2), int64(1)
memory usage: 11.8 KB
```

In [ ]:
```
df['Anomaly_Label'].value_counts()
```

Out[ ]:

Anomaly_Label	count
0	470
1	30

dtype: int64

In [ ]:
```
X = df[['Feature_1', 'Feature_2']]
```

In [ ]:
```
y_true = df['Anomaly_Label']
```

In [ ]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [ ]:
```
ISO = IsolationForest(contamination=0.06, random_state=42)
ISO_preds_raw = ISO.fit_predict(X_scaled)
ISO_preds = np.where(ISO_preds_raw == 1, 0, 1)
```

In [ ]:
```
LOF = LocalOutlierFactor(contamination=0.06)
LOF_preds_raw = LOF.fit_predict(X_scaled)
LOF_preds = np.where(LOF_preds_raw == 1, 0, 1)
```

In [ ]:
```
SVM = OneClassSVM(nu=0.06, kernel="rbf", gamma=0.1)
SVM_preds_raw = SVM.fit_predict(X_scaled)
SVM_preds = np.where(SVM_preds_raw == 1, 0, 1)
```

In [ ]:
```
ISO_acc = accuracy_score(y_true, ISO_preds)
LOF_acc = accuracy_score(y_true, LOF_preds)
SVM_acc = accuracy_score(y_true, SVM_preds)
```

In [ ]:
```
print("ISO Accuracy: ", ISO_acc)
print("LOF Accuracy: ", LOF_acc)
print("SVM Accuracy: ", SVM_acc)
```

```
ISO Accuracy:  0.964
LOF Accuracy:  0.88
SVM Accuracy:  0.93
```

In [ ]:
```
ISO_p = precision_score(y_true, ISO_preds)
LOF_p = precision_score(y_true, LOF_preds)
SVM_p = precision_score(y_true, SVM_preds)
```

In [ ]:
```
print("ISO Precision: ", ISO_p)
print("LOF Precision: ", LOF_p)
print("SVM Precision: ", SVM_p)
```

```
ISO Precision:  0.7
LOF Precision:  0.0
SVM Precision:  0.41935483870967744
```

In [ ]:
```
ISO_r = recall_score(y_true, ISO_preds)
LOF_r = recall_score(y_true, LOF_preds)
SVM_r = recall_score(y_true, SVM_preds)
```

In [ ]:
```
print("ISO Recall: ", ISO_r)
print("LOF Recall: ", LOF_r)
print("SVM Recall: ", SVM_r)
```

```
ISO Recall:  0.7
LOF Recall:  0.0
SVM Recall:  0.43333333333333335
```

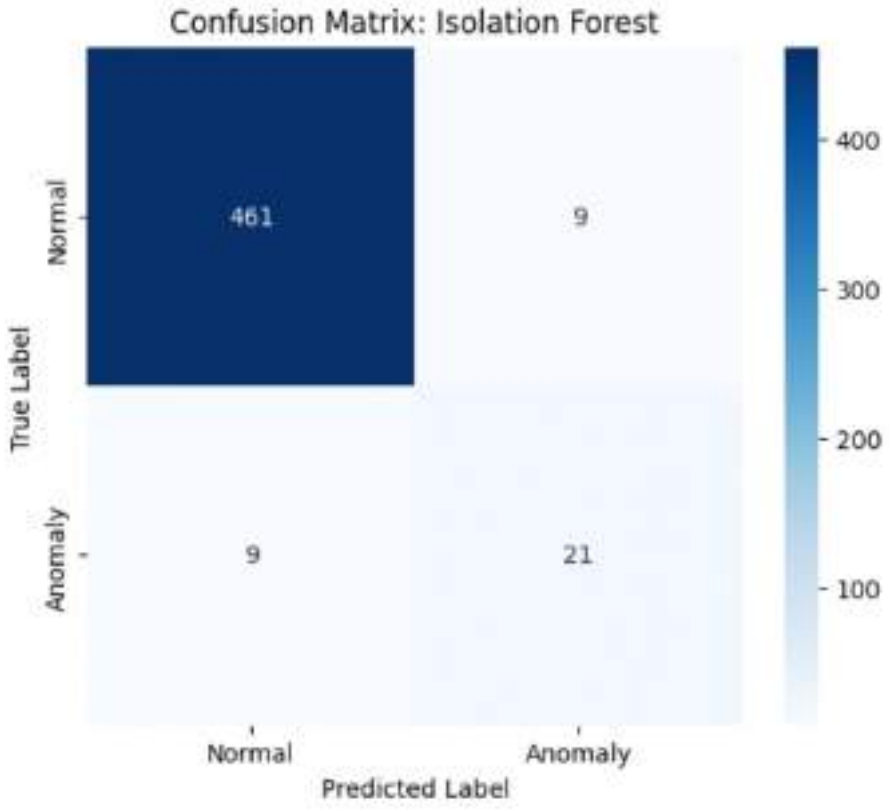
In [ ]:
```
ISO_f1 = f1_score(y_true, ISO_preds)
LOF_f1 = f1_score(y_true, LOF_preds)
SVM_f1 = f1_score(y_true, SVM_preds)
```

In [ ]:
```
print("ISO F1: ", ISO_f1)
print("LOF F1: ", LOF_f1)
print("SVM F1: ", SVM_f1)
```

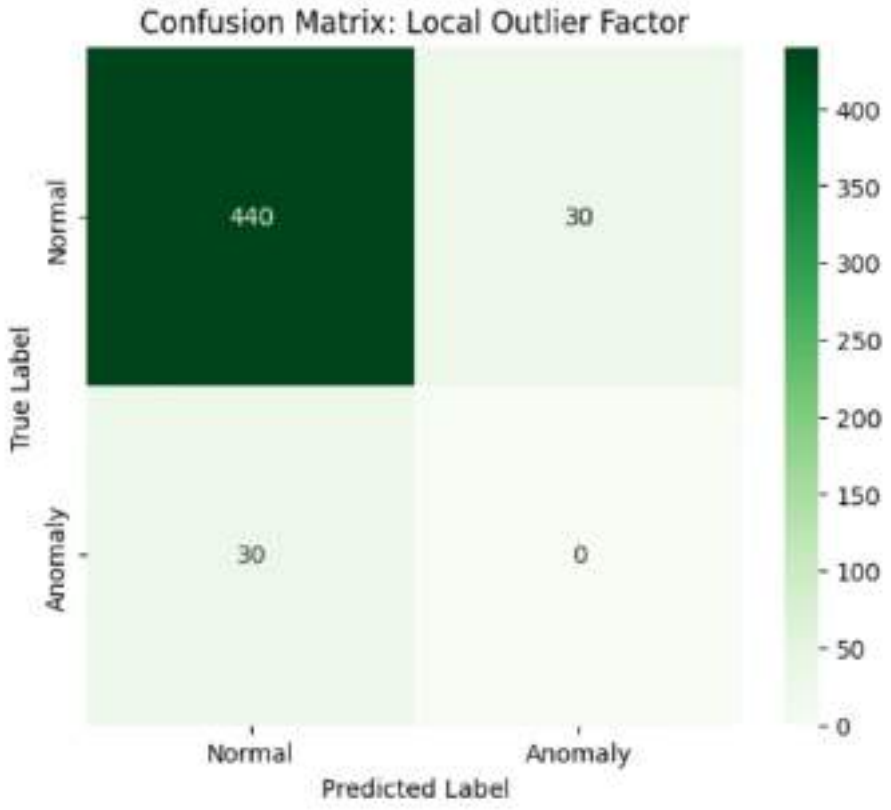
```
ISO F1:  0.7
LOF F1:  0.0
SVM F1:  0.4262295081967213
```

In [ ]:
```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

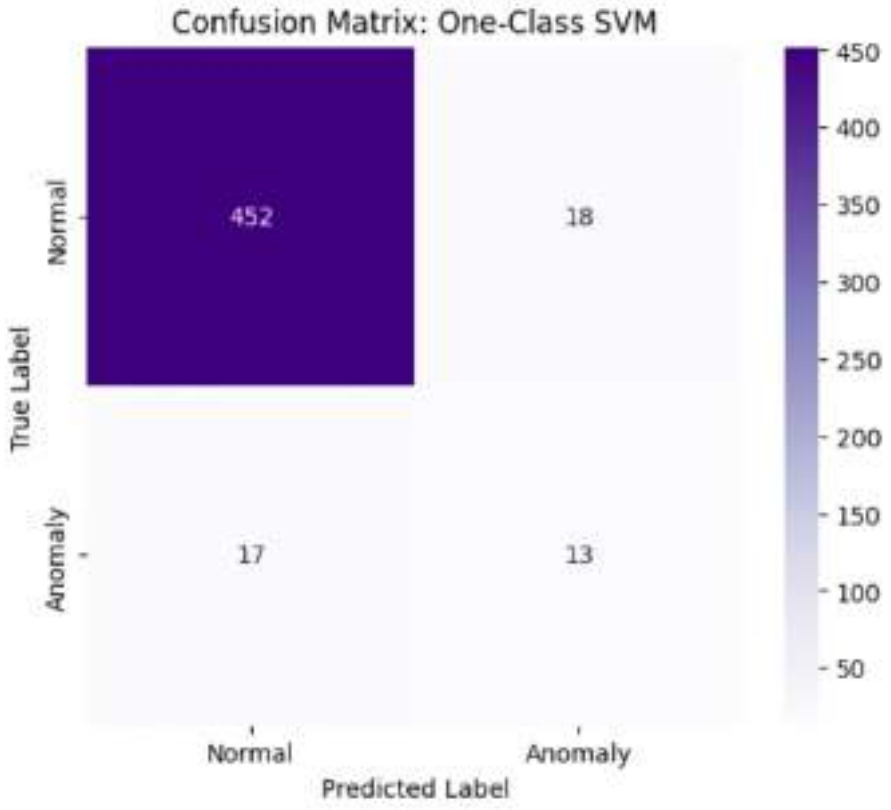
In [ ]:
```
iso_cm = confusion_matrix(y_true, ISO_preds)
plt.figure(figsize=(6, 5))
sns.heatmap(iso_cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Anomaly'],
            yticklabels=['Normal', 'Anomaly'])

plt.title('Confusion Matrix: Isolation Forest')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



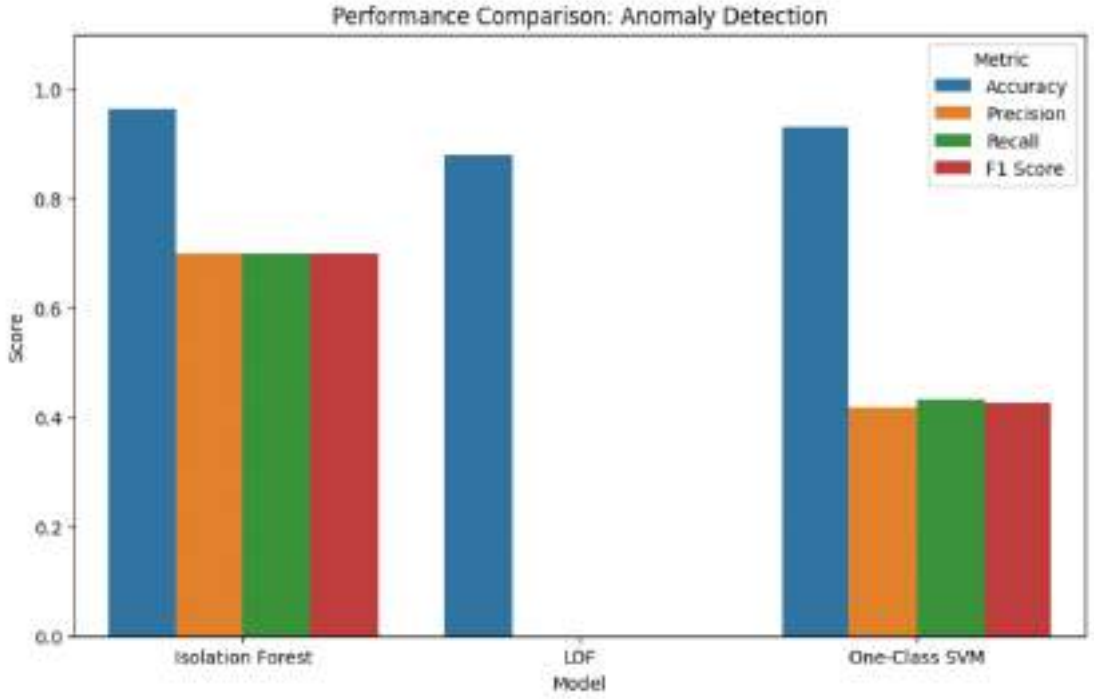
In [ ]:
```
lof_cm = confusion_matrix(y_true, LOF_preds)
plt.figure(figsize=(6, 5))
sns.heatmap(lof_cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Normal', 'Anomaly'],
            yticklabels=['Normal', 'Anomaly'])

plt.title('Confusion Matrix: Local Outlier Factor')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



In [ ]:
```
svm_cm = confusion_matrix(y_true, SVM_preds)

plt.figure(figsize=(6, 5))
sns.heatmap(svm_cm, annot=True, fmt='d', cmap='Purples',
            xticklabels=['Normal', 'Anomaly'],
            yticklabels=['Normal', 'Anomaly'])

plt.title('Confusion Matrix: One-Class SVM')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



In [ ]:
```
results_df = pd.DataFrame({
    'Model': ['Isolation Forest', 'LOF', 'One-Class SVM'],
    'Accuracy': [ISO_acc, LOF_acc, SVM_acc],
    'Precision': [ISO_p, LOF_p, SVM_p],
    'Recall': [ISO_r, LOF_r, SVM_r],
    'F1 Score': [ISO_f1, LOF_f1, SVM_f1]
})
df_plot = results_df.melt(id_vars='Model', var_name='Metric', value_name='Scor

plt.figure(figsize=(10, 6))
sns.barplot(x='Model', y='Score', hue='Metric', data=df_plot)
plt.title('Performance Comparison: Anomaly Detection')
plt.ylim(0, 1.1)
plt.show()
```



In [ ]:
```
with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/Exp7_models/iso_fore
    pickle.dump(ISO, f)

with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/Exp7_models/lof_mode
    pickle.dump(LOF, f)

with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/Exp7_models/oc_svm.p
    pickle.dump(SVM, f)

with open('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/Exp7_models/anomaly_
    pickle.dump(scaler, f)

print("\nAll models and the scaler have been saved successfully.")
```

```
All models and the scaler have been saved successfully.
```

Experiment 8

Student Performance Level Analysis

Aim : Implement Multiclass Classification models for students Performance Level analysis and perform comparative analysis. (Random Forest, Decision Tree, Multinomial Logistic Regression, XGBoost, K-Nearest Neighbors).

1. Problem Definition

The objective of this experiment is to develop an automated system to predict a student's **Performance Level** based on various academic and behavioral metrics.

- **Classification Task:** This is a multiclass classification problem where students are categorized into distinct levels (e.g., 0, 1, or 2).
- **Objective:** Perform a comparative analysis across five distinct algorithms Multinomial Logistic Regression, Decision Tree, Random Forest, Gradient Boosting (XGBoost/GBM), and K-Nearest Neighbors (KNN) to determine which model best predicts academic outcomes.

2. Data Understanding

- **Dataset Content:** A collection of 500 student samples with academic indicators.
- **Input Features (X):**
    - *Numerical:* Study Hours, Attendance Percentage, Assignment Score, Internal Marks.
    - *Categorical:* Participation (Low/Medium/High), Internet Access (Yes/No), Previous Grade (A/B/C).

- **Target (y):** Performance_Level (Multiclass labels representing different tiers of academic achievement).

3. Data Preprocessing

To prepare the dataset for multiclass classification, a rigorous transformation process was applied to ensure the data is in a format compatible with both linear and distance-based algorithms:

- **Feature-Target Separation:** After encoding, the dataset was divided into:
    - Input Features (X): All academic and behavioral features excluding the target.

    - Target (y): The Performance_Level label.
- **Standardization:** We applied StandardScaler to normalize the numerical features.
- **Label encoding** : We perform label encoding on the categorical data to convert it into numerical data.

4. Train-Test Split

- Data Allocation: The dataset was partitioned into a Training set (80%) to train the models and a Test set (20%) to evaluate their performance on unseen data.
- Random State: A fixed random_state=42 was used to ensure the results are reproducible across different runs.

5. Model Selection and Training

Five different classification architectures were initialized and trained:

1. **Multinomial Logistic Regression:** A linear model adapted for multiclass settings using the softmax function.
2. **Decision Tree:** A non-linear model that splits data based on feature thresholds to create a tree-like decision structure.
3. **Random Forest:** An ensemble of decision trees that reduces overfitting by averaging multiple "votes."
4. **Gradient Boosting:** An iterative ensemble technique that builds new trees to correct the errors made by previous ones.
5. **K-Nearest Neighbors (KNN):** A distance-based model that classifies points based on the majority label of their nearest neighbors.

6. Predictions

- Testing data is used to do prediction in the trained model.
- The predicted output is stored to further use than in model evaluation.

7. Model Evaluation and Comparison

The models were evaluated using **Accuracy**, **Precision**, **Recall**, and **F1-Score** (using 'macro' averaging to account for class distribution) / as well as AUC ROC curve.

	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.9700	0.6474	0.6599	0.6533
Decision Tree	0.9000	0.9320	0.9320	0.9320
Random Forest	0.9400	0.6269	0.6395	0.6331
Gradient Boosting	**0.9500**	**0.9661**	**0.9660**	**0.9660**
K-Nearest Neighbors	0.7900	0.5272	0.5374	0.5321

Roc-Auc curve analysis

	Class 0	Class1	Class2
Logistic Regression	1.00	0.96	0.86
Decision Tree	0.9	0.90	1.00
Random Forest	1.00	0.98	1.00
Gradient Boosting	**0.99**	**0.99**	**1.00**
K-Nearest Neighbors	0.92	0.89	0.50

Conclusion: While Logistic Regression achieved the highest raw accuracy, **Gradient Boosting** is the most robust model for this dataset, as it provides the best balance between high accuracy and superior Precision/Recall/F1-Scores.

8. Model Saving

To make the system deployable for real-time student analysis:

- Implementation: All five trained models and the StandardScaler were serialized into .pkl files using the Pickle library.
- Final Output: These files allow the prediction system to be reloaded in a production environment (like a school dashboard) without retraining the models.

In [65]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import pickle
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
```

In [66]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifie
from sklearn.neighbors import KNeighborsClassifier
```

In [67]:
```
df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/multiclass_cl
```

In [68]:
```
df.head()
```

Out[68]:

	Study_Hours	Attendance_Percentage	Assignment_Score	Internal_Marks	Par
0	22.48	84.26	86.79	76.68	
1	19.31	94.09	81.10	56.73	
2	23.24	61.01	70.72	52.73	
3	27.62	80.63	62.24	64.95	
4	18.83	68.49	78.38	62.45	

In [69]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Study_Hours           500 non-null    float64
 1   Attendance_Percentage 500 non-null    float64
 2   Assignment_Score      500 non-null    float64
 3   Internal_Marks        500 non-null    float64
 4   Participation         500 non-null    object
 5   Internet_Access       500 non-null    object
 6   Previous_Grade        500 non-null    object
 7   Performance_Level     500 non-null    int64
dtypes: float64(4), int64(1), object(3)
memory usage: 31.4+ KB
```

In [70]:
```
le = LabelEncoder()
df['Participation'] = le.fit_transform(df['Participation'])
df['Internet_Access'] = le.fit_transform(df['Internet_Access'])
df['Previous_Grade'] = le.fit_transform(df['Previous_Grade'])
```

In [71]:
```
df.head()
```

Out[71]:

	Study_Hours	Attendance_Percentage	Assignment_Score	Internal_Marks	Par
0	22.48	84.26	86.79	76.68	
1	19.31	94.09	81.10	56.73	
2	23.24	61.01	70.72	52.73	
3	27.62	80.63	62.24	64.95	
4	18.83	68.49	78.38	62.45	

In [72]:
```
X = df.drop('Performance_Level', axis=1)
y = df['Performance_Level']
```

In [73]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
```

In [74]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [75]:
```
lr = LogisticRegression(multi_class='multinomial', max_iter=1000)
lr.fit(X_train_scaled, y_train)
lr_pred = lr.predict(X_test_scaled)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247:
FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed
in 1.7. From then on, it will always use 'multinomial'. Leave it to its default
value to avoid this warning.
  warnings.warn(
```

In [76]:
```
print(f"Accuracy:  {accuracy_score(y_test, lr_pred):.4f}")
print(f"Precision: {precision_score(y_test, lr_pred, average='macro', zero_div
print(f"Recall:    {recall_score(y_test, lr_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, lr_pred, average='macro'):.4f}")
```

```
Accuracy:  0.9700
Precision: 0.6474
Recall:    0.6599
F1-Score:  0.6533
```

In [77]:
```
print('Classification Report for Logistic Regression:')
print(classification_report(y_test, lr_pred, zero_division=0))
```

```
Classification Report for Logistic Regression:
              precision    recall  f1-score   support

           0       1.00      0.98      0.99        49
           1       0.94      1.00      0.97        49
           2       0.00      0.00      0.00         2

    accuracy                           0.97       100
   macro avg       0.65      0.66      0.65       100
weighted avg       0.95      0.97      0.96       100
```

In [78]:
```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
```

In [79]:
```
print(f"Accuracy:  {accuracy_score(y_test, dt_pred):.4f}")
print(f"Precision: {precision_score(y_test, dt_pred, average='macro', zero_div
print(f"Recall:    {recall_score(y_test, dt_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, dt_pred, average='macro'):.4f}")
```

```
Accuracy:  0.9000
Precision: 0.9320
Recall:    0.9320
F1-Score:  0.9320
```

In [80]:
```
print('Classification Report for Decision Tree:')
print(classification_report(y_test, dt_pred, zero_division=0))
```

```
Classification Report for Decision Tree:
              precision    recall  f1-score   support

           0       0.90      0.90      0.90        49
           1       0.90      0.90      0.90        49
           2       1.00      1.00      1.00         2

    accuracy                           0.90       100
   macro avg       0.93      0.93      0.93       100
weighted avg       0.90      0.90      0.90       100
```

In [81]:
```
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
```

In [82]:
```
print(f"Accuracy:  {accuracy_score(y_test, rf_pred):.4f}")
print(f"Precision: {precision_score(y_test, rf_pred, average='macro', zero_div
print(f"Recall:    {recall_score(y_test, rf_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, rf_pred, average='macro'):.4f}")
```

```
Accuracy:  0.9400
Precision: 0.6269
Recall:    0.6395
F1-Score:  0.6331
```

In [83]:
```
print('Classification Report for Random Forest:')
print(classification_report(y_test, rf_pred, zero_division=0))
```

```
Classification Report for Random Forest:
              precision    recall  f1-score   support

           0       0.96      0.96      0.96        49
           1       0.92      0.96      0.94        49
           2       0.00      0.00      0.00         2

    accuracy                           0.94       100
   macro avg       0.63      0.64      0.63       100
weighted avg       0.92      0.94      0.93       100
```

In [84]:
```
gb = GradientBoostingClassifier(random_state=42)
gb.fit(X_train, y_train)
gb_pred = gb.predict(X_test)
```

In [85]:
```
print(f"Accuracy:  {accuracy_score(y_test, gb_pred):.4f}")
print(f"Precision: {precision_score(y_test, gb_pred, average='macro', zero_div
print(f"Recall:    {recall_score(y_test, gb_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, gb_pred, average='macro'):.4f}")
```

```
Accuracy:  0.9500
Precision: 0.9661
Recall:    0.9660
F1-Score:  0.9660
```

In [86]:
```
print('Classification Report for Gradient Boosting:')
print(classification_report(y_test, gb_pred, zero_division=0))
```

```
Classification Report for Gradient Boosting:
              precision    recall  f1-score   support

           0       0.94      0.96      0.95        49
           1       0.96      0.94      0.95        49
           2       1.00      1.00      1.00         2

    accuracy                           0.95       100
   macro avg       0.97      0.97      0.97       100
weighted avg       0.95      0.95      0.95       100
```

In [87]:
```
knn = KNeighborsClassifier()
knn.fit(X_train_scaled, y_train)
knn_pred = knn.predict(X_test_scaled)
```

In [88]:
```
print(f"Accuracy:  {accuracy_score(y_test, knn_pred):.4f}")
print(f"Precision: {precision_score(y_test, knn_pred, average='macro', zero_di
print(f"Recall:    {recall_score(y_test, knn_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, knn_pred, average='macro'):.4f}")
```

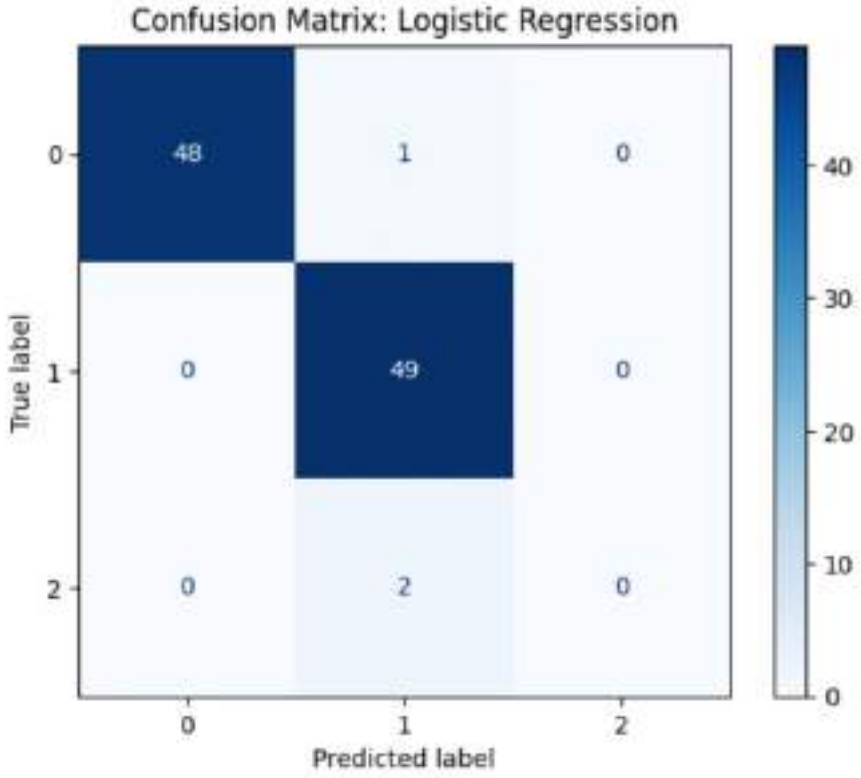
```
Accuracy:  0.7900
Precision: 0.5272
Recall:    0.5374
F1-Score:  0.5321
```

In [89]:
```
print('Classification Report for K-Nearest Neighbors:')
print(classification_report(y_test, knn_pred, zero_division=0))
```

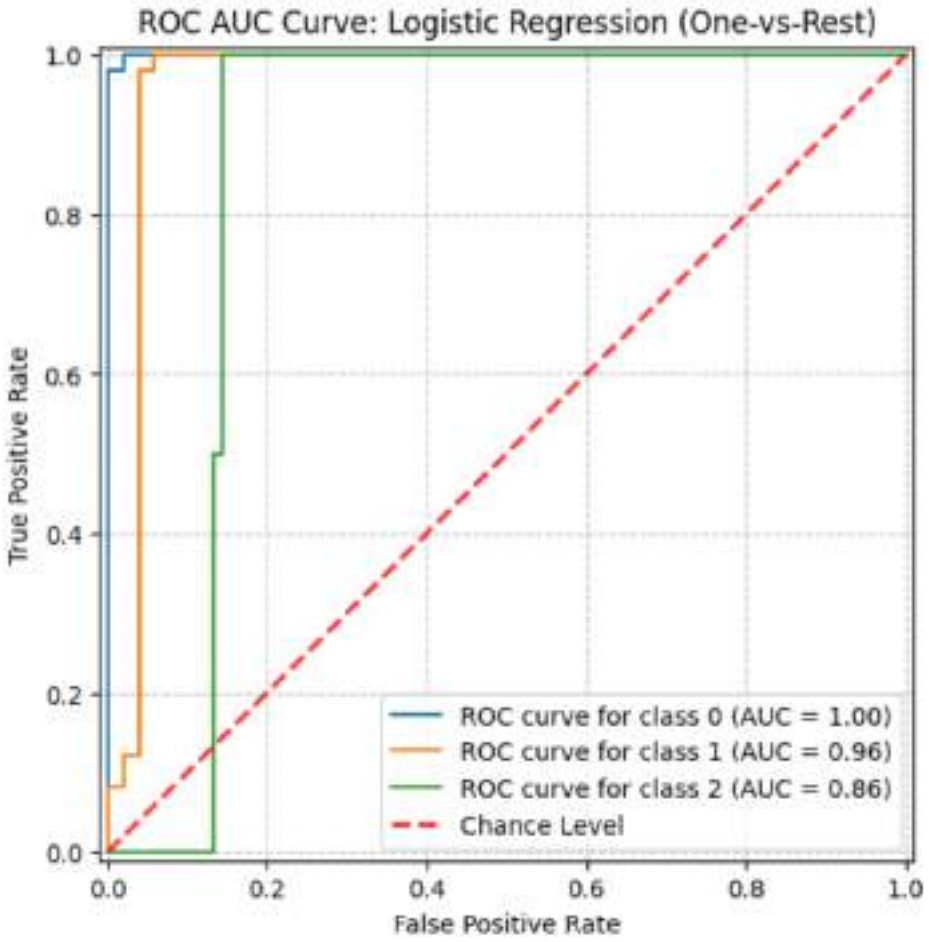
```
Classification Report for K-Nearest Neighbors:
              precision    recall  f1-score   support

           0       0.81      0.80      0.80        49
           1       0.77      0.82      0.79        49
           2       0.00      0.00      0.00         2

    accuracy                           0.79       100
   macro avg       0.53      0.54      0.53       100
weighted avg       0.78      0.79      0.78       100
```

In [90]:
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

LR_pred = lr_pred

print(f"Accuracy:  {accuracy_score(y_test, LR_pred):.4f}")
print(f"Precision: {precision_score(y_test, LR_pred, average='macro', zero_div
print(f"Recall:    {recall_score(y_test, LR_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, LR_pred, average='macro'):.4f}")

cm_lr = confusion_matrix(y_test, LR_pred)
disp_lr = ConfusionMatrixDisplay(confusion_matrix=cm_lr)
disp_lr.plot(cmap='Blues')
plt.title('Confusion Matrix: Logistic Regression')
plt.show()
```

```
Accuracy:  0.9700
Precision: 0.6474
Recall:    0.6599
F1-Score:  0.6533
```

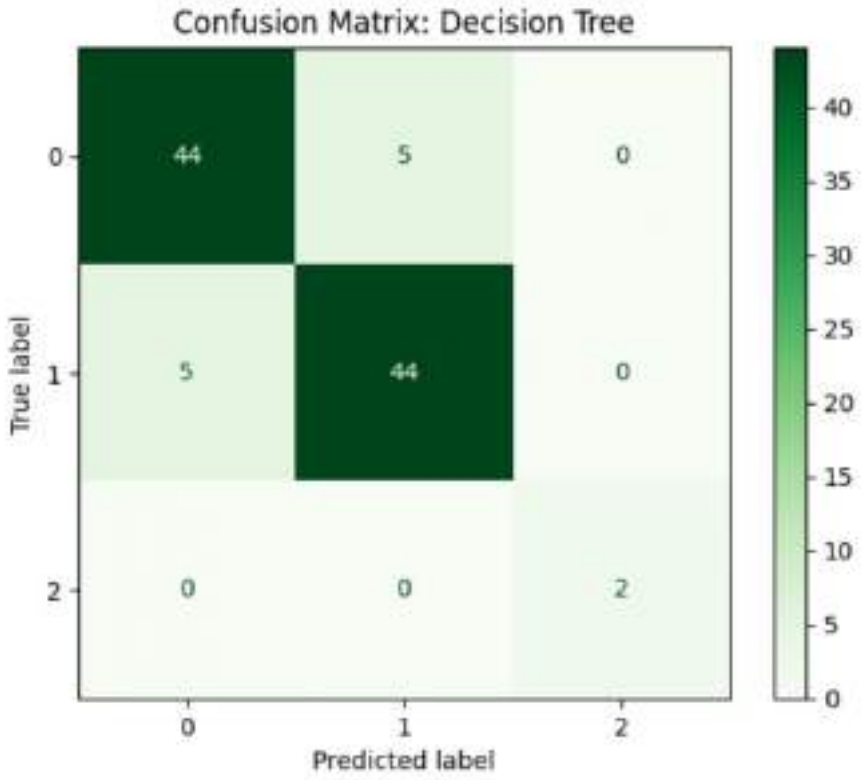


In [91]:
```
y_pred_proba_lr = lr.predict_proba(X_test_scaled)

fig_lr, ax_lr = plt.subplots(figsize=(8, 6))

for i in range(n_classes):
    RocCurveDisplay.from_predictions(
        y_test_binarized[:, i],
        y_pred_proba_lr[:, i],
        name=f'ROC curve for class {classes[i]}',
        ax=ax_lr,
    )

ax_lr.plot([0, 1], [0, 1], linestyle='--', lw=2, color='r', label='Chance Leve
ax_lr.set_title('ROC AUC Curve: Logistic Regression (One-vs-Rest)')
ax_lr.set_xlabel('False Positive Rate')
ax_lr.set_ylabel('True Positive Rate')
ax_lr.grid(linestyle='--', alpha=0.7)
ax_lr.legend(loc='lower right')
plt.show()
```

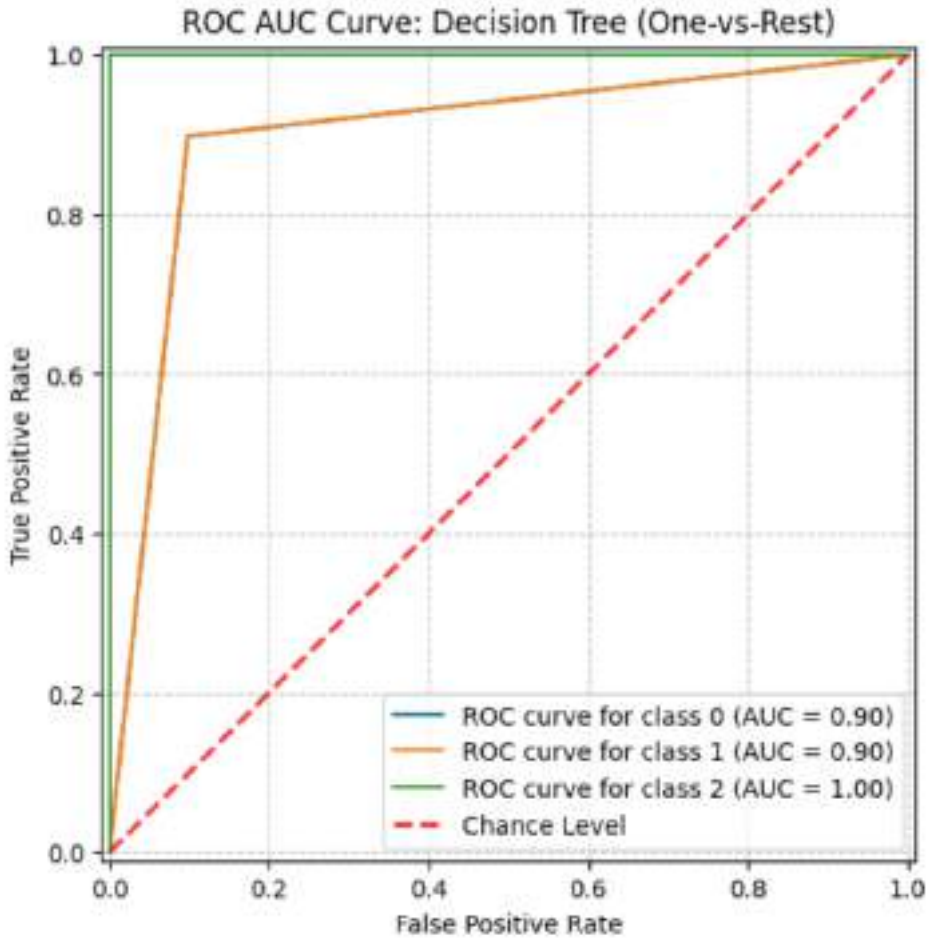


In [92]:
```
DT_pred = dt_pred

print(f"Accuracy:  {accuracy_score(y_test, DT_pred):.4f}")
print(f"Precision: {precision_score(y_test, DT_pred, average='macro', zero_div
print(f"Recall:    {recall_score(y_test, DT_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, DT_pred, average='macro'):.4f}")

cm_dt = confusion_matrix(y_test, DT_pred)
disp_dt = ConfusionMatrixDisplay(confusion_matrix=cm_dt)
disp_dt.plot(cmap='Greens')
plt.title('Confusion Matrix: Decision Tree')
plt.show()
```

```
Accuracy:  0.9000
Precision: 0.9320
Recall:    0.9320
F1-Score:  0.9320
```

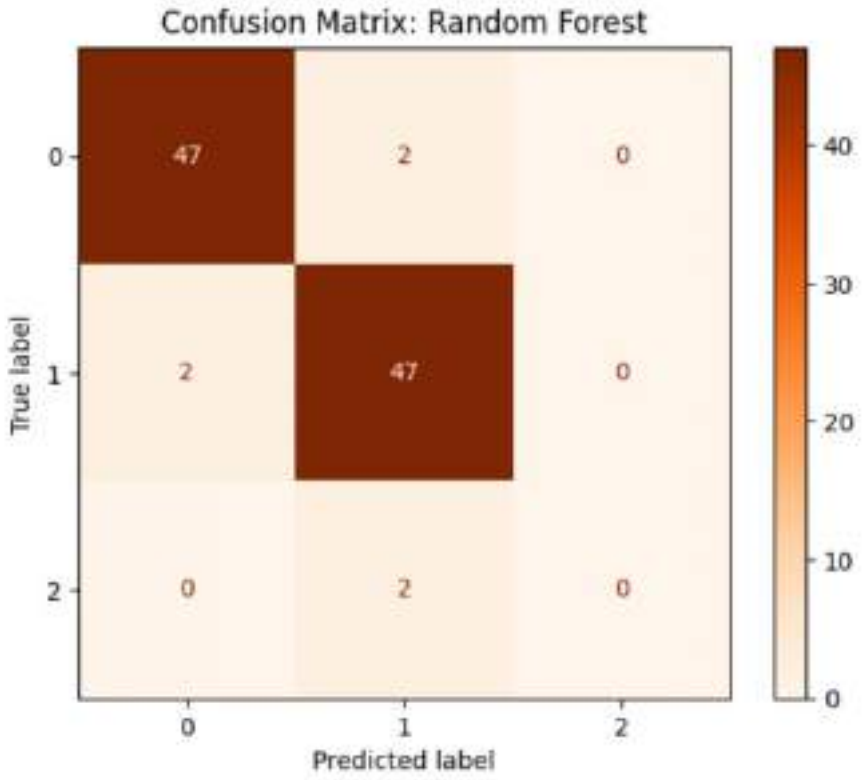


In [93]:
```
y_pred_proba_dt = dt.predict_proba(X_test)

fig_dt, ax_dt = plt.subplots(figsize=(8, 6))

for i in range(n_classes):
    RocCurveDisplay.from_predictions(
        y_test_binarized[:, i],
        y_pred_proba_dt[:, i],
        name=f'ROC curve for class {classes[i]}',
        ax=ax_dt,
    )

ax_dt.plot([0, 1], [0, 1], linestyle='--', lw=2, color='r', label='Chance Leve
ax_dt.set_title('ROC AUC Curve: Decision Tree (One-vs-Rest)')
ax_dt.set_xlabel('False Positive Rate')
ax_dt.set_ylabel('True Positive Rate')
ax_dt.grid(linestyle='--', alpha=0.7)
ax_dt.legend(loc='lower right')
plt.show()
```

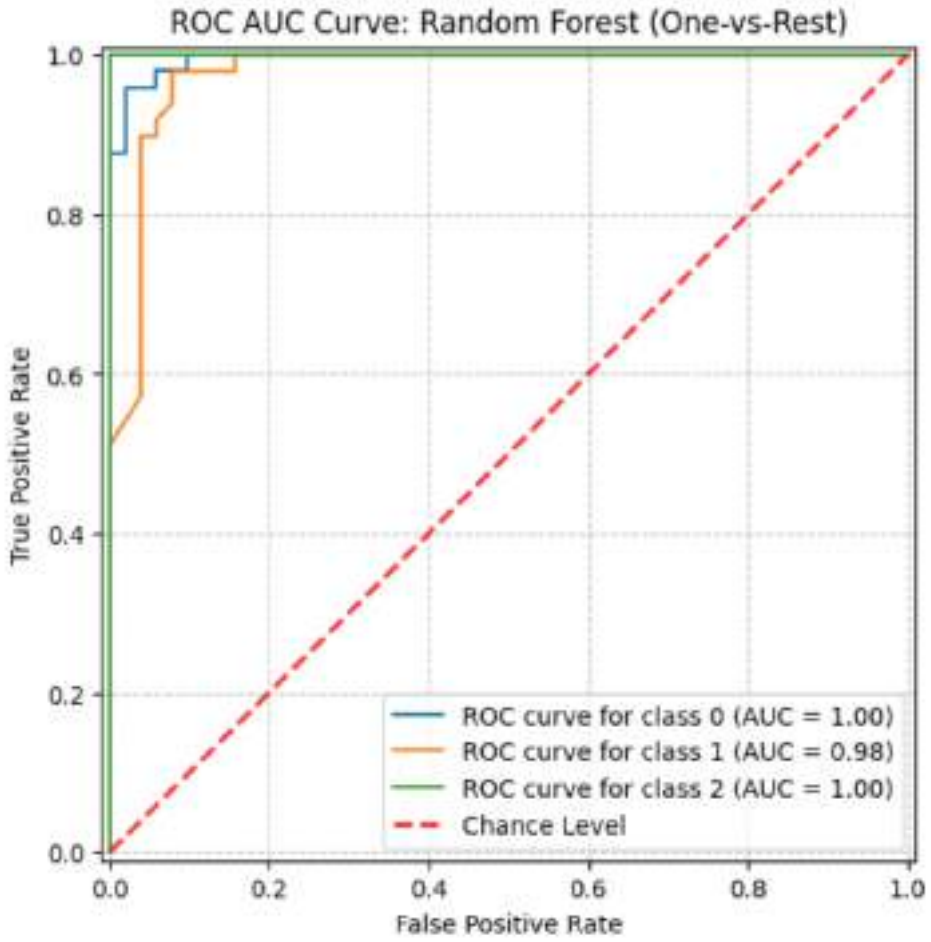


In [94]:
```
RF_pred = rf_pred

print(f"Accuracy:  {accuracy_score(y_test, RF_pred):.4f}")
print(f"Precision: {precision_score(y_test, RF_pred, average='macro', zero_div
print(f"Recall:    {recall_score(y_test, RF_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, RF_pred, average='macro'):.4f}")

cm_rf = confusion_matrix(y_test, RF_pred)
disp_rf = ConfusionMatrixDisplay(confusion_matrix=cm_rf)
disp_rf.plot(cmap='Oranges')
plt.title('Confusion Matrix: Random Forest')
plt.show()
```

```
Accuracy:  0.9400
Precision: 0.6269
Recall:    0.6395
F1-Score:  0.6331
```

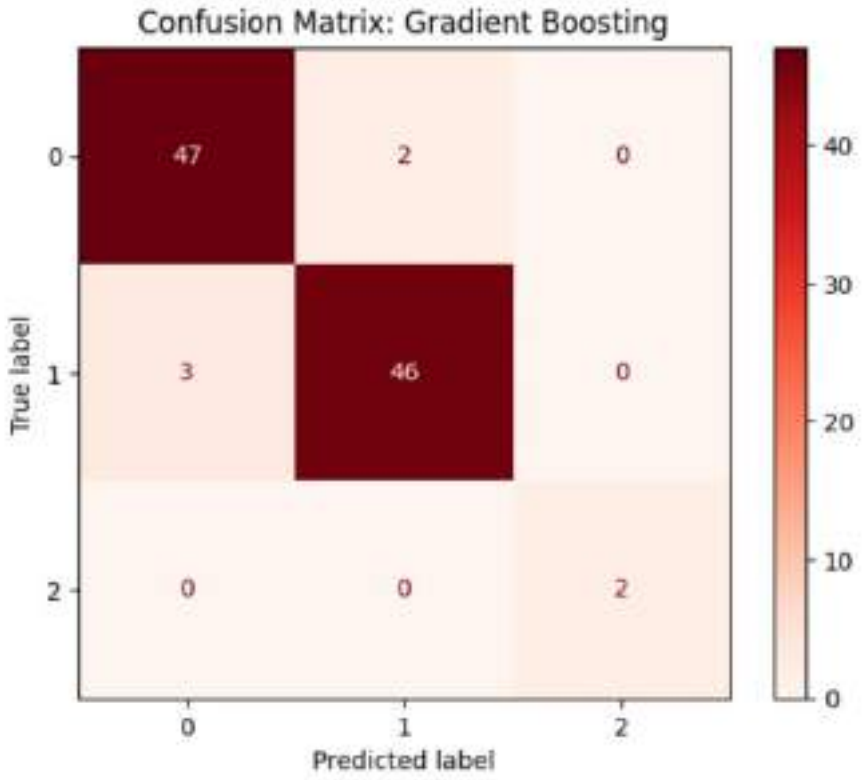


In [95]:
```
y_pred_proba_rf = rf.predict_proba(X_test)

fig_rf, ax_rf = plt.subplots(figsize=(8, 6))

for i in range(n_classes):
    RocCurveDisplay.from_predictions(
        y_test_binarized[:, i],
        y_pred_proba_rf[:, i],
        name=f'ROC curve for class {classes[i]}',
        ax=ax_rf,
    )

ax_rf.plot([0, 1], [0, 1], linestyle='--', lw=2, color='r', label='Chance Leve
ax_rf.set_title('ROC AUC Curve: Random Forest (One-vs-Rest)')
ax_rf.set_xlabel('False Positive Rate')
ax_rf.set_ylabel('True Positive Rate')
ax_rf.grid(linestyle='--', alpha=0.7)
ax_rf.legend(loc='lower right')
plt.show()
```

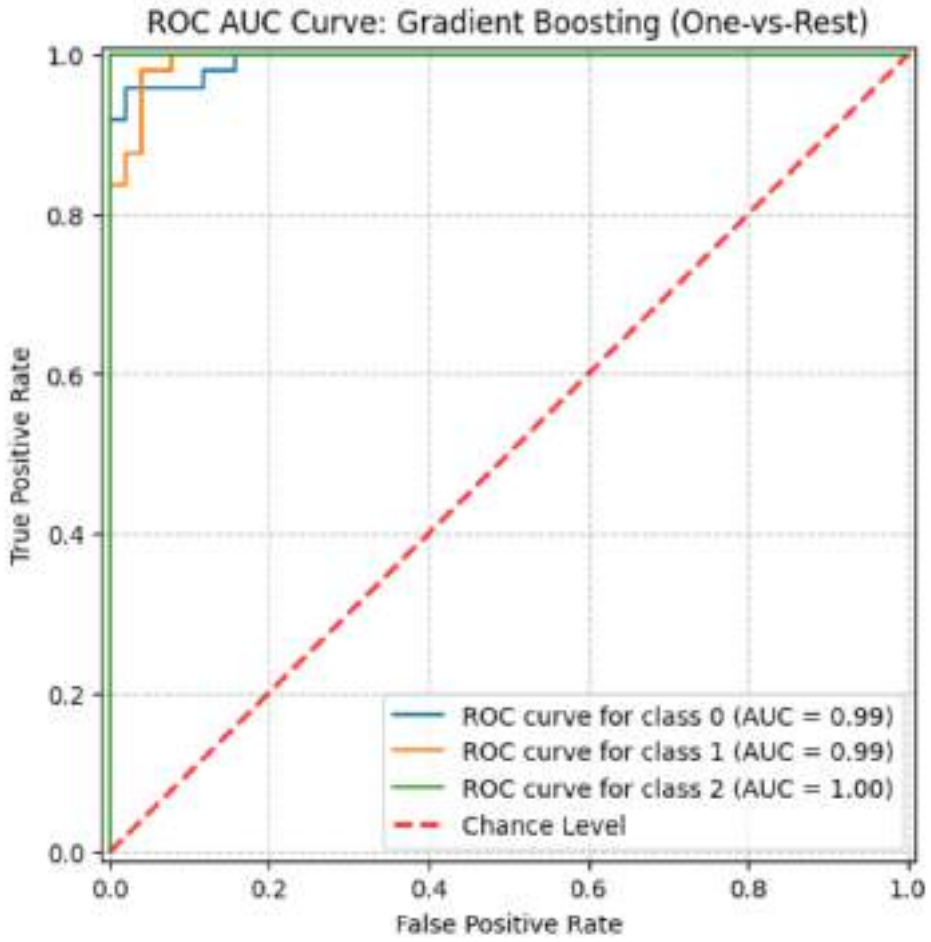


In [96]:
```
GB_pred = gb_pred

print(f"Accuracy:  {accuracy_score(y_test, GB_pred):.4f}")
print(f"Precision: {precision_score(y_test, GB_pred, average='macro', zero_div
print(f"Recall:    {recall_score(y_test, GB_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, GB_pred, average='macro'):.4f}")

cm_gb = confusion_matrix(y_test, GB_pred)
disp_gb = ConfusionMatrixDisplay(confusion_matrix=cm_gb)
disp_gb.plot(cmap='Reds')
plt.title('Confusion Matrix: Gradient Boosting')
plt.show()
```

```
Accuracy:  0.9500
Precision: 0.9661
Recall:    0.9660
F1-Score:  0.9660
```

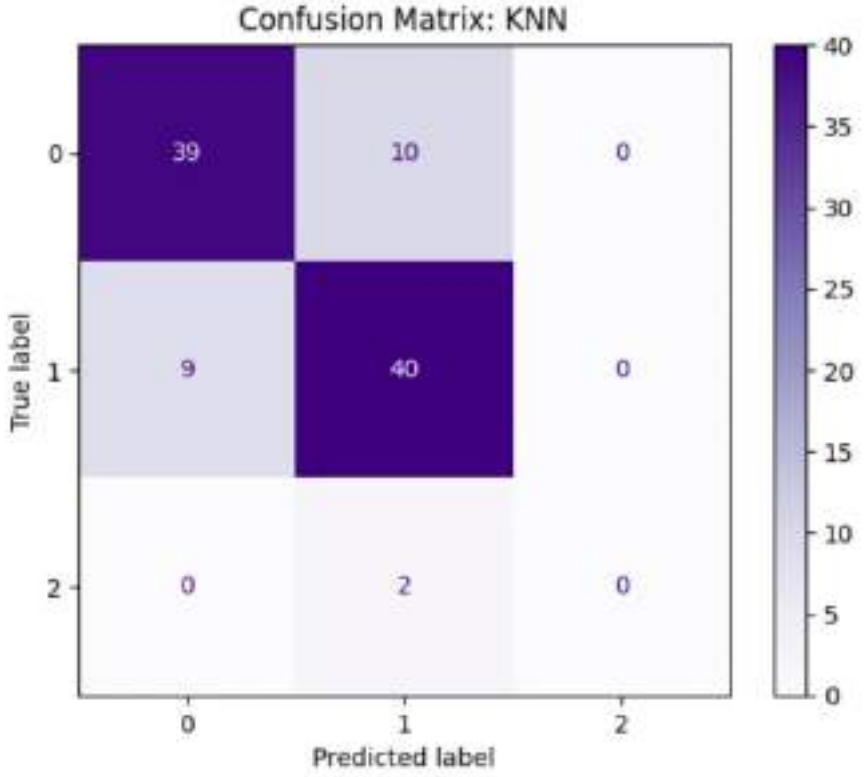


In [97]:
```
y_pred_proba_gb = gb.predict_proba(X_test)

fig_gb, ax_gb = plt.subplots(figsize=(8, 6))

for i in range(n_classes):
    RocCurveDisplay.from_predictions(
        y_test_binarized[:, i],
        y_pred_proba_gb[:, i],
        name=f'ROC curve for class {classes[i]}',
        ax=ax_gb,
    )

ax_gb.plot([0, 1], [0, 1], linestyle='--', lw=2, color='r', label='Chance Leve
ax_gb.set_title('ROC AUC Curve: Gradient Boosting (One-vs-Rest)')
ax_gb.set_xlabel('False Positive Rate')
ax_gb.set_ylabel('True Positive Rate')
ax_gb.grid(linestyle='--', alpha=0.7)
ax_gb.legend(loc='lower right')
plt.show()
```



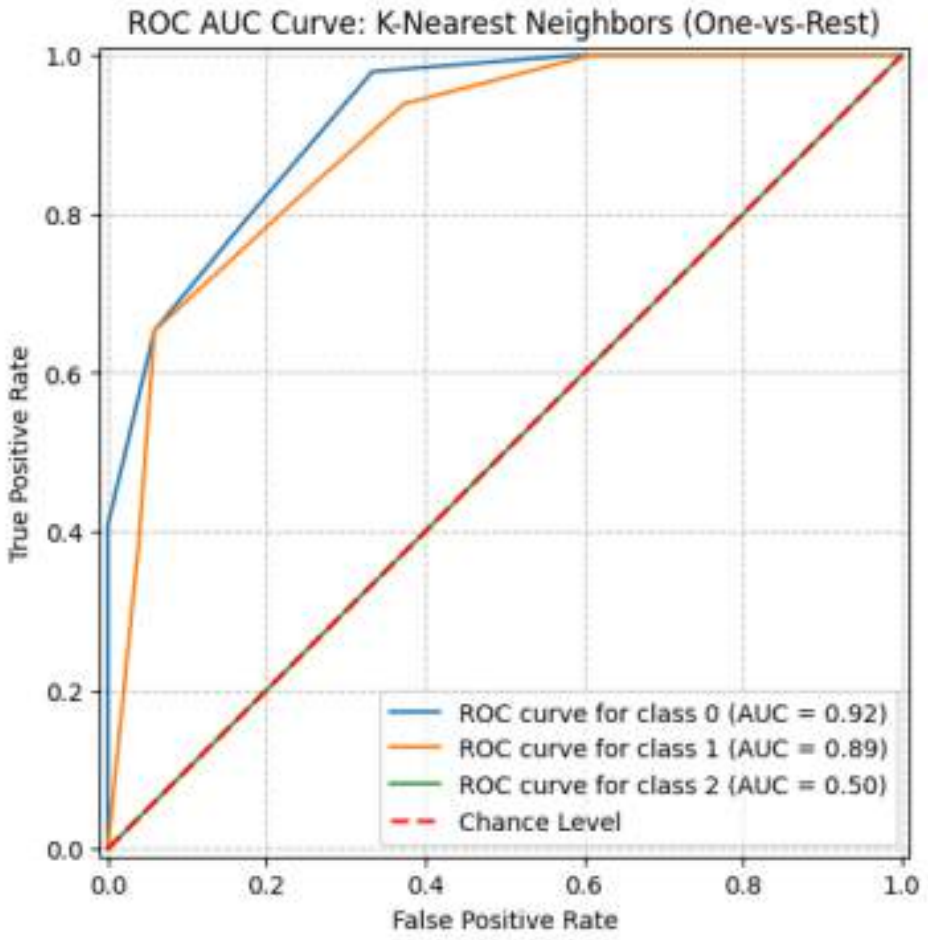
In [98]:
```
KNN_pred = knn_pred

print(f"Accuracy:  {accuracy_score(y_test, KNN_pred):.4f}")
print(f"Precision: {precision_score(y_test, KNN_pred, average='macro', zero_di
print(f"Recall:    {recall_score(y_test, KNN_pred, average='macro'):.4f}")
print(f"F1-Score:  {f1_score(y_test, KNN_pred, average='macro'):.4f}")

cm_knn = confusion_matrix(y_test, KNN_pred)
disp_knn = ConfusionMatrixDisplay(confusion_matrix=cm_knn)
disp_knn.plot(cmap='Purples')
plt.title('Confusion Matrix: KNN')
plt.show()
```

```
Accuracy:  0.7900
Precision: 0.5272
Recall:    0.5374
F1-Score:  0.5321
```

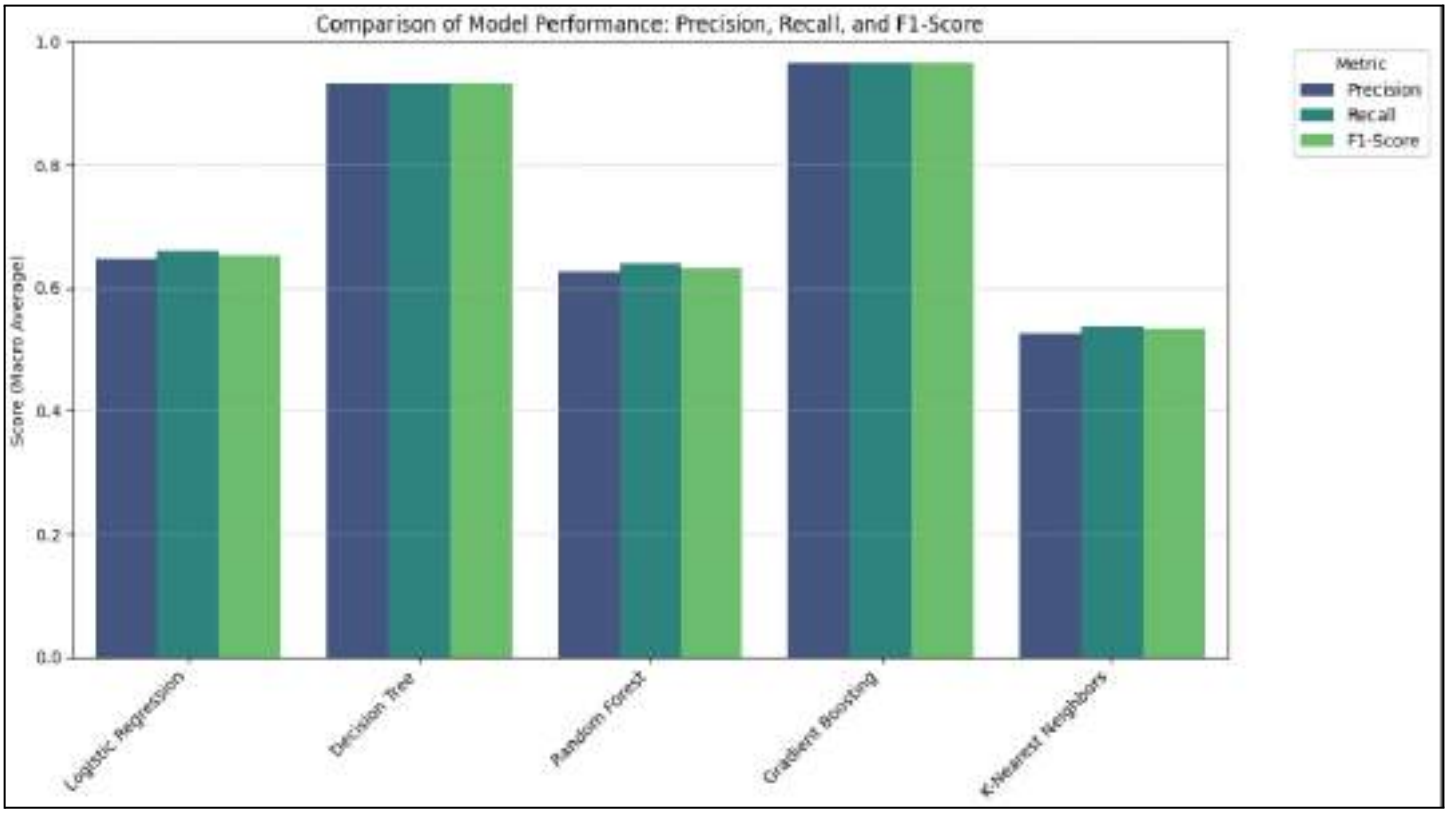


In [99]:

```
from sklearn.metrics import RocCurveDisplay
from sklearn.preprocessing import label_binarize
import numpy as np
import matplotlib.pyplot as plt
y_pred_proba_knn = knn.predict_proba(X_test_scaled)
classes = np.unique(y_test)
n_classes = len(classes)
y_test_binarized = label_binarize(y_test, classes=classes)

fig, ax = plt.subplots(figsize=(8, 6))

for i in range(n_classes):
    RocCurveDisplay.from_predictions(
        y_test_binarized[:, i],
        y_pred_proba_knn[:, i],
        name=f'ROC curve for class {classes[i]}',
        ax=ax,
    )

ax.plot([0, 1], [0, 1], linestyle='--', lw=2, color='r', label='Chance Level',
ax.set_title('ROC AUC Curve: K-Nearest Neighbors (One-vs-Rest)')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.grid(linestyle='--', alpha=0.7)
ax.legend(loc='lower right')
plt.show()
```



In [ ]:
```
import pickle

output_dir = '/content/drive/MyDrive/AIML_lab_sem4/exp 5,6,7/Exp8_models'

with open(os.path.join(output_dir, 'Logistic_Regression_model.pkl'), 'wb') as
    pickle.dump(lr, file)
with open(os.path.join(output_dir, 'Decision_Tree_model.pkl'), 'wb') as file:
    pickle.dump(dt, file)
with open(os.path.join(output_dir, 'Random_Forest_model.pkl'), 'wb') as file:
    pickle.dump(rf, file)
with open(os.path.join(output_dir, 'Gradient_Boosting_model.pkl'), 'wb') as fi
    pickle.dump(gb, file)
with open(os.path.join(output_dir, 'K_Nearest_Neighbors_model.pkl'), 'wb') as
    pickle.dump(knn, file)
```


Comparison of Model Performance: Precision, Recall, and F1-Score
Score (Macro Average)
1.0
0.8
0.6
0.4
0.2
0.0
Metric
Precision
Recall
F1-Score
Logistic Regression
Decision Tree
Random Forest
Gradient Boosting
K-Nearest Neighbors

Experiment 9
Physiological signal classification

Aim : Implement a classification model to distinguish between normal and abnormal

physiological signals using extracted signal features. (perform comparative analysis on different ML models)

Pipeline :

1.Problem Definition

The primary goal of this experiment is to automate the detection of abnormal heart rhythms (arrhythmias) which is critical for early diagnosis of cardiovascular diseases:

- Classification Task: Categorize heartbeat samples into "Normal" (0) and "Abnormal" (1).

- Objective: Perform a comparative analysis using a Decision Tree, Random Forest, and SVM, and ultimately combine them into a Voting Classifier to improve overall prediction stability and accuracy.

2. Data Understanding

- Dataset Content: The dataset consists of ECG recordings from the PTB Diagnostic Data sets, containing thousands of heartbeat samples.
- Input Features (X): 187 numerical features representing the normalized intensity of the ECG signal over a single heartbeat period.
- Target (y): Binary labels where 0 represents a Normal heartbeat and 1 represents an Abnormal heartbeat

3. Data Preprocessing

To prepare the medical signal data for the machine learning models:

- Data Integration: Combined separate CSV files for normal and abnormal cases into a single unified dataframe.
- Feature Extraction: Separated the signal data (first 187 columns) from the ground truth labels (last column).
- Class Labeling: Explicitly assigned numerical values (0 and 1) to distinguish between the two health states.s.

4. Train-Test Split

To validate the model's diagnostic reliability on unseen patients:

- Data Allocation: The combined dataset was split into a Training set (80%) to teach the models and a Test set (20%) for final validation.
- Random State: A fixed seed (42) was used to ensure the results are reproducible across different runs.

5. Model Selection and Training

Four distinct configurations were trained to compare individual performance vs. ensemble performance:

- Decision Tree: A baseline model using 'entropy' as the split criterion with a maximum depth of 10.
- Random Forest: An ensemble of 100 trees to reduce variance and improve accuracy through bagging.
- Support Vector Machine (SVM): A high-dimensional classifier used to find the optimal hyperplane between normal and abnormal signals.
- Voting Classifier (Soft Voting): A meta-classifier that aggregates the probability predictions of the three models above to make a final "consensus" decision.

6. Predictions

- Each trained model was used to predict the labels for the unseen samples in the test set.
- The predicted data is further used in evaluation of the model.

7. Model Evaluation and Comparison

The performance was evaluated using Precision, Recall, and F1-Score, specifically focusing on the model's ability to identify abnormal cases correctly.

	Accuracy	Precision	Recall	F1
Decision tree	0.8952	0.9355	0.9182	0.9268
Random forest	0.9705	0.9701	0.9895	0.9797
SVM	0.9024	0.9139	0.9548	0.9339
Voting ensemble	0.9443	0.9567	0.9667	0.9617

Auc-roc curve:

Decision Tree	Randon Forest	SVM	Voting Ensemble
0.93	0.99	0.94	0.98

8. Model Saving

The final step ensures the diagnostic tool is ready for clinical application:

- Serialization: The trained Voting Classifier was saved into a .pkl file using the Pickle library.
- Deployment Ready: This file allows medical software to load the model and analyze live ECG streams without needing to retrain the algorithms.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, VotingClassifier, Bagging
from sklearn.svm import SVC
```

In [3]:
```
train_df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/Exp 9 , 10/mitbih
test_df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/Exp 9 , 10/mitbih_
df_normal = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/Exp 9 , 10/ptbdb
df_abnormal = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/Exp 9 , 10/ptb
```

In [4]:
```
df_normal[187] = 0
df_abnormal[187] = 1
```

In [5]:
```
df_combined = pd.concat([df_normal, df_abnormal], axis=0)
X = df_combined.iloc[:, :-1].values
y = df_combined.iloc[:, -1].values
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [7]:
```
dt_model = DecisionTreeClassifier(criterion='entropy', max_depth=10, random_st
```

In [8]:
```
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
```

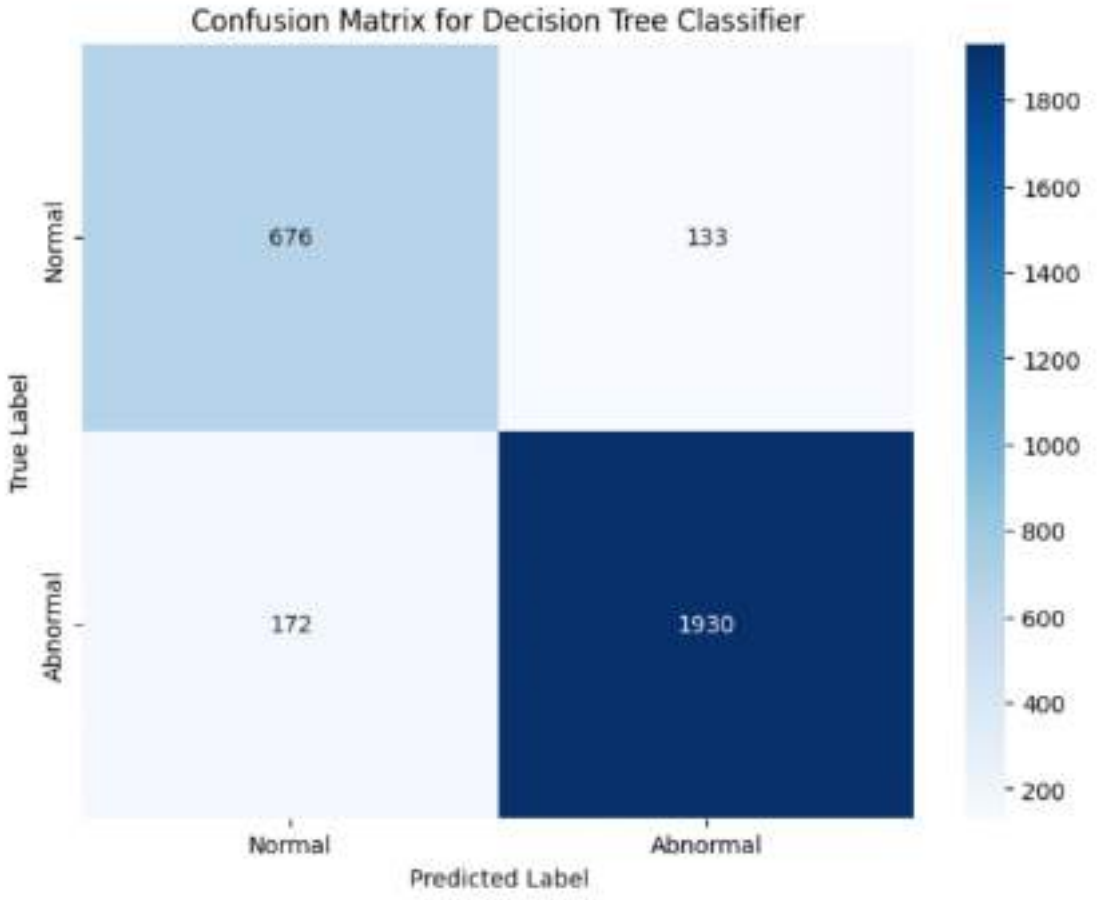
In [9]:
```
print(classification_report(y_test, y_pred_dt))
```

```
              precision    recall  f1-score   support

           0       0.80      0.84      0.82       809
           1       0.94      0.92      0.93      2102

    accuracy                           0.90      2911
   macro avg       0.87      0.88      0.87      2911
weighted avg       0.90      0.90      0.90      2911
```

In [10]:
```
print(f"Accuracy:  {accuracy_score(y_test, y_pred_dt):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_dt, zero_division=0):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred_dt):.4f}")
print(f"F1-Score:  {f1_score(y_test, y_pred_dt):.4f}")
```

```
Accuracy:  0.8952
Precision: 0.9355
Recall:    0.9182
F1-Score:  0.9268
```

In [11]:
```
cm = confusion_matrix(y_test, y_pred_dt)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Normal', 'Abr
plt.title('Confusion Matrix for Decision Tree Classifier')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



In [12]:
```
rf_model = RandomForestClassifier(n_estimators=100, criterion='gini', n_jobs=-
```

In [13]:
```
rf_model.fit(X_train, y_train)
```

Out[13]:
```
RandomForestClassifier(n_jobs=-1, random_state=42)
```

In [14]:
```
y_pred_rf = rf_model.predict(X_test)
```

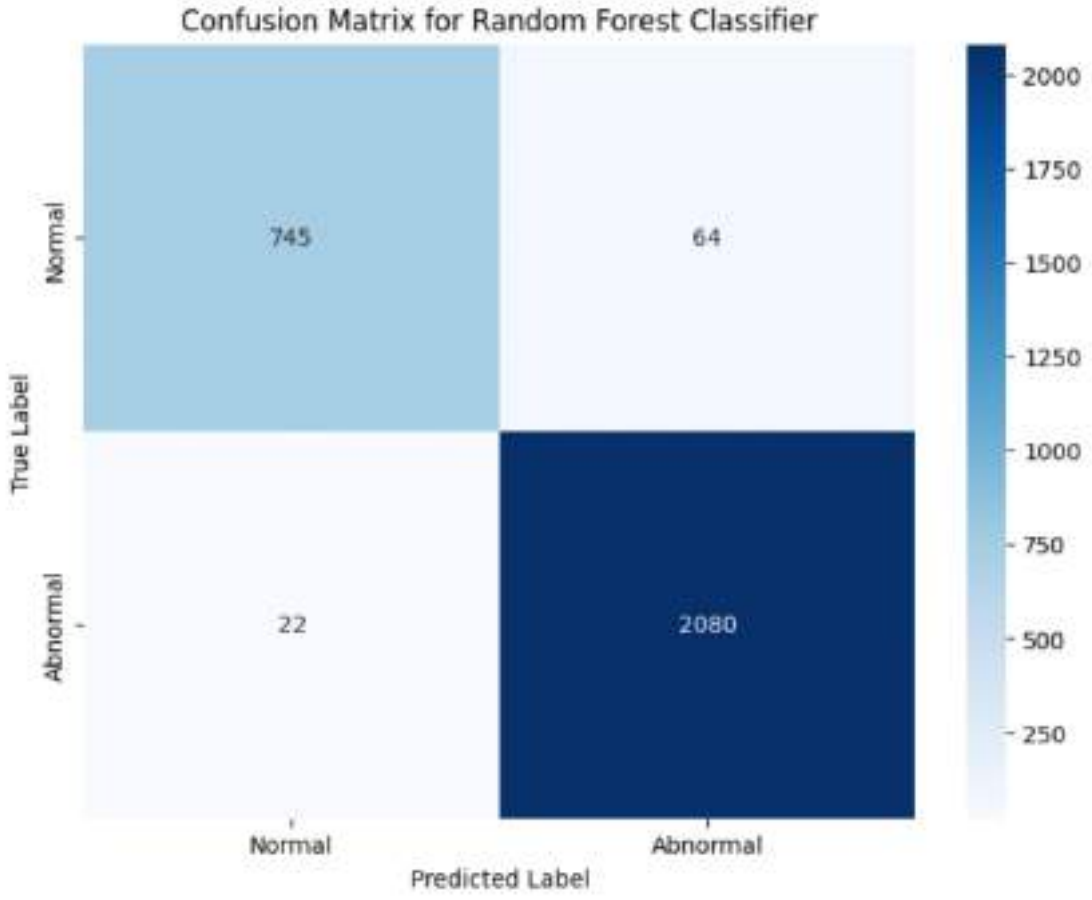
In [15]:
```
print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support

           0       0.97      0.92      0.95       809
           1       0.97      0.99      0.98      2102

    accuracy                           0.97      2911
   macro avg       0.97      0.96      0.96      2911
weighted avg       0.97      0.97      0.97      2911
```

In [16]:
```
print(f"Accuracy:  {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_rf, zero_division=0):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred_rf):.4f}")
print(f"F1-Score:  {f1_score(y_test, y_pred_rf):.4f}")
```

```
Accuracy:  0.9705
Precision: 0.9701
Recall:    0.9895
F1-Score:  0.9797
```

In [17]:
```
cm_rf = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues', xticklabels=['Normal', '
plt.title('Confusion Matrix for Random Forest Classifier')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



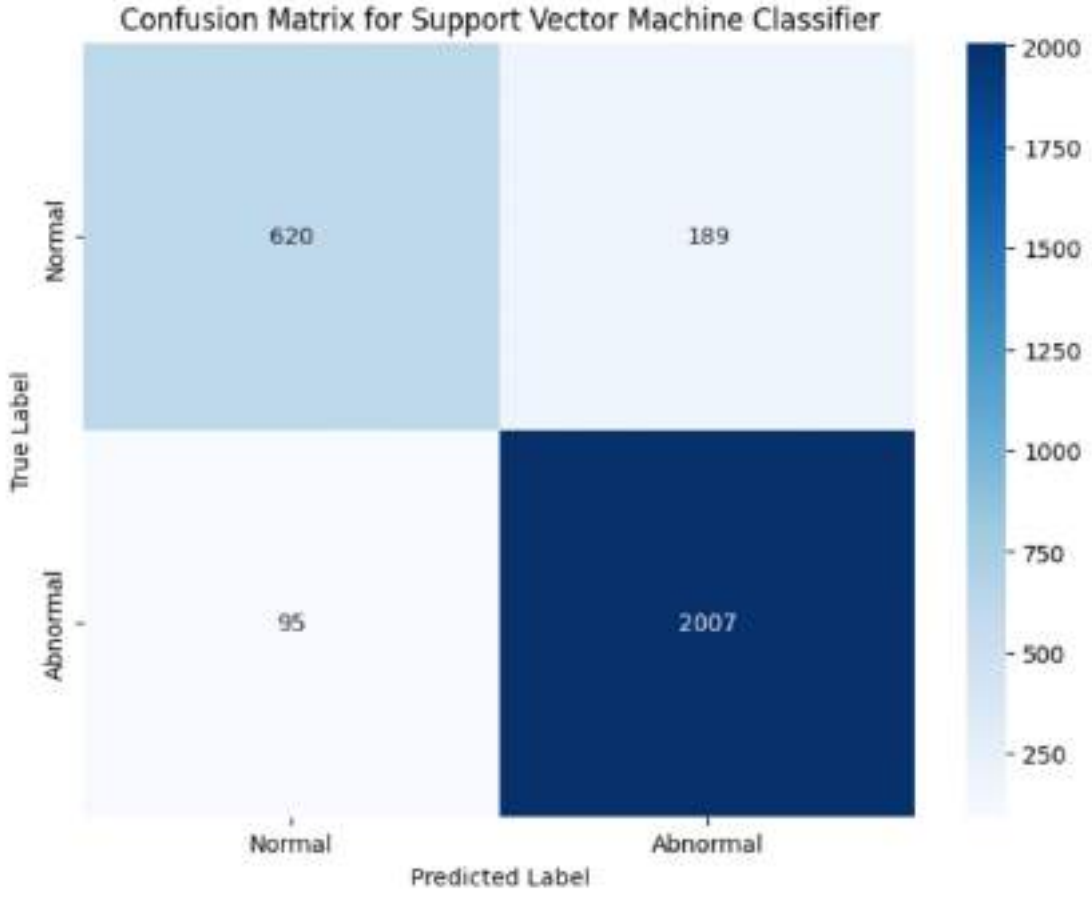

```
In [18]: svm_model = SVC(random_state=42)

In [19]: svm_model.fit(X_train, y_train)

Out[19]: SVC(random_state=42)

In [20]: y_pred_svm = svm_model.predict(X_test)

In [21]: print(classification_report(y_test, y_pred_svm))
```

	precision	recall	f1-score	support
0	0.87	0.77	0.81	809
1	0.91	0.95	0.93	2102
accuracy			0.90	2911
macro avg	0.89	0.86	0.87	2911
weighted avg	0.90	0.90	0.90	2911

In [22]:
```
print(f"Accuracy:  {accuracy_score(y_test, y_pred_svm):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_svm, zero_division=0):.4f}"
print(f"Recall:    {recall_score(y_test, y_pred_svm):.4f}")
print(f"F1-Score:  {f1_score(y_test, y_pred_svm):.4f}")
```

```
Accuracy:  0.9024
Precision: 0.9139
Recall:    0.9548
F1-Score:  0.9339
```

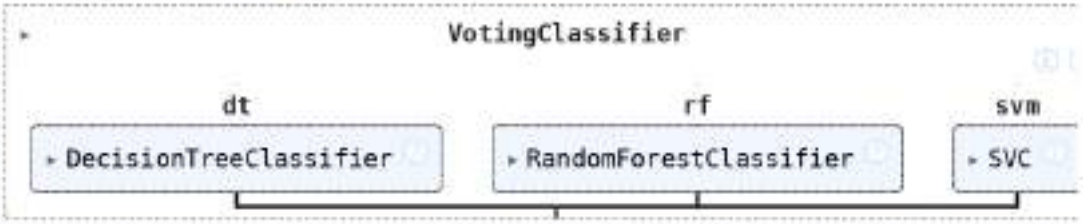
In [23]:
```
cm_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_svm, annot=True, fmt='d', cmap='Blues', xticklabels=['Normal',
plt.title('Confusion Matrix for Support Vector Machine Classifier')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



In [24]:
```
voting_classifier = VotingClassifier(estimators=[
    ('dt', DecisionTreeClassifier(criterion='entropy', max_depth=10, random_st
    ('rf', RandomForestClassifier(n_estimators=100, criterion='gini', n_jobs=-
    ('svm', SVC(random_state=42, probability=True))
], voting='soft', n_jobs=-1)
```

In [25]:
```
voting_classifier.fit(X_train, y_train)
```

Out[25]:



In [26]:
```
y_pred_voting = voting_classifier.predict(X_test)
```

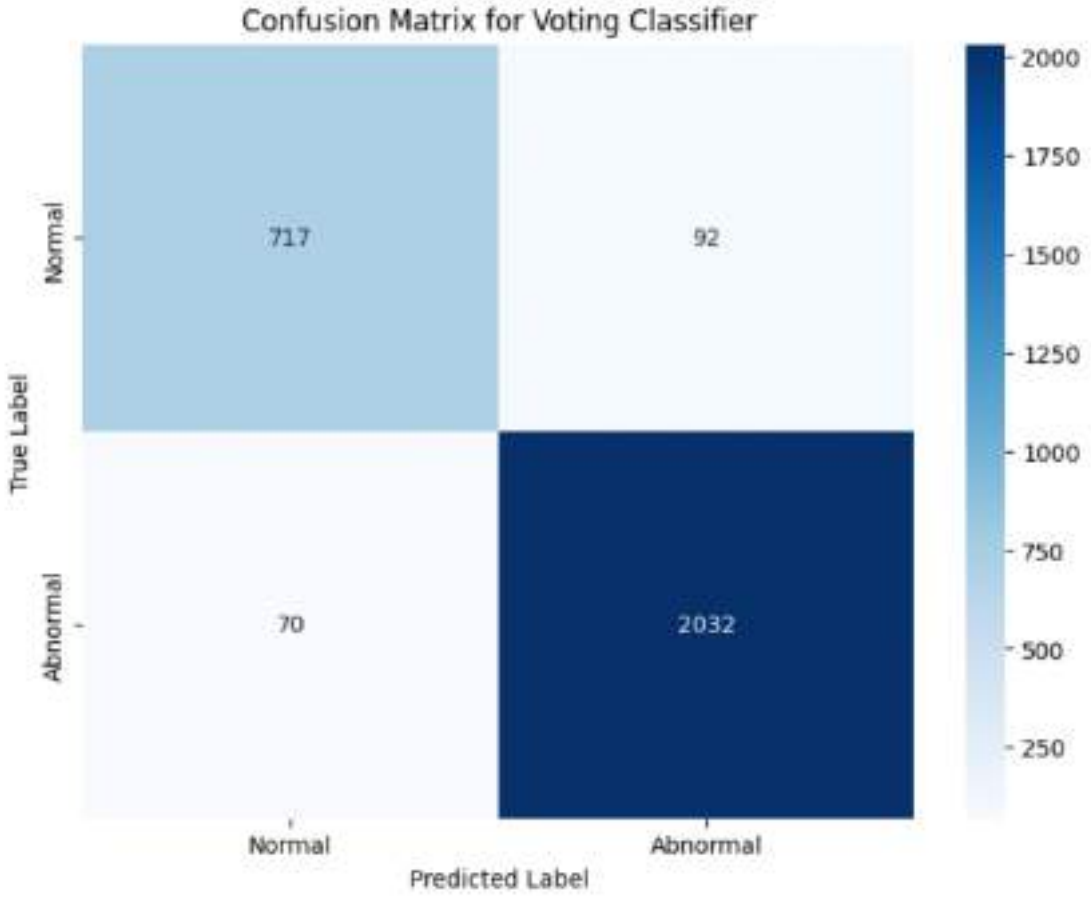
In [27]:
```
print(classification_report(y_test, y_pred_voting))
```

```
              precision    recall  f1-score   support

           0       0.91      0.89      0.90       809
           1       0.96      0.97      0.96      2102

    accuracy                           0.94      2911
   macro avg       0.93      0.93      0.93      2911
weighted avg       0.94      0.94      0.94      2911
```

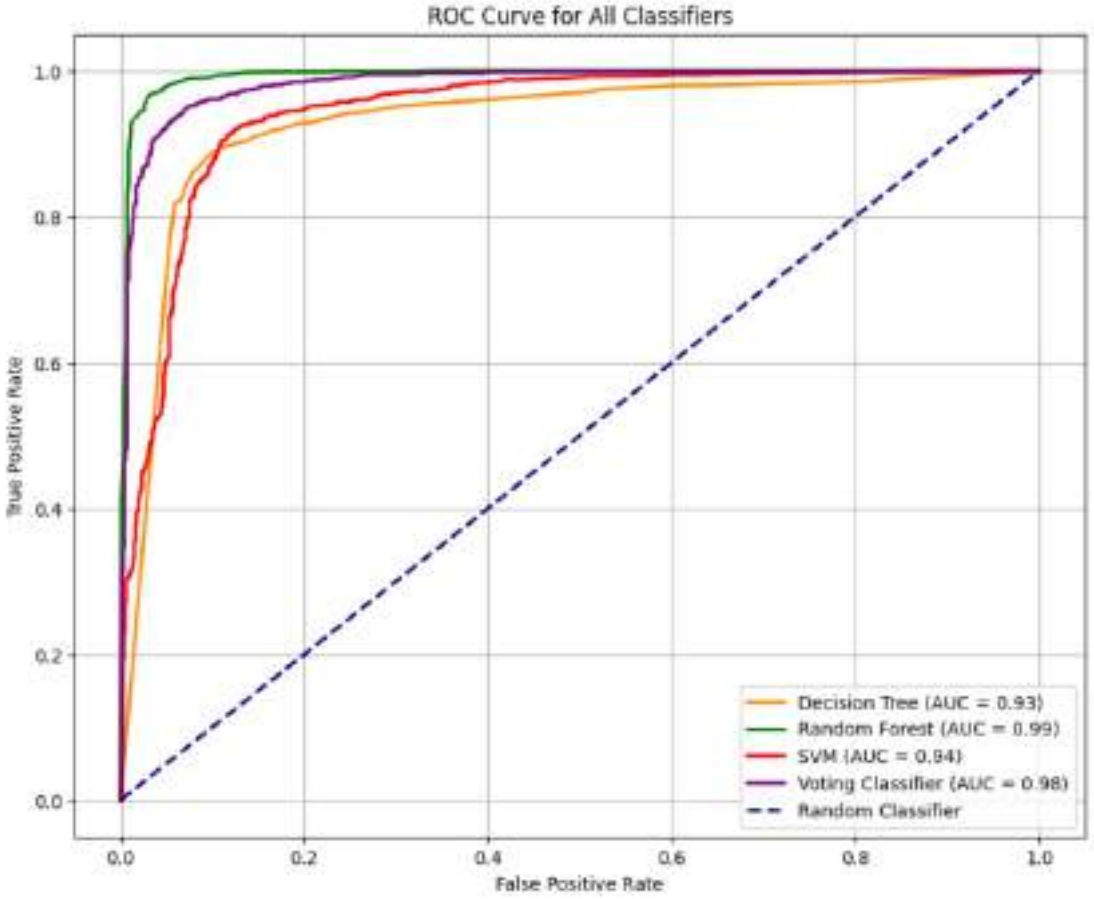
In [28]:
```
print(f"Accuracy:  {accuracy_score(y_test, y_pred_voting):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_voting, zero_division=0):.4
print(f"Recall:    {recall_score(y_test, y_pred_voting):.4f}")
print(f"F1-Score:  {f1_score(y_test, y_pred_voting):.4f}")
```

```
Accuracy:  0.9443
Precision: 0.9567
Recall:    0.9667
F1-Score:  0.9617
```

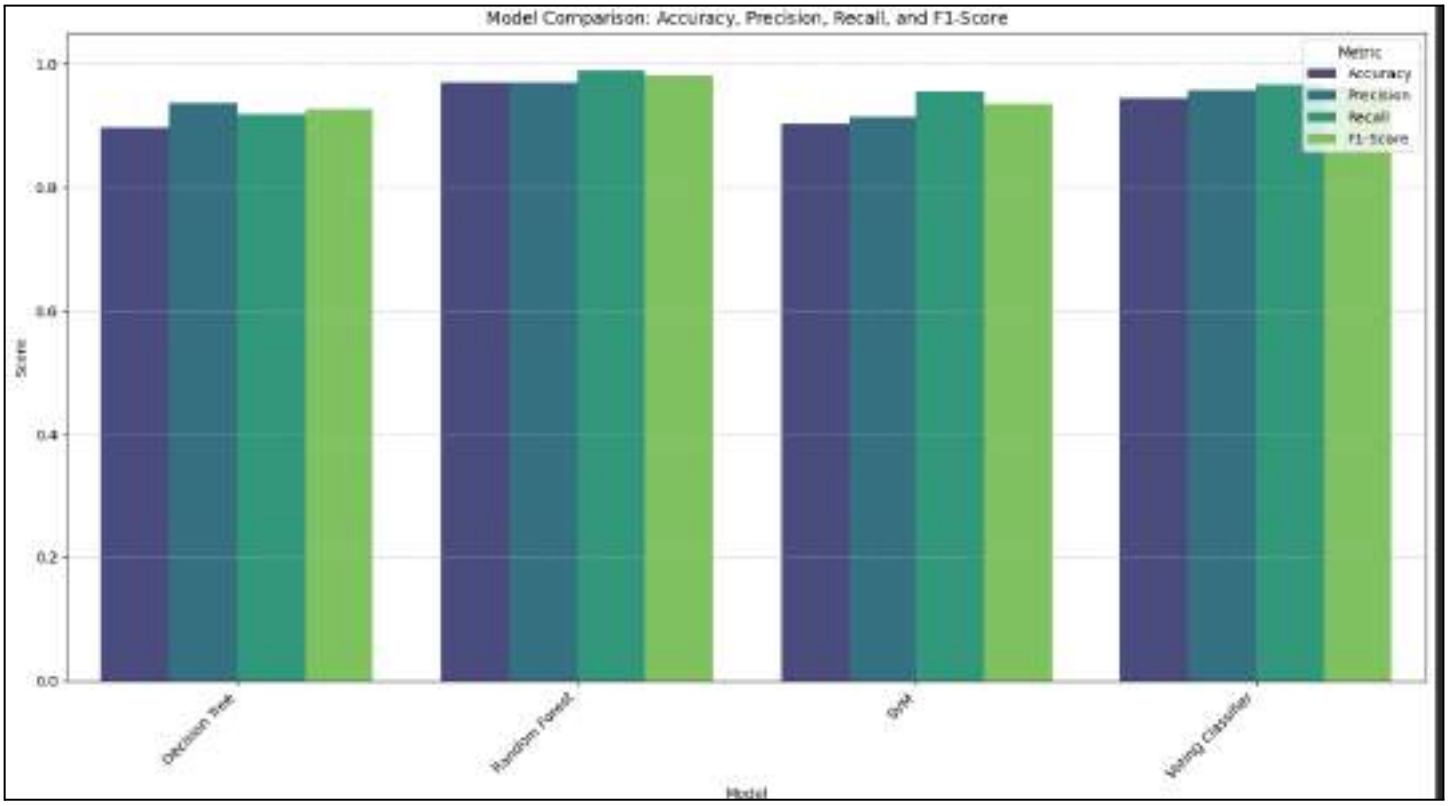
In [29]:
```
cm_voting = confusion_matrix(y_test, y_pred_voting)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_voting, annot=True, fmt='d', cmap='Blues', xticklabels=['Normal
plt.title('Confusion Matrix for Voting Classifier')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```



In [36]:
```
plt.figure(figsize=(10, 8))

plt.plot(fpr_dt, tpr_dt, color='darkorange', lw=2, label=f'Decision Tree (AUC
plt.plot(fpr_rf, tpr_rf, color='green', lw=2, label=f'Random Forest (AUC = {rc
plt.plot(fpr_svm, tpr_svm, color='red', lw=2, label=f'SVM (AUC = {roc_auc_svm:
plt.plot(fpr_voting, tpr_voting, color='purple', lw=2, label=f'Voting Classifi

plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Cla
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for All Classifiers')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



In [30]:
```
import pickle

filename = '/content/drive/MyDrive/AIML_lab_sem4/Exp 9 , 10/voting_classifier_

with open(filename, 'wb') as file:
    pickle.dump(voting_classifier, file)
```


Model Comparison: Accuracy, Precision, Recall, and F1-Score
Metric
Accuracy
Precision
Recall
F1-Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
Decision Tree
Random Forest
SVM
Voting Classifier
Model

Experiment - 10
Iris Classification

Aim : Iris Flower Classification: Use the Iris dataset to build a classification model that predicts the species of iris flowers. [Dataset: Load dataset from sk-learn] .

Pipeline :

1. Problem Definition

The primary goal of this experiment is to develop a robust automated classification system to identify biological species based on physical measurements:

- Classification Task: We must determine if a data point belongs to the *Setosa*, *Versicolor*, or *Virginica* class.
- Objective: We perform a detailed comparative analysis across three distinct algorithms Decision Tree, K-Nearest Neighbors (KNN), and Logistic Regression to identify the most accurate architecture for multiclass classification.

2. Data Understanding

- Dataset Content: We utilize the classic Iris dataset consisting of 150 samples with four physical attributes: sepal length, sepal width, petal length, and petal width.
- Input Features (X): We analyze numerical data representing the four flower measurements.
- Target (y): We use labels where 0, 1, and 2 represent the three specific species of iris.

3. Data Preprocessing

To ensure the data is suitable for algorithmic processing, we perform the following:

- Data Formatting: We separate the dataset into a feature matrix (x) and a target vector (y).
- Cleanliness: As the Scikit-Learn iris dataset is a standard benchmark, the data is verified to be clean and correctly formatted for supervised learning.

4. Train-Test Split

To evaluate how well the system identifies species in new samples, we strategically divide the dataset:

- Data Allocation: We split the dataset into a Training set (80%) to allow the models to learn the patterns and a Test set (20%) to validate accuracy.
- Reproducibility: We use a random_state of 42 to ensure that the data split remains consistent across different experimental runs.

5. Model Selection and Training

We initialize and train three different classification models on the same data to ensure a fair comparison:

- Decision Tree Classifier: We use this tree-based model that breaks down the dataset into smaller subsets while at the same time an associated decision tree is incrementally developed.
- K-Nearest Neighbors (KNN): We implement this instance-based method that classifies a point based on the majority class of its nearest neighbors in the feature space.
- Logistic Regression: We task this linear model with learning the probability of a sample belonging to each of the three species classes.

6. Predictions

- Execution: Once training is complete, we task each model with classifying the 30 unseen samples in our test set.
- Output Generation: We generate prediction arrays where each algorithm flags points as either class 0, 1, or 2.

7. Model Evaluation and Comparison

The performance of each model is measured using a Confusion Matrix for visual verification and a Classification Report containing Precision, Recall, and F1-Score. Based on the experimental outcomes:

	Accuracy	Precision	Recall	F1
Decision Tree	1.00	1.00	1.00	1.00
KNN	1.00	1.00	1.00	1.00
Logistic Regression	1.00	1.00	1.00	1.00

As the data is very simple and got perfect scores in all the models the auc-roc of all the models was also 1.00.

8. Model Saving

The final step is ensuring our models are persistent for future deployment:

- Implementation: We save the top-performing trained model (Logistic Regression) using the Pickle library to serialize the object into a permanent .pkl file.
- Final Output: We generate the file logistic_iris.pkl to allow for immediate deployment in an automated classification system without the need for retraining.

In [1]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
```

In [2]:
```
iris = load_iris()
X = iris.data
y = iris.target
```

In [3]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
```

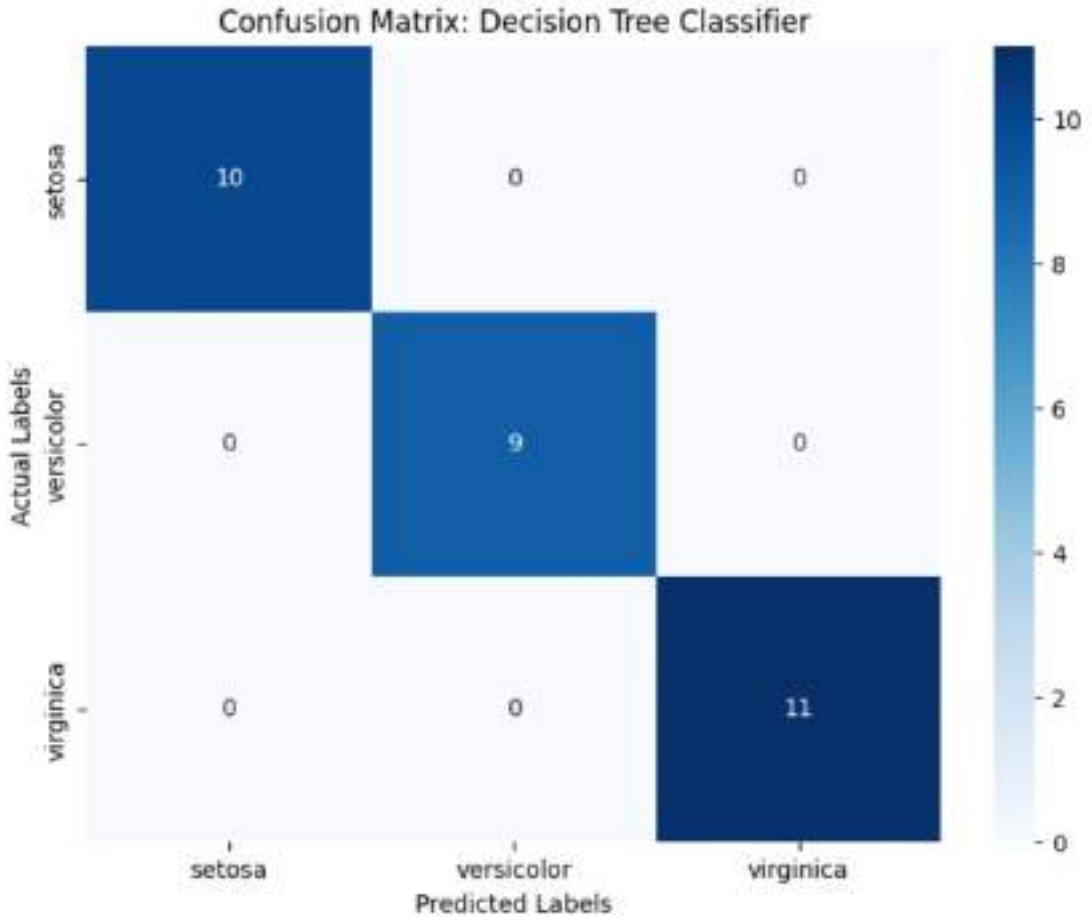
In [4]:
```
dt_model = DecisionTreeClassifier()
```

In [5]:
```
dt_model.fit(X_train, y_train)
```

Out[5]:
```
▾DecisionTreeClassifier
DecisionTreeClassifier()
```

In [6]:
```
y_pred = dt_model.predict(X_test)
```

In [7]:
```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
plt.xlabel('Predicted Labels')
plt.ylabel('Actual Labels')
plt.title('Confusion Matrix: Decision Tree Classifier')
plt.show()
```



In [8]:
```
from sklearn.metrics import classification_report
```

In [9]:
```
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

In [10]:
```
from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)
```

Out[10]:

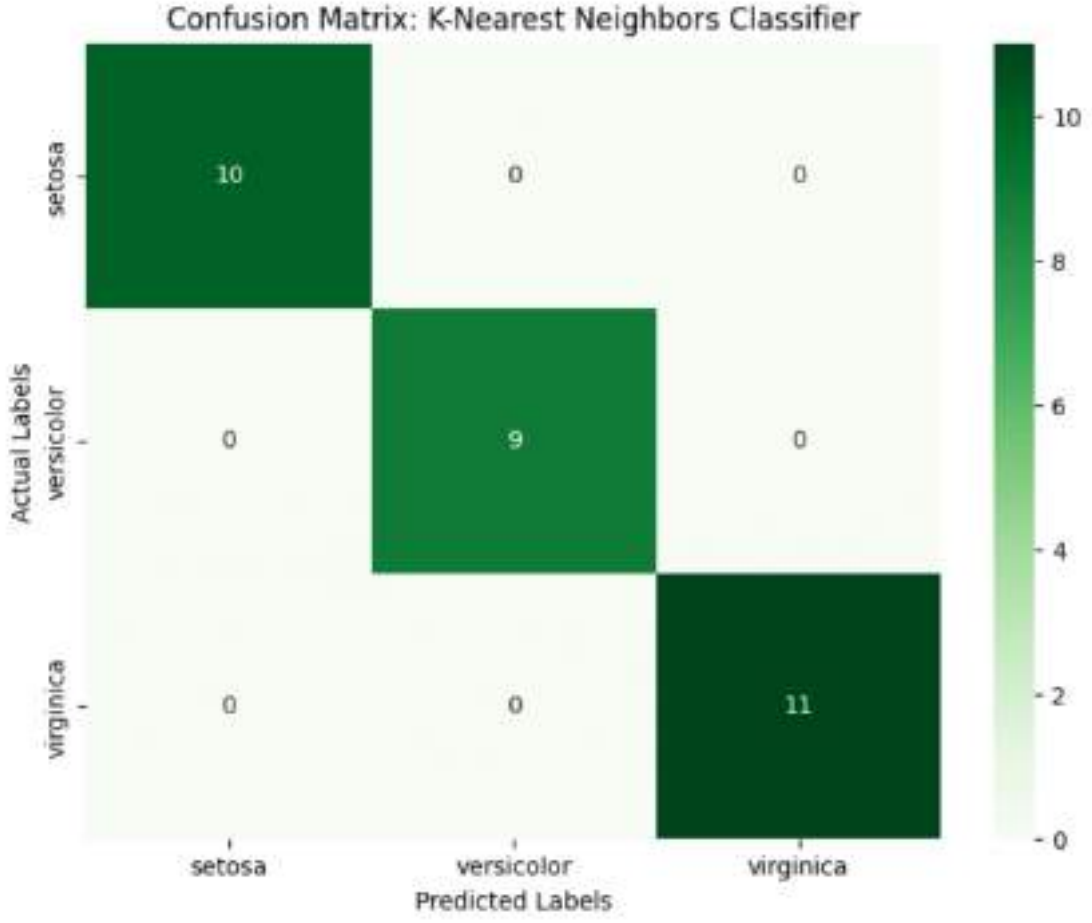
```
KNeighborsClassifier()
```

In [11]:

```
y_pred_knn = knn_model.predict(X_test)
```

In [12]:

```
cm_knn = confusion_matrix(y_test, y_pred_knn)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Greens',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
plt.xlabel('Predicted Labels')
plt.ylabel('Actual Labels')
plt.title('Confusion Matrix: K-Nearest Neighbors Classifier')
plt.show()
```



In [13]:

```
print(classification_report(y_test, y_pred_knn, target_names=iris.target_names
```

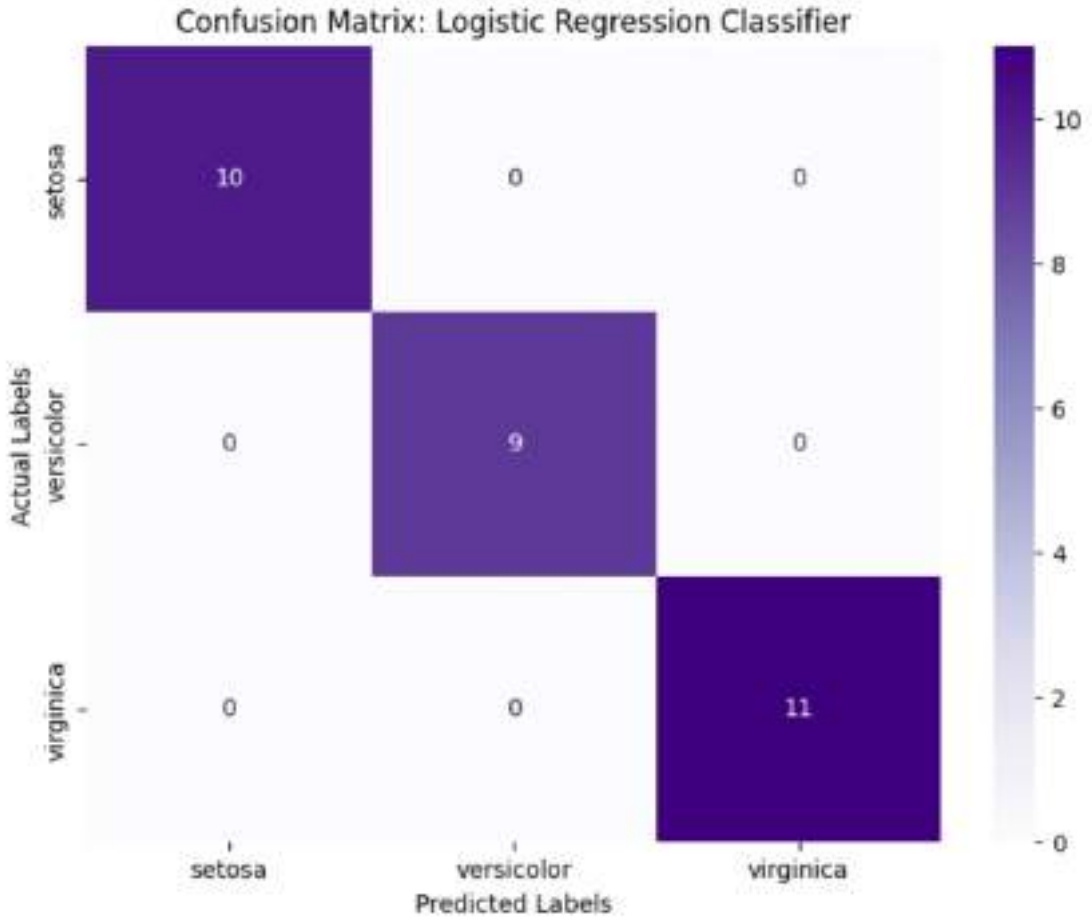
```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

In [14]:
```
from sklearn.linear_model import LogisticRegression
logreg_model = LogisticRegression(max_iter=200, random_state=42)
logreg_model.fit(X_train, y_train)
```

Out[14]:
```
LogisticRegression(max_iter=200, random_state=42)
```

In [15]:
```
y_pred_logreg = logreg_model.predict(X_test)
```

In [16]:
```
cm_logreg = confusion_matrix(y_test, y_pred_logreg)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_logreg, annot=True, fmt='d', cmap='Purples',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
plt.xlabel('Predicted Labels')
plt.ylabel('Actual Labels')
plt.title('Confusion Matrix: Logistic Regression Classifier')
plt.show()
```



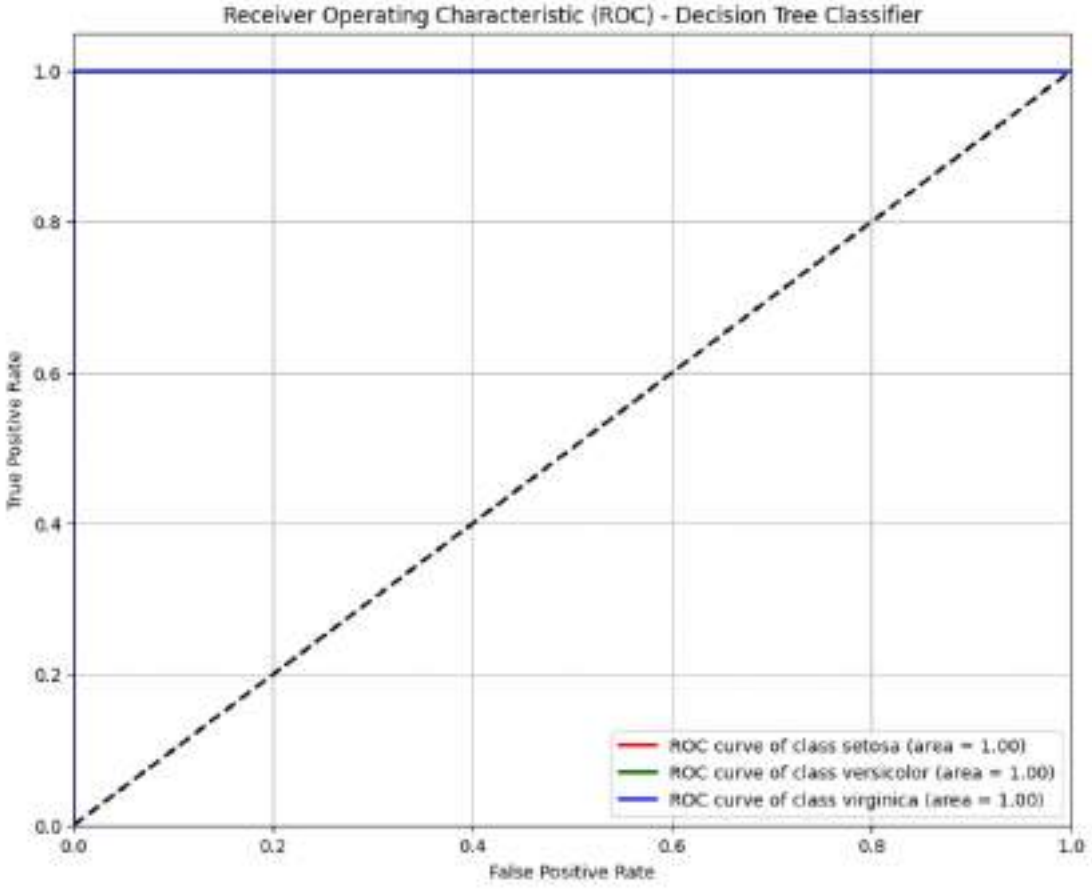
In [17]:
```
print(classification_report(y_test, y_pred_logreg, target_names=iris.target_na
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

In [19]:
```
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize
import numpy as np

# Binarize the output for ROC AUC calculation (one-vs-rest)
y_test_bin = label_binarize(y_test, classes=np.unique(y))
n_classes = y_test_bin.shape[1]
```

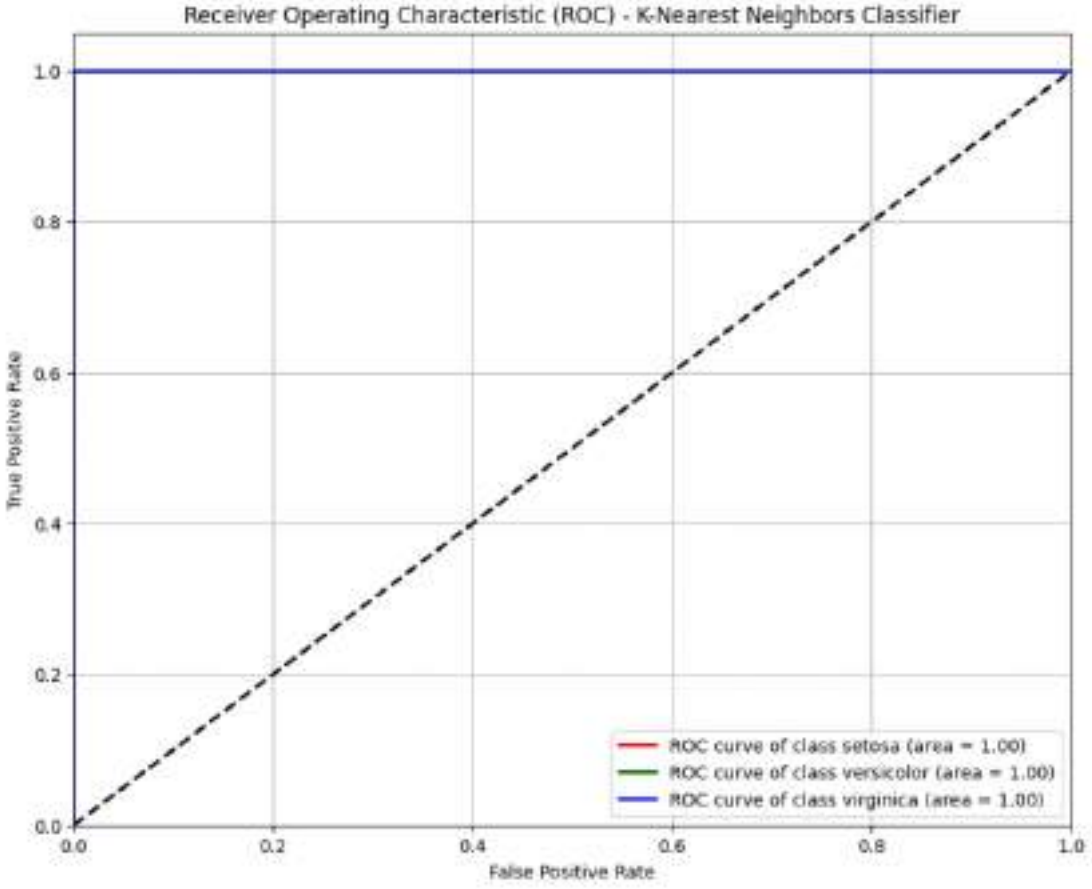
ROC AUC Curve for Decision Tree Classifier

In [20]:
```
y_prob_dt = dt_model.predict_proba(X_test)

plt.figure(figsize=(10, 8))
colors = ['red', 'green', 'blue']
for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob_dt[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, color=colors[i], lw=2, label=f'ROC curve of class {iris

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) - Decision Tree Classifier'
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



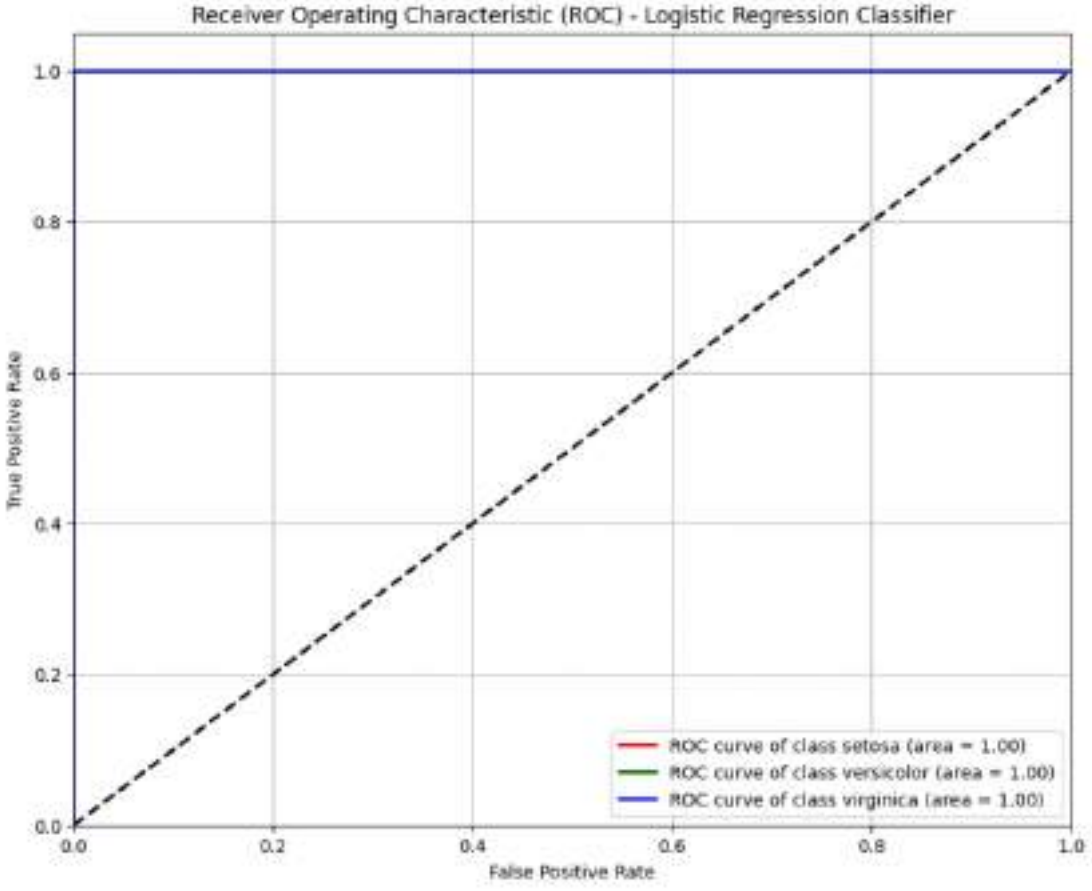
ROC AUC Curve for K-Nearest Neighbors Classifier

In [21]:
```
y_prob_knn = knn_model.predict_proba(X_test)

plt.figure(figsize=(10, 8))
colors = ['red', 'green', 'blue']
for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob_knn[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, color=colors[i], lw=2, label=f'ROC curve of class {iris

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) - K-Nearest Neighbors Class
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



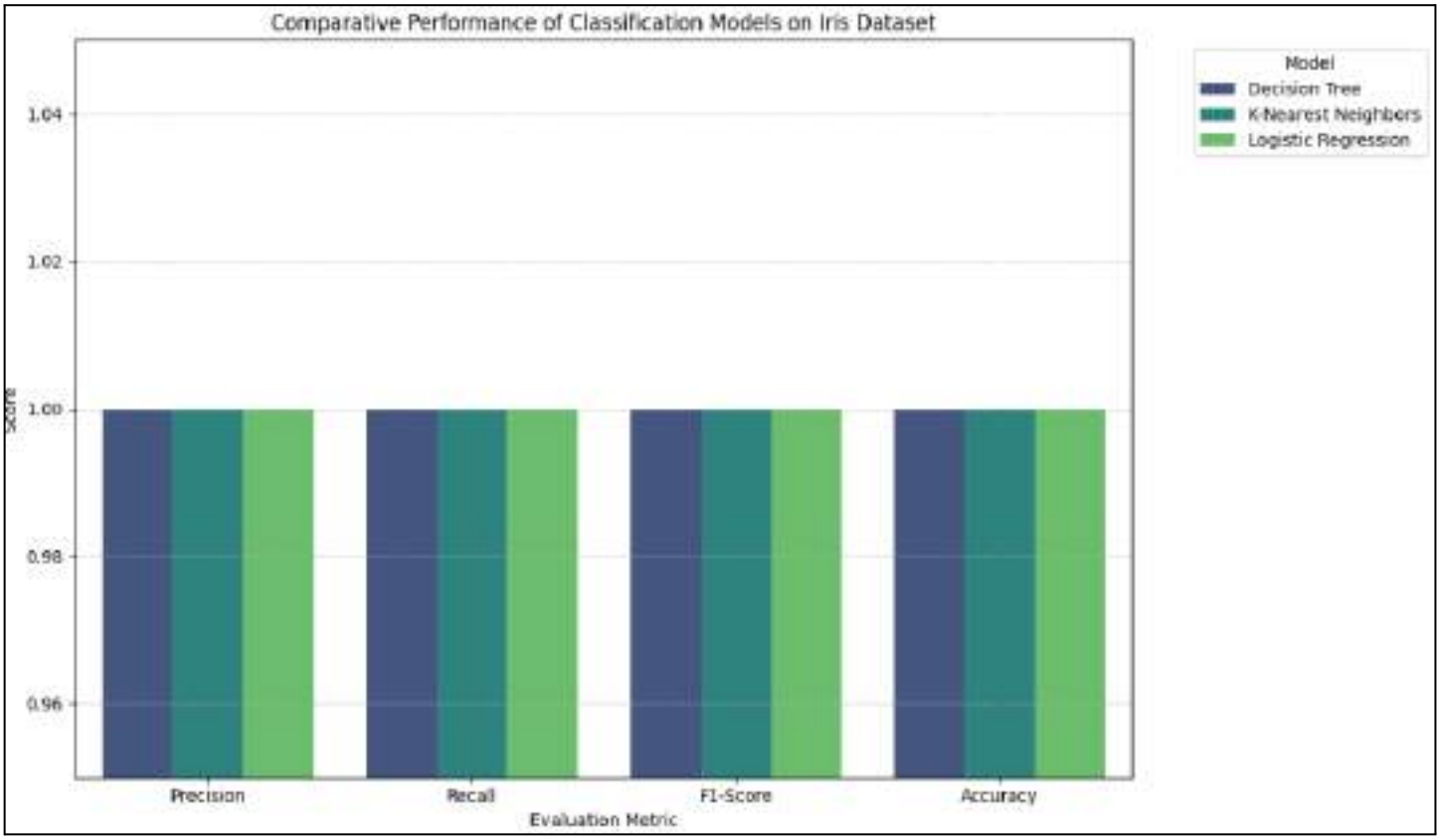
ROC AUC Curve for Logistic Regression Classifier

In [22]:
```
y_prob_logreg = logreg_model.predict_proba(X_test)

plt.figure(figsize=(10, 8))
colors = ['red', 'green', 'blue']
for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob_logreg[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, color=colors[i], lw=2, label=f'ROC curve of class {iris

plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) - Logistic Regression Class
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



In [18]:
```
import pickle
```

```
model_filename = '/content/drive/MyDrive/AIML_lab_sem4/Exp 9 , 10/logistic_iri

with open(model_filename, 'wb') as file:
    pickle.dump(logreg_model, file)
```



Experiment 11
Diabetes prediction using bagging classifier

Aim = To predict whether a person has diabetes based on features such as blood pressure, skin thickness, age, etc., using the bagging ensemble technique. Also perform comparative analysis among the bagging classifier, random forest, and the decision tree classifier.

Pipeline =>

1. Problem Definition

The goal of this experiment is to implement an automated classification system to predict student diabetes outcomes based on diagnostic metrics.

- Classification Task: This is a binary classification problem where samples are categorized into two distinct classes (Outcome 0 or 1).

- Objective: Perform a comparative analysis between a single Decision Tree and ensemble methods, specifically Random Forest and Bagging, to determine which architecture offers the most reliable predictive performance.

2. Data Understanding

The experiment utilizes a diabetes dataset containing several medical indicator features:

- Input Features (X): Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, and Age.
- Target (y): Outcome (Binary labels where 1 indicates the presence of diabetes and 0 indicates absence).

3. Data Preprocessing

To prepare the data for training, the following cleaning and transformation steps were executed:

- Missing Value Handling: Columns containing invalid zero values (Glucose, BloodPressure, SkinThickness, Insulin, BMI) were identified. These zeros were replaced with NaN and subsequently imputed using the median value of each column.

- Feature-Target Separation: The dataset was divided into input features (X) and the target variable (y).

- Standardization: The StandardScaler was applied to normalize the feature set, ensuring that variables with different scales do not bias the algorithms.

4. Train-Test Split

- Data Allocation: The dataset was partitioned into a Training set (80%) to build the models and a Test set (20%) to validate performance on unseen data.

- Random State: A fixed random_state=42 was used to ensure reproducibility of the data split and model results.

5. Model Selection and Training

Three distinct classification architectures were initialized and trained:

- Decision Tree: A base non-linear model that creates decision rules based on feature thresholds.
- Random Forest: An ensemble technique that builds multiple decision trees on different data subsets and averages their results to improve accuracy and control overfitting.
- Bagging Classifier: An ensemble method that uses a DecisionTreeClassifier as the base estimator, training 200 different versions of the model on boosted samples to reduce variance.
- The all three models are trained on the training data.

6. Predictions

- The standardized test set was processed by all three trained models to generate predicted labels for the student samples.

- The models transformed medical input patterns into binary classifications (0 or 1), which were then compared against the actual ground-truth outcomes from the test set.

7. Model Evaluation and Comparison

The models were evaluated using a suite of classification metrics and visualization tools:

	Accuracy	Precision	Recall	F1
Decision Tree	0.71	0.59	0.61	0.60
Random Forest	0.74	0.63	0.65	0.64
Bagging(Decision Tree)	0.77	0.67	0.70	0.69

Auc-Roc Score:

Decision Tree	Random Forest	Bagging
0.69	0.83	0.83

By looking at the results we can conclude that the Bagging classifier out performs random forest and decision tree in all the metrics.

8. Model Saving

To ensure the models are ready for deployment or future use without retraining:

- Serialization: All three trained models (Decision Tree, Random Forest, and Bagging Classifier) were exported into .pkl files using the pickle library.
- Utility: These files allow for the immediate reloading of the predictive system into a production environment for real-time analysis.

In [2]:
```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
```

In [3]:
```
df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/exp 11,12/dataset_diabe
```

In [4]:
```
df.head()
```

Out[4]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesP
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

In [5]:
```
df.shape
```

Out[5]:
```
(768, 9)
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

In [7]:
```
cols_with_zeros = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BM
df[cols_with_zeros] = df[cols_with_zeros].replace(0, np.nan)
df.fillna(df.median(), inplace=True)
```

In [8]:
```
scaler = StandardScaler()
X = scaler.fit_transform(df.drop('Outcome', axis=1))
y = df['Outcome']
```

In [9]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc
```

In [10]:
```
dt= DecisionTreeClassifier(random_state=42)
```

In [11]:
```
dt.fit(X_train, y_train)
```

Out[11]:

DecisionTreeClassifier

DecisionTreeClassifier(random_state=42)

In [12]:
```
y_pred_dt = dt.predict(X_test)
```

In [13]:
```
accuracy = accuracy_score(y_test, y_pred_dt)
precision = precision_score(y_test, y_pred_dt)
recall = recall_score(y_test, y_pred_dt)
f1 = f1_score(y_test, y_pred_dt)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

```
Accuracy: 0.7143
Precision: 0.5965
Recall: 0.6182
F1 Score: 0.6071
```

In [14]:
```
print(classification_report(y_test, y_pred_dt))
```

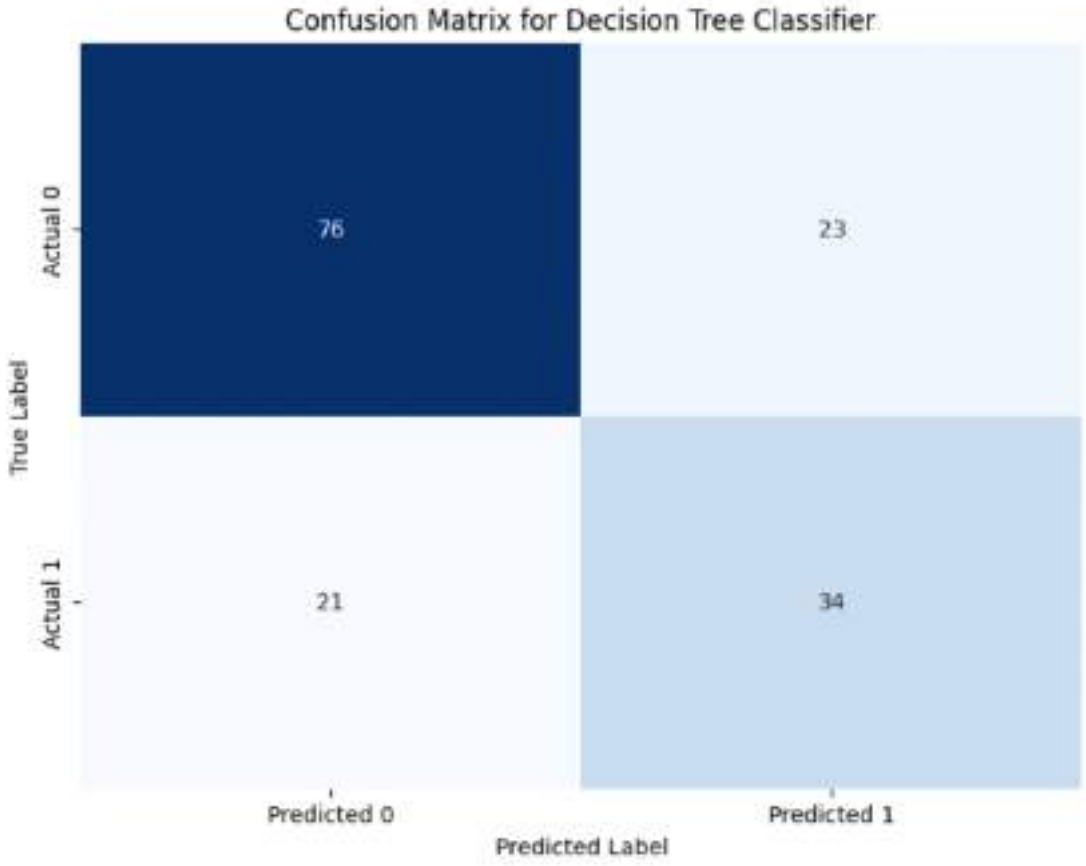
```
              precision    recall  f1-score   support

           0       0.78      0.77      0.78        99
           1       0.60      0.62      0.61        55

    accuracy                           0.71       154
   macro avg       0.69      0.69      0.69       154
weighted avg       0.72      0.71      0.72       154
```

In [15]:
```
from sklearn.metrics import confusion_matrix
cm_dt = confusion_matrix(y_test, y_pred_dt)
```

In [16]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

In [17]:
```
plt.figure(figsize=(8, 6))
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Blues', cbar=False,
```

```
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for Decision Tree Classifier')
plt.show()
```



In [18]:
```
rf = RandomForestClassifier(n_estimators = 100 ,random_state=42)
rf.fit(X_train, y_train)
```

Out[18]:
```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

In [19]:
```
y_pred_rf = rf.predict(X_test)
```

In [20]:
```
cm_rf = confusion_matrix(y_test, y_pred_rf)
```

In [21]:
```
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf)
recall_rf = recall_score(y_test, y_pred_rf)
```

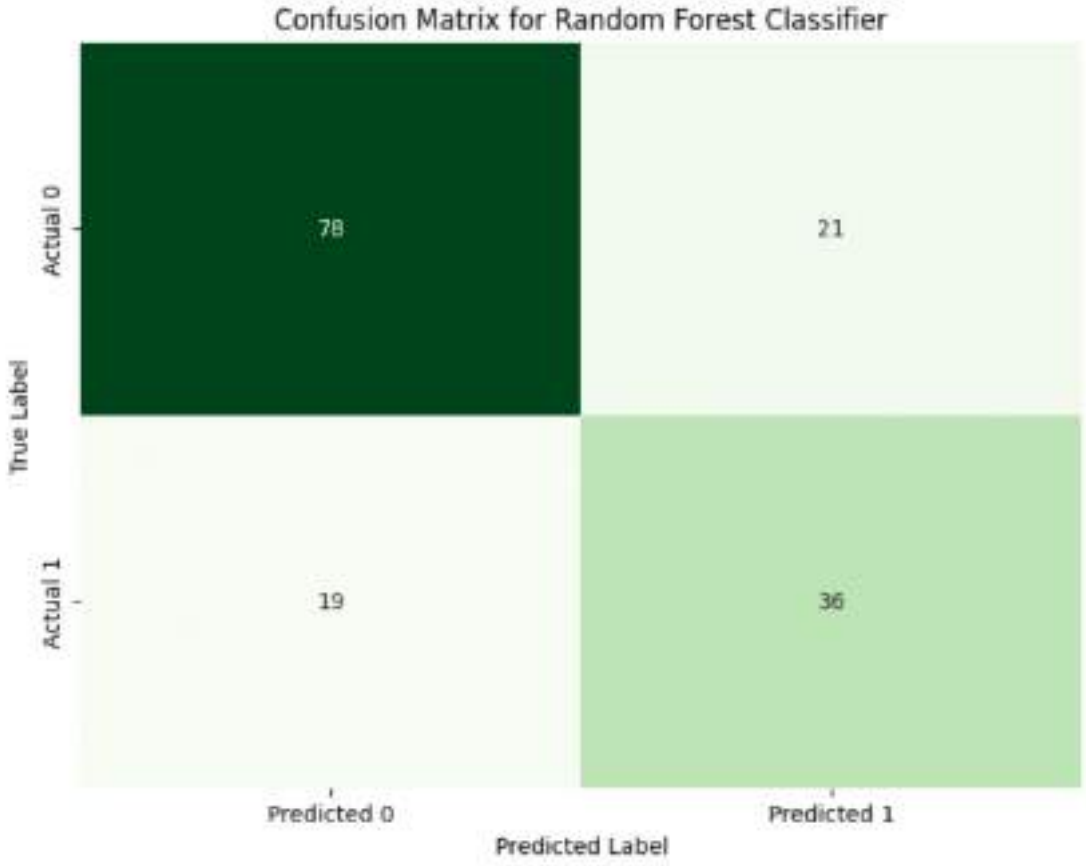
```
f1_rf = f1_score(y_test, y_pred_rf)

print("Random Forest Classifier Metrics:")
print(f"Accuracy: {accuracy_rf:.4f}")
print(f"Precision: {precision_rf:.4f}")
print(f"Recall: {recall_rf:.4f}")
print(f"F1 Score: {f1_rf:.4f}")
```

```
Random Forest Classifier Metrics:
Accuracy: 0.7403
Precision: 0.6316
Recall: 0.6545
F1 Score: 0.6429
```

In [22]:
```
print("\nClassification Report for Random Forest Classifier:")
print(classification_report(y_test, y_pred_rf))
```

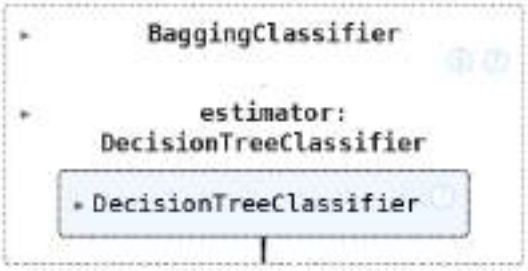
```
Classification Report for Random Forest Classifier:
              precision    recall  f1-score   support

           0       0.80      0.79      0.80        99
           1       0.63      0.65      0.64        55

    accuracy                           0.74       154
   macro avg       0.72      0.72      0.72       154
weighted avg       0.74      0.74      0.74       154
```

In [23]:
```
plt.figure(figsize=(8, 6))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Greens', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for Random Forest Classifier')
plt.show()
```



In [24]:
```
bagging_classifier = BaggingClassifier(estimator=DecisionTreeClassifier(random
bagging_classifier.fit(X_train, y_train)
```

Out[24]:



In [25]:
```
y_pred_bagging = bagging_classifier.predict(X_test)
```

In [26]:
```
cm_bagging = confusion_matrix(y_test, y_pred_bagging)
```

In [27]:
```
accuracy_bagging = accuracy_score(y_test, y_pred_bagging)
precision_bagging = precision_score(y_test, y_pred_bagging)
recall_bagging = recall_score(y_test, y_pred_bagging)
f1_bagging = f1_score(y_test, y_pred_bagging)
```

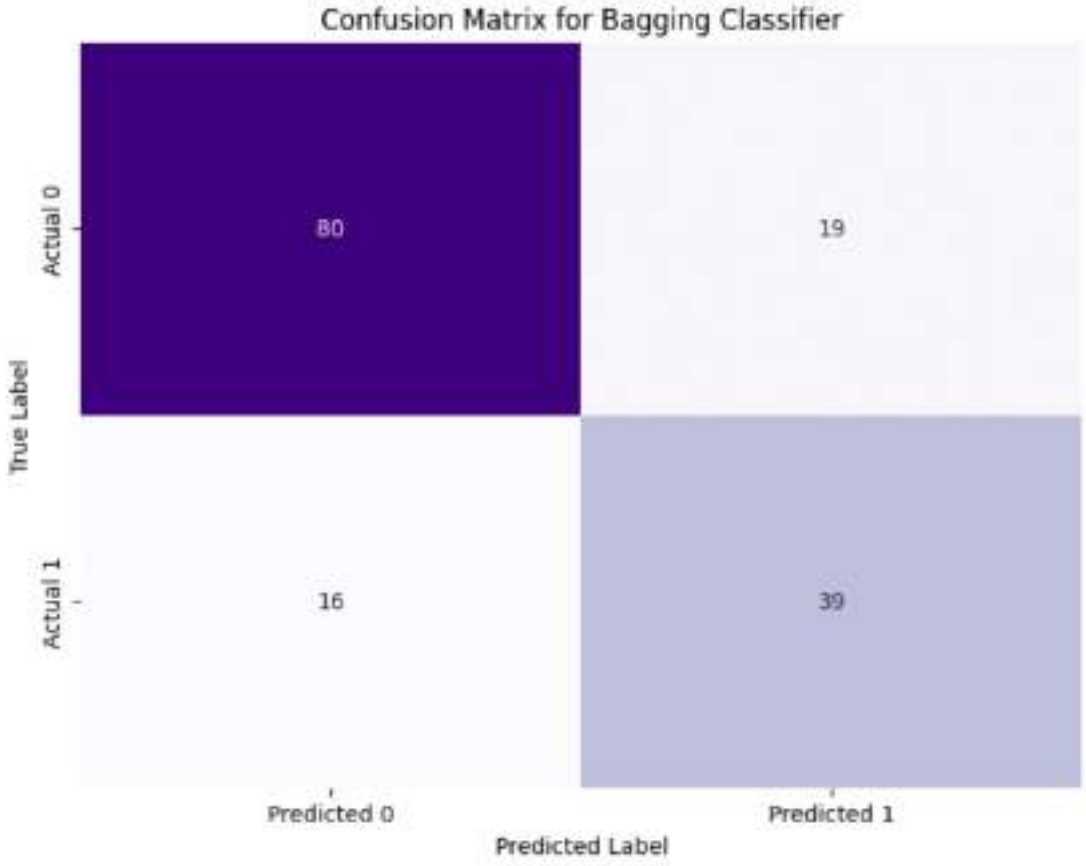
```
print("Bagging Classifier Metrics (50 Decision Trees):")
print(f"Accuracy: {accuracy_bagging:.4f}")
print(f"Precision: {precision_bagging:.4f}")
print(f"Recall: {recall_bagging:.4f}")
print(f"F1 Score: {f1_bagging:.4f}")
```

```
Bagging Classifier Metrics (50 Decision Trees):
Accuracy: 0.7727
Precision: 0.6724
Recall: 0.7091
F1 Score: 0.6903
```

In [28]:
```
print("\nClassification Report for Bagging Classifier:")
print(classification_report(y_test, y_pred_bagging))
```

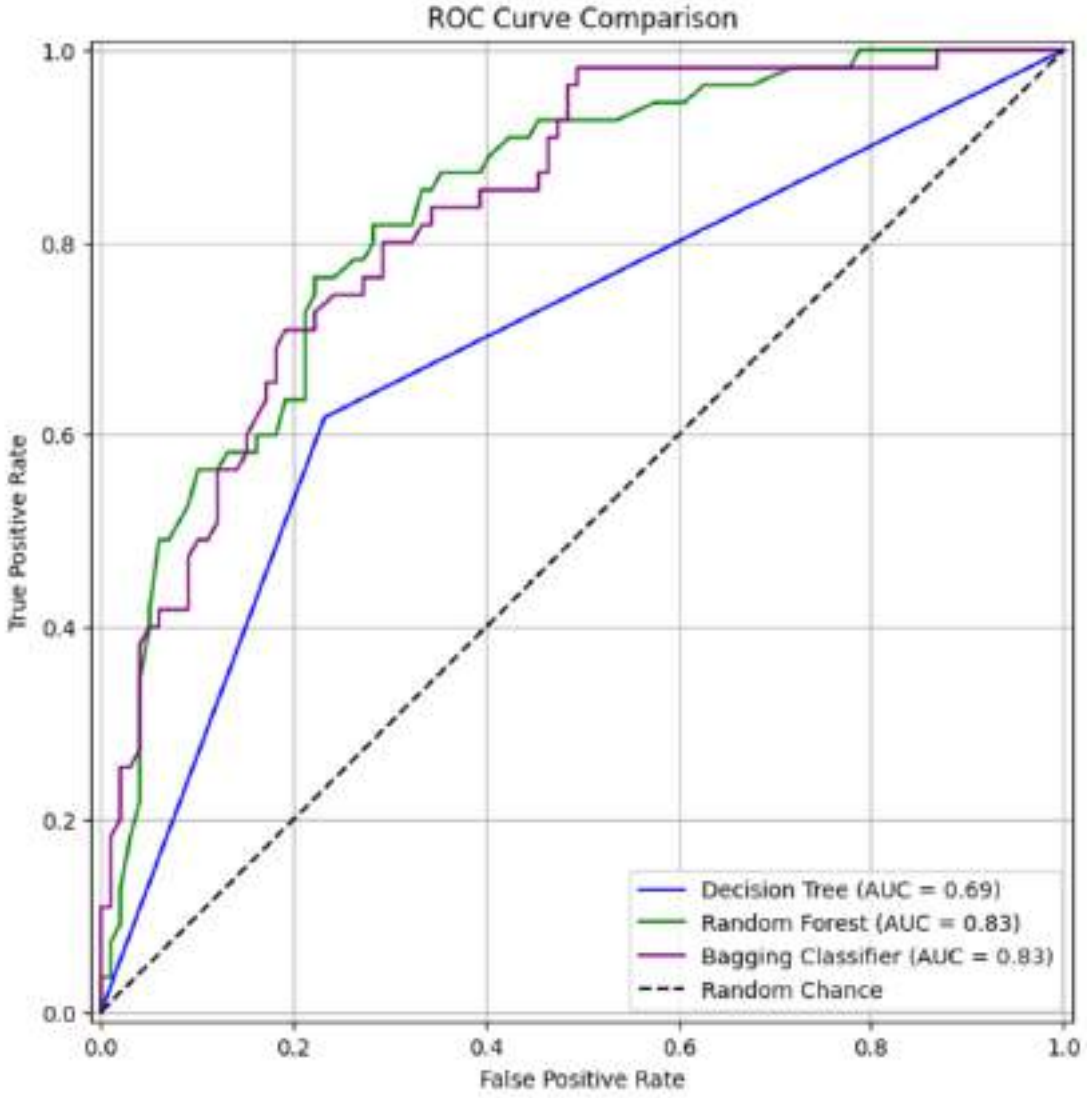
```
Classification Report for Bagging Classifier:
              precision    recall  f1-score   support

           0       0.83      0.81      0.82        99
           1       0.67      0.71      0.69        55

    accuracy                           0.77       154
   macro avg       0.75      0.76      0.76       154
weighted avg       0.78      0.77      0.77       154
```

In [29]:
```
plt.figure(figsize=(8, 6))
sns.heatmap(cm_bagging, annot=True, fmt='d', cmap='Purples', cbar=False,
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for Bagging Classifier')
plt.show()
```



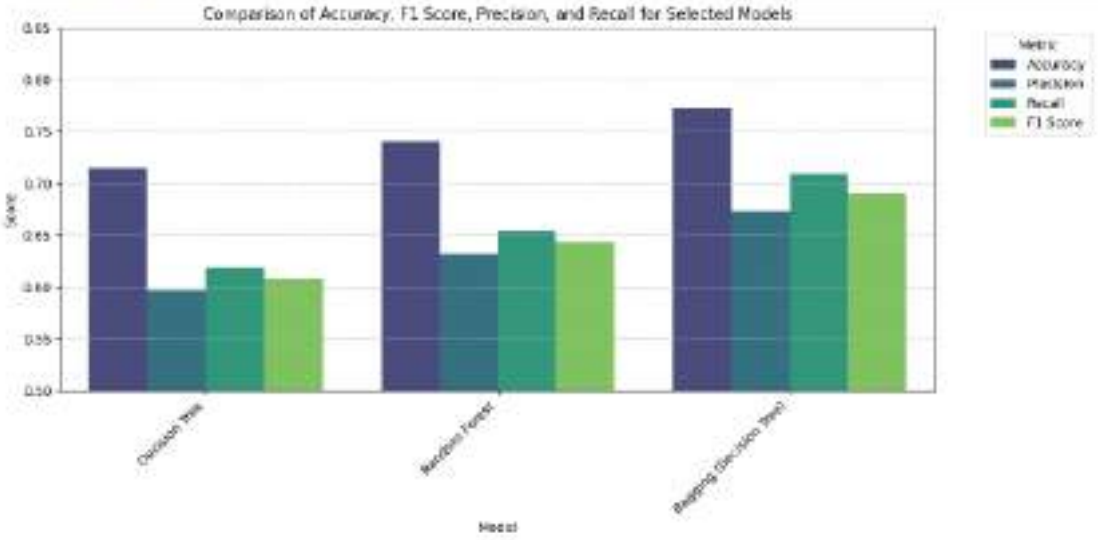
In [30]:
```
from sklearn.metrics import roc_curve, auc, RocCurveDisplay

plt.figure(figsize=(10, 8))

probs_dt = dt.predict_proba(X_test)[:, 1]
fpr_dt, tpr_dt, _ = roc_curve(y_test, probs_dt)
roc_auc_dt = auc(fpr_dt, tpr_dt)
RocCurveDisplay(fpr=fpr_dt, tpr=tpr_dt, roc_auc=roc_auc_dt, estimator_name='De

probs_rf = rf.predict_proba(X_test)[:, 1]
fpr_rf, tpr_rf, _ = roc_curve(y_test, probs_rf)
roc_auc_rf = auc(fpr_rf, tpr_rf)
RocCurveDisplay(fpr=fpr_rf, tpr=tpr_rf, roc_auc=roc_auc_rf, estimator_name='Ra

probs_bagging = bagging_classifier.predict_proba(X_test)[:, 1]
fpr_bagging, tpr_bagging, _ = roc_curve(y_test, probs_bagging)
roc_auc_bagging = auc(fpr_bagging, tpr_bagging)
RocCurveDisplay(fpr=fpr_bagging, tpr=tpr_bagging, roc_auc=roc_auc_bagging, est

plt.plot([0, 1], [0, 1], 'k--', label='Random Chance')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison')
```

```
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



In [34]:
```
selected_models = ['Decision Tree', 'Random Forest', 'Bagging (Decision Tree)'
filtered_metrics_df = metrics_df[metrics_df['Model'].isin(selected_models)]

plt.figure(figsize=(12, 6))
sns.barplot(x='Model', y='Score', hue='Metric', data=filtered_metrics_df, pale
plt.title('Comparison of Accuracy, F1 Score, Precision, and Recall for Selecte
plt.ylabel('Score')
plt.ylim(0.5, 0.85)
plt.legend(title='Metric', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xticks(rotation=45, ha='right')
```

```
plt.tight_layout()
plt.show()
```



In [ ]:
```
import pickle
with open('/content/drive/MyDrive/AIML_lab_sem4/exp 11,12/Exp11_models/decisic
    pickle.dump(dt, file)
```

In [ ]:
```
with open('/content/drive/MyDrive/AIML_lab_sem4/exp 11,12/Exp11_models/random_
    pickle.dump(rf, file)
```

In [ ]:
```
with open('/content/drive/MyDrive/AIML_lab_sem4/exp 11,12/Exp11_models/bagging
    pickle.dump(bagging_classifier, file)
```

Experiment 12
Regularization

Aim : Implement L1 (Lasso) and L2 (Ridge) regularization on the given Melbourne House Price Dataset.

Pipeline

1. Problem Definition

The primary goal is to develop a predictive model for real estate valuation using advanced regression techniques:

- Regression Task: We aim to predict the continuous numerical value of house prices based on various structural and locational features.
- Objective: We perform a comparative analysis between Standard Linear Regression, Lasso (L1), and Ridge (L2) regularization to determine which approach best handles multicollinearity and prevents model over-complexity.

2. Data Understanding

- Dataset Content: We utilize the Melbourne House Price dataset, which contains records of real estate transactions.
- Input Features (X):
    - Numerical: Rooms, Distance, Bedroom2, Bathroom, Car, Landsize, BuildingArea, YearBuilt, Propertycount.
    - Categorical: Suburb, Address, Type, Method, SellerG, Date, CouncilArea, Regionname.
- Target (y): Price (The market value of the property in AUD)

3. Data Preprocessing

To ensure the dataset is optimized for regularization, a rigorous transformation process is applied:

- **Handling Missing Values:** We address null entries in features like 'Car', 'BuildingArea', and 'YearBuilt' using zero-filling or mean/median imputation.
- **Categorical Encoding:** We apply **One-Hot Encoding** (via get_dummies) to transform non-numeric features into binary columns compatible with regression algorithms.
- **Feature-Target Separation:** The dataset is divided into the feature matrix (X) and the target variable (y).

4. Train-Test Split

- **Data Allocation:** The dataset is partitioned into a Training set (80%) to fit the model and a Test set (20%) to evaluate its performance on unseen data.
- **Random State:** A fixed random_state is used to ensure the results and data splits are reproducible across different runs.

5. Model Selection and Training

We initialize and train three different regression architectures to observe the impact of penalties:

- **Linear Regression:** A baseline model that calculates coefficients without any regularization constraints.
- **Lasso Regression (L1):** A model that adds an absolute value penalty to the loss function, which can shrink some coefficients to exactly zero, performing automatic feature selection.
- **Ridge Regression (L2):** A model that adds a squared magnitude penalty to the loss function, which helps manage multicollinearity by shrinking coefficients but not setting them to zero.

6. Predictions

- **Execution:** The trained models (Linear, Lasso, and Ridge) are used to predict house prices for the properties in the test set.
- **Storage:** Predicted values are stored to calculate error metrics and compare against the actual house price

7. Model Evaluation and Comparison

The models are evaluated using the R2 score (Coefficient of Determination) to measure how well the models generalize

Linear Regression	Ridge	Lasso
0.6426	0.6529	0.6530

8. Model Saving

- **Implementation:** The optimized regularized model and any preprocessing objects are serialized into .pkl files using the **Pickle** library.
- **Final Output:** These files allow the prediction system to be reloaded for real-time house price estimations without needing to retrain the model

In [17]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncode
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import r2_score


df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/exp 11,12/dataset_Melbc
```

In [18]:
```
df.head()
```

Out[18]:

	Suburb	Address	Rooms	Type	Price	Method	SellerG	Date	Distar
0	Abbotsford	68 Studley St	2	h	NaN	SS	Jellis	3/09/2016	
1	Abbotsford	85 Turner St	2	h	1480000.0	S	Biggin	3/12/2016	
2	Abbotsford	25 Bloomburg St	2	h	1035000.0	S	Biggin	4/02/2016	
3	Abbotsford	18/659 Victoria St	3	u	NaN	VB	Rounds	4/02/2016	
4	Abbotsford	5 Charles St	3	h	1465000.0	SP	Biggin	4/03/2017	

5 rows × 21 columns

In [19]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34857 entries, 0 to 34856
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   Suburb         34857 non-null  object 
 1   Address        34857 non-null  object 
 2   Rooms          34857 non-null  int64  
 3   Type           34857 non-null  object 
 4   Price          27247 non-null  float64
 5   Method         34857 non-null  object 
 6   SellerG        34857 non-null  object 
 7   Date           34857 non-null  object 
 8   Distance       34856 non-null  float64
 9   Postcode       34856 non-null  float64
 10  Bedroom2       26640 non-null  float64
 11  Bathroom       26631 non-null  float64
 12  Car            26129 non-null  float64
 13  Landsize       23047 non-null  float64
 14  BuildingArea   13742 non-null  float64
 15  YearBuilt      15551 non-null  float64
 16  CouncilArea    34854 non-null  object 
 17  Lattitude      26881 non-null  float64
 18  Longtitude     26881 non-null  float64
 19  Regionname     34854 non-null  object 
 20  Propertycount  34854 non-null  float64
dtypes: float64(12), int64(1), object(8)
memory usage: 5.6+ MB
```

In [20]:
```
df.isnull().sum()
```

Out[20]:

	0
Suburb	0
Address	0
Rooms	0
Type	0
Price	7610
Method	0
SellerG	0
Date	0
Distance	1
Postcode	1
Bedroom2	8217
Bathroom	8226
Car	8728
Landsize	11810
BuildingArea	21115
YearBuilt	19306
CouncilArea	3
Lattitude	7976
Longtitude	7976
Regionname	3
Propertycount	3

dtype: int64

In [21]:
```
df = df.dropna(subset=['Price'])
```

In [22]:
```
num_cols = ['Rooms', 'Distance', 'Postcode', 'Bedroom2', 'Bathroom', 'Car', 'L
cat_cols = ['Type', 'Method', 'CouncilArea', 'Regionname']
```

In [23]:
```
X = df[num_cols + cat_cols]
y = df['Price']
```

In [24]:
```
df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

In [25]:
```
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

In [26]:
```
df = df.drop(['Address', 'Date'], axis=1)

df = pd.get_dummies(df, drop_first=True)
```

In [27]:
```
X = df.drop('Price', axis=1)
y = df['Price']
```

In [28]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [29]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [29]:
```

```

In [33]:
```
from sklearn.metrics import mean_squared_error

# Apply Ridge (L2 Regularization)
ridge = Ridge(alpha=100)
ridge.fit(X_train_scaled, y_train)

y_pred_ridge = ridge.predict(X_test_scaled)

print("\nRidge Results:")
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
rmse_ridge = np.sqrt(mse_ridge)
print("MSE:", mse_ridge)
print("RMSE:", rmse_ridge)
print("R2 Score:", r2_score(y_test, y_pred_ridge))
```

```
Ridge Results:
MSE: 148308986100.42667
RMSE: 385109.05741156836
R2 Score: 0.6529238980216601
```

In [34]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Apply Linear Regression
linear_reg = LinearRegression()
linear_reg.fit(X_train_scaled, y_train)

y_pred_linear = linear_reg.predict(X_test_scaled)

print("\nLinear Regression Results:")
mse_linear = mean_squared_error(y_test, y_pred_linear)
rmse_linear = np.sqrt(mse_linear)
```

```
print("MSE:", mse_linear)
print("RMSE:", rmse_linear)
print("R2 Score:", r2_score(y_test, y_pred_linear))
```

```
Linear Regression Results:
MSE: 152722264764.61996
RMSE: 390796.9610483428
R2 Score: 0.6425958417387105
```

In [37]:
```
from sklearn.linear_model import Lasso

# Apply Lasso (L1 Regularization)
lasso = Lasso(alpha=5.0) # You can tune alpha as needed
lasso.fit(X_train_scaled, y_train)

y_pred_lasso = lasso.predict(X_test_scaled)

print("\nLasso Results:")
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
rmse_lasso = np.sqrt(mse_lasso)
print("MSE:", mse_lasso)
print("RMSE:", rmse_lasso)
print("R2 Score:", r2_score(y_test, y_pred_lasso))
```

```
Lasso Results:
MSE: 148275760119.93
RMSE: 385065.91659082216
R2 Score: 0.6530016542257742
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning: Objective did not converge. You might want to increase the number of iterations, check the scale of the features or consider increasing regularisation. Duality gap: 5.717e+14, tolerance: 8.882e+11
  model = cd_fast.enet_coordinate_descent(
```

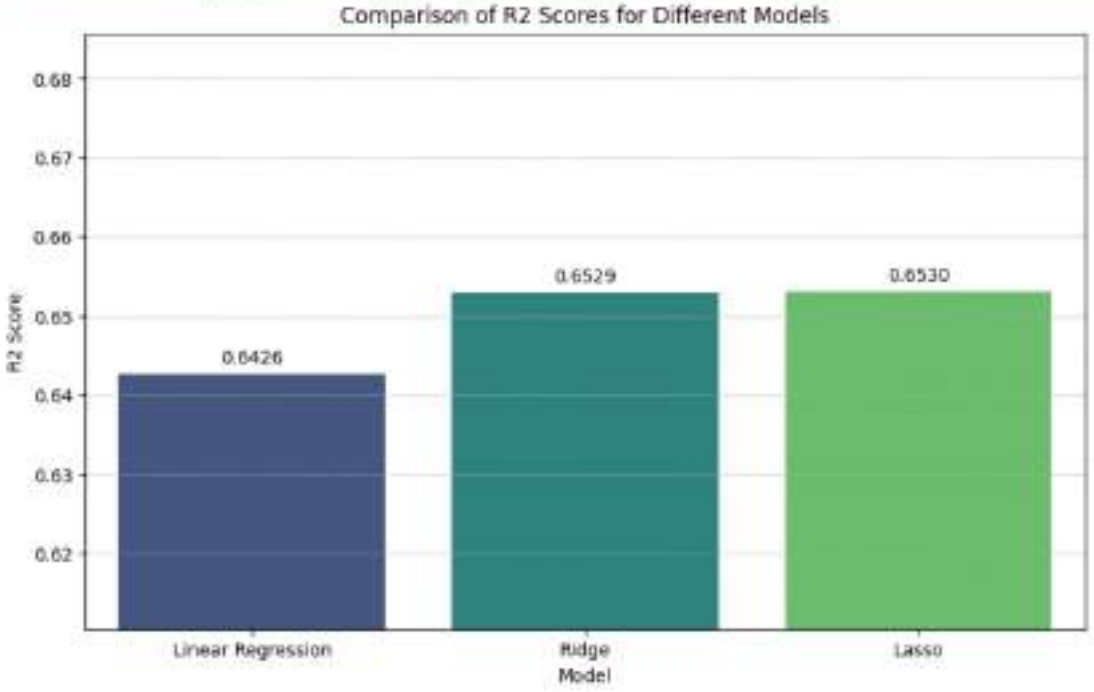
In [36]:
```
lasso_coeff = pd.Series(lasso.coef_, index=X.columns)
ridge_coeff = pd.Series(ridge.coef_, index=X.columns)

print("\nNumber of features used by Lasso:", sum(lasso_coeff != 0))
print("Number of features used by Ridge:", len(ridge_coeff))
```

```
Number of features used by Lasso: 721
Number of features used by Ridge: 749
```

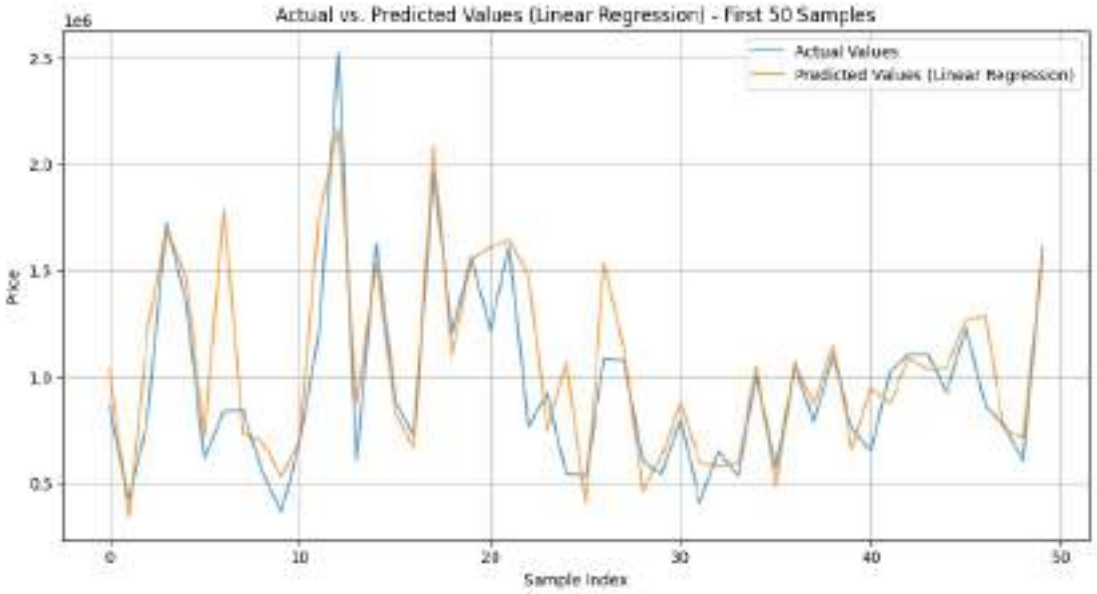
In [41]:
```
r2_linear = r2_score(y_test, y_pred_linear)
r2_ridge = r2_score(y_test, y_pred_ridge)
r2_lasso = r2_score(y_test, y_pred_lasso)

models = ['Linear Regression', 'Ridge', 'Lasso']
r2_scores = [r2_linear, r2_ridge, r2_lasso]

plt.figure(figsize=(10, 6))
ax = sns.barplot(x=models, y=r2_scores, palette='viridis')
plt.title('Comparison of R2 Scores for Different Models')
plt.xlabel('Model')
```

```
plt.ylabel('R2 Score')
plt.ylim(min(r2_scores) * 0.95, max(r2_scores) * 1.05)
plt.grid(axis='y', linestyle='--', alpha=0.7)
for p in ax.patches:
    ax.annotate(f'{p.get_height():.4f}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 9),
                textcoords='offset points')

plt.show()
```

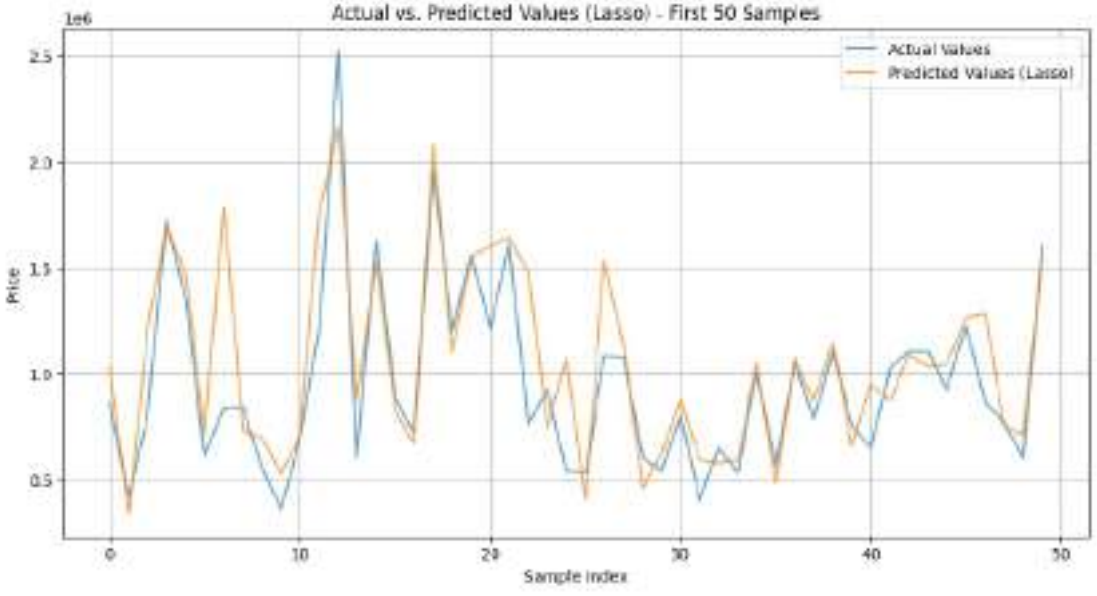
```
/tmp/ipykernel_17265/2865464898.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same e
ffect.

  ax = sns.barplot(x=models, y=r2_scores, palette='viridis')
```



In [43]:
```
plt.figure(figsize=(12, 6))
plt.plot(y_test.values[:50], label='Actual Values', alpha=0.7)
plt.plot(y_pred_linear[:50], label='Predicted Values (Linear Regression)', alp
plt.title('Actual vs. Predicted Values (Linear Regression) - First 50 Samples'
plt.xlabel('Sample Index')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()
```

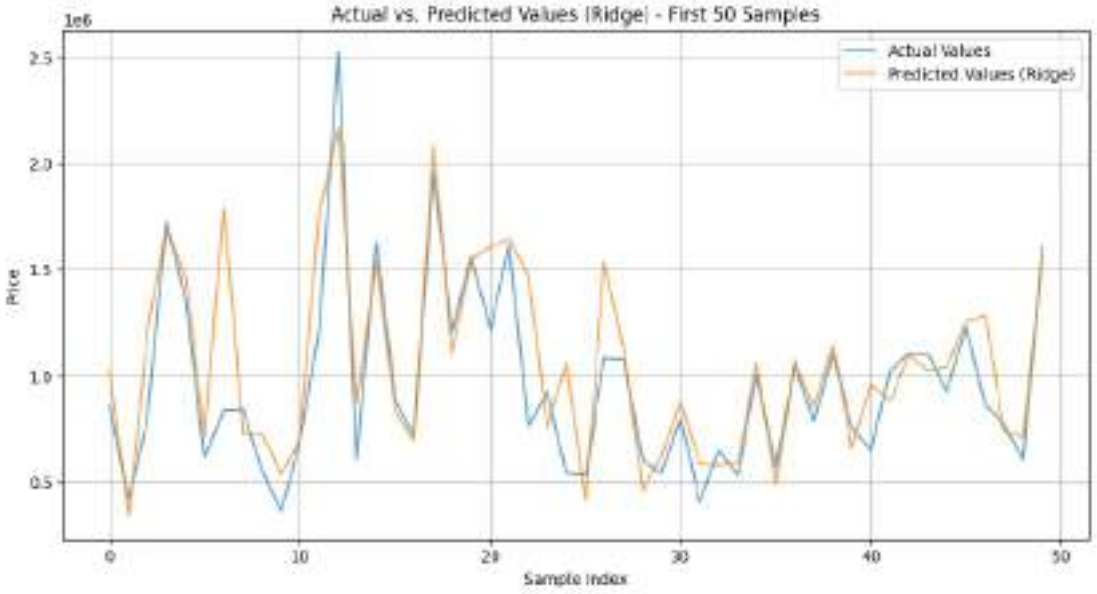


In [44]:
```
plt.figure(figsize=(12, 6))
plt.plot(y_test.values[:50], label='Actual Values', alpha=0.7)
plt.plot(y_pred_lasso[:50], label='Predicted Values (Lasso)', alpha=0.7)
plt.title('Actual vs. Predicted Values (Lasso) - First 50 Samples')
plt.xlabel('Sample Index')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()
```



In [45]:
```
plt.figure(figsize=(12, 6))
```

```
plt.plot(y_test.values[:50], label='Actual Values', alpha=0.7)
plt.plot(y_pred_ridge[:50], label='Predicted Values (Ridge)', alpha=0.7)
plt.title('Actual vs. Predicted Values (Ridge) - First 50 Samples')
plt.xlabel('Sample Index')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()
```



In [46]:
```
print("\n--- Model Performance Summary ---")
print("\nLinear Regression:")
print(f"  R2 Score: {r2_linear:.4f}")
print(f"  MSE: {mse_linear:.2f}")
print(f"  RMSE: {rmse_linear:.2f}")

print("\nRidge Regression:")
print(f"  R2 Score: {r2_ridge:.4f}")
print(f"  MSE: {mse_ridge:.2f}")
print(f"  RMSE: {rmse_ridge:.2f}")

print("\nLasso Regression:")
print(f"  R2 Score: {r2_lasso:.4f}")
print(f"  MSE: {mse_lasso:.2f}")
print(f"  RMSE: {rmse_lasso:.2f}")
```

```
--- Model Performance Summary ---

Linear Regression:
  R2 Score: 0.6426
  MSE: 152722264764.62
  RMSE: 390796.96

Ridge Regression:
  R2 Score: 0.6529
  MSE: 148308986100.43
  RMSE: 385109.06

Lasso Regression:
  R2 Score: 0.6530
  MSE: 148275760119.93
  RMSE: 385065.92
```

In [47]:
```python
import pickle


lasso_path = "/content/drive/MyDrive/AIML_lab_sem4/exp 11,12/Exp12_models/lass
with open(lasso_path, 'wb') as f:
    pickle.dump(lasso, f)

ridge_path = "/content/drive/MyDrive/AIML_lab_sem4/exp 11,12/Exp12_models/ridg
with open(ridge_path, 'wb') as f:
    pickle.dump(ridge, f)

scaler_path = "/content/drive/MyDrive/AIML_lab_sem4/exp 11,12/Exp12_models/sca
with open(scaler_path, 'wb') as f:
    pickle.dump(scaler, f)

print("Models and scaler saved successfully!")
```

```
Models and scaler saved successfully!
```

Applied Machine Learning

School of Computer Science, UPES, Dehradun.

MINI PROJECT

B.TECH. - IV Semester

Jan - May. (2026)

SUBMITTED TO :-

Dr. Sahinur Rahman Laskar ,

Assistant Professor,

SoCS, UPES

SUBMITTED BY :-

Name : Pranav Joshi

Sap ID : 590012855

Batch : 19

Automobile Price Prediction

Dataset	automobiles.csv (UCI Automobile Dataset) Data_link
Target Variable	Price (continuous numeric — USD)

1. Problem Definition

The goal of this project is to build a machine learning regression model that can accurately predict the price of an automobile given its technical and physical specifications. Unlike classification tasks, here the output is a continuous numeric value (price in USD), making it a pure regression problem.

Parameter	Detail
Target Variable (y)	price the listed selling price of the car in USD
Nature of Target	Continuous numeric; ranges from ~$5,000 to ~$45,000 in this dataset

2. Data Understanding

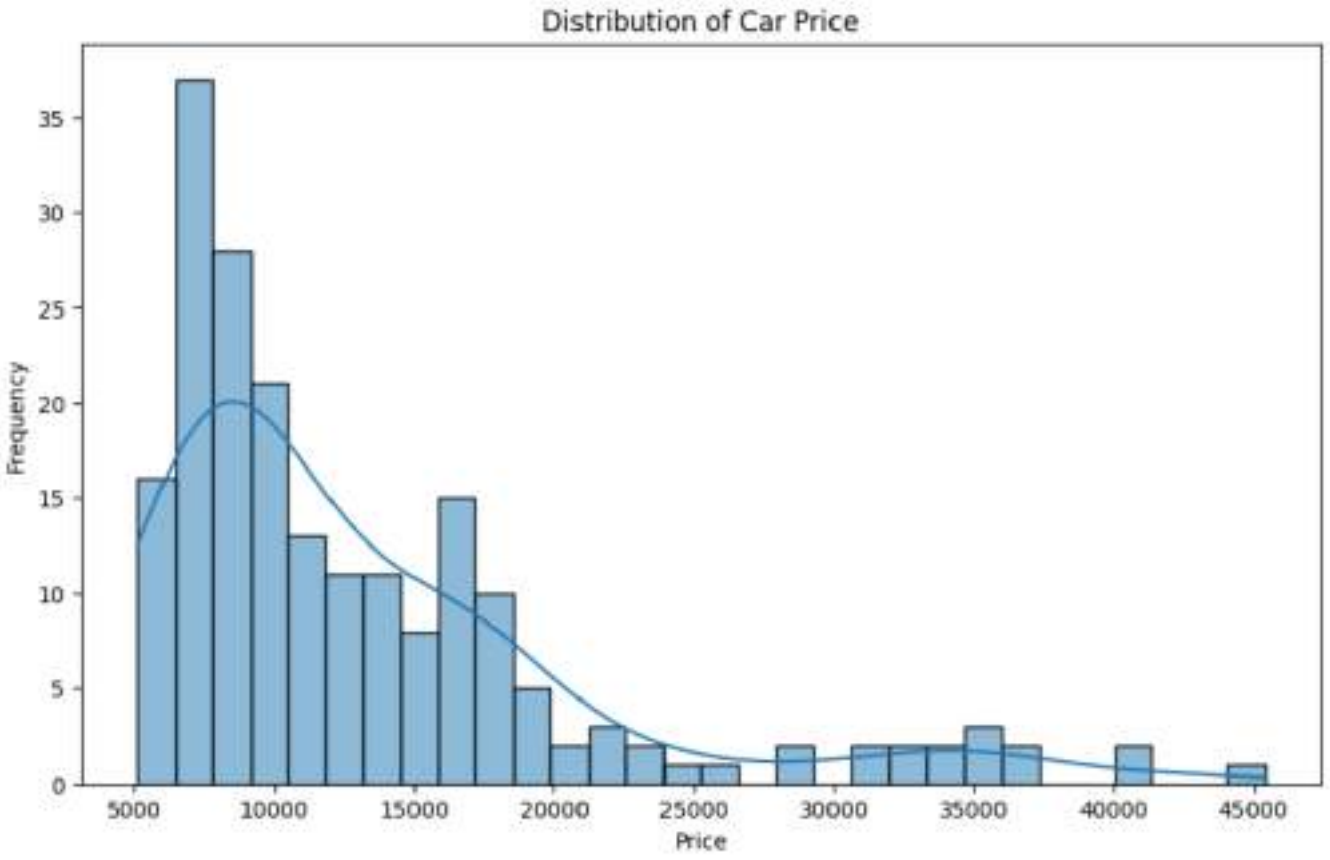
The dataset is the UCI Automobile Dataset loaded from automobiles.csv. Each row represents one car model. The dataset contains 26 columns in total, including categorical specs (make, body-style, drive-wheels) and numerical measurements (engine-size, horsepower, mpg). The target column price appears in the last column.

Feature Inventory (Input Features used in X):

Numerical Features (X): normalized-losses, wheel-base, length, width, height, curb-weight, engine-size, bore, stroke, compression-ratio, horsepower, peak-rpm, city-mpg, highway-mpg.

Categorical Features (X): make, fuel-type, aspiration, num-of-doors, body-style, drive-wheels, engine-location, engine-type, num-of-cylinders, fuel-system.

Target Variable (y): price



3. Data Pre-Processing

Raw data from automobiles.csv contains missing values encoded as '?' strings, mixed numeric/text columns, and categorical features that must be converted before any ML algorithm can process them. The preprocessing pipeline performs the following steps in strict order:

Step 3a: Replace '?' with NaN

The raw CSV uses '?' as a placeholder for missing values. Replacing with NaN allows pandas/sklearn to detect and handle them properly.

df.replace('?', np.nan)

Step 3b: Drop rows with missing Price

Price is the target variable a row without a known price cannot be used for supervised learning and is removed entirely.

df.dropna(subset=['price'])df['price'] = pd.to_numeric(df['price'])

Step 3c: Cast numerical columns to float

14 columns (engine-size, horsepower, bore, etc.) are stored as object dtype due to the '?' contamination. Explicit casting enables median imputation in the next step.

for col in real_num_cols: df[col] = pd.to_numeric(df[col])

Step 3d: Median Imputation for Numerical Missing Values

Remaining NaN values in numerical columns are filled using the median (not mean) because median is robust to outliers automobile specs like horsepower can have extreme values that would distort mean imputation.

imputer = SimpleImputer(strategy='median')df[real_num_cols] = imputer.fit_transform(df[real_num_cols])

Step 3e: One-Hot Encoding of Categorical Features

Categorical columns (make, body-style, fuel-type, drive-wheels, engine-type, etc.) are converted into binary (0/1) indicator columns using One-Hot Encoding. drop_first=True avoids the dummy-variable trap (multicollinearity).

df_encoded = pd.get_dummies(df, drop_first=True)

Step 3f: Feature Scaling StandardScaler

All numerical features are standardised to zero-mean and unit-variance. This is critical for regularised models (Ridge, Lasso) because they penalise coefficients — without scaling, features with large ranges dominate the penalty term.

scaler = StandardScaler()X_scaled = scaler.fit_transform(X)

4. Splitting

After preprocessing, the full feature matrix X_scaled and target vector y are split into training and testing subsets using sklearn's train_test_split.

Split Parameter	Value	Reasoning
test_size	0.2 (20%)	Standard hold-out ratio. Leaves enough data for reliable evaluation without shrinking the training set too much.
Training samples	~160 rows	80% of the cleaned dataset used to fit all regression models.

Testing samples	~40 rows	20% completely unseen during training used only for final evaluation.

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

5. Model Training

Five regression algorithms are trained and compared. Each model was chosen for a specific reason based on the nature of the automobile dataset:

1. Linear Regression

Acts as a benchmark assuming linear price-feature relationships; no hyperparameters required for simple performance validation.

model_lr = LinearRegression()model_lr.fit(X_train, y_train)

2. Ridge Regression (L2 Regularisation)

Prevents overfitting in datasets with many dummy variables by using L2 penalties to shrink coefficients.

model_ridge = Ridge(alpha=1.0)model_ridge.fit(X_train, y_train)

3. Lasso Regression (L1 Regularisation)

Uses L1 penalties for automatic feature selection, zeroing out irrelevant features to create a sparser model.

model_lasso = Lasso(alpha=1.0, max_iter=10000)model_lasso.fit(X_train, y_train)

4. Decision Tree Regressor

Handles non-linear price impacts across feature regions without requiring data scaling for original splits.

model_dt = DecisionTreeRegressor(random_state=42)model_dt.fit(X_train, y_train)

5. Random Forest Regressor

Ensemble of 100 trees that reduces variance, captures complex feature interactions, and provides robustness against price outliers.

model_rf = RandomForestRegressor(random_state=42)model_rf.fit(X_train, y_train)

6. Model testing and evaluation

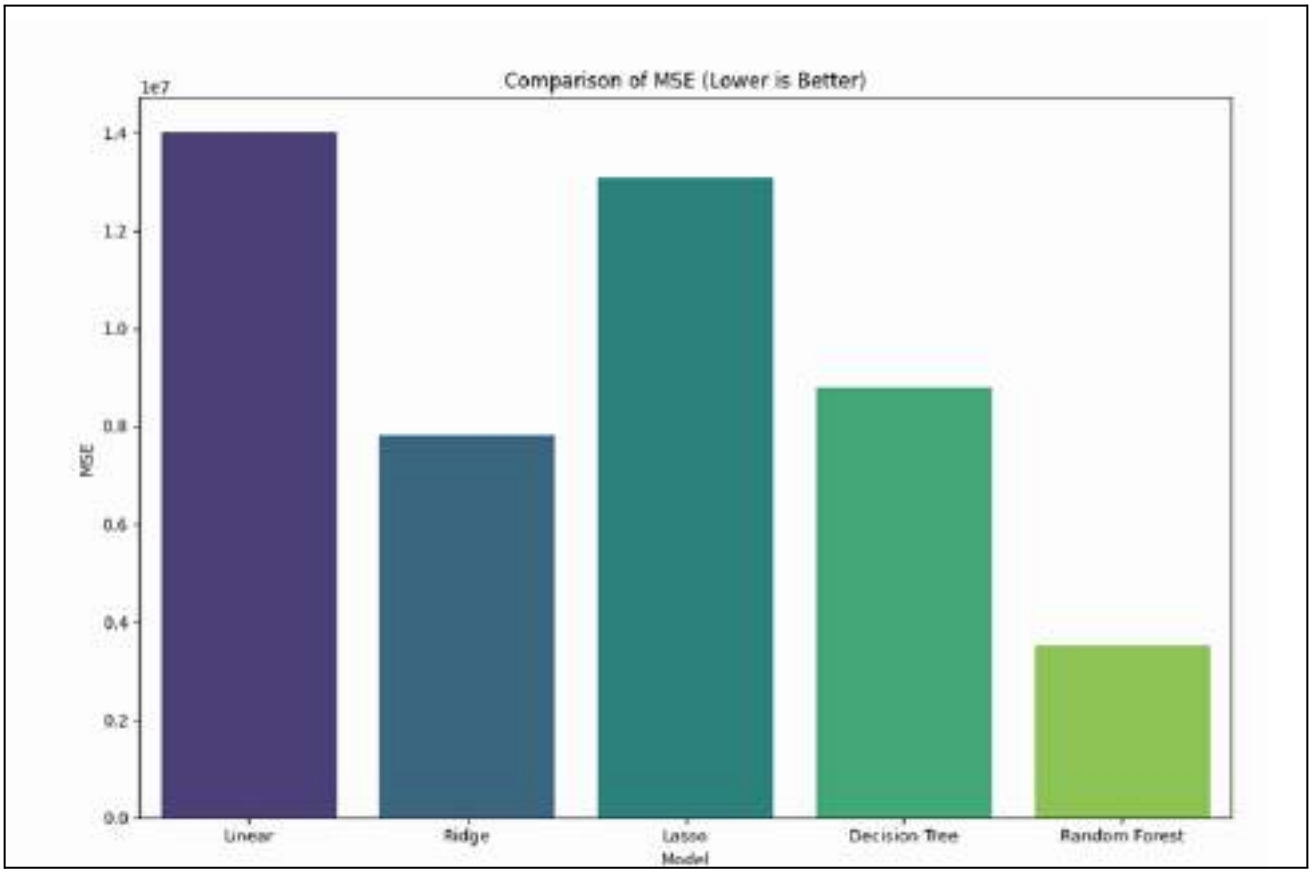
After training, every model predicts prices on the held-out test set and is evaluated using three complementary methods: numeric metrics, visual actual-vs-predicted scatter plots, and a final side-by-side comparison chart.

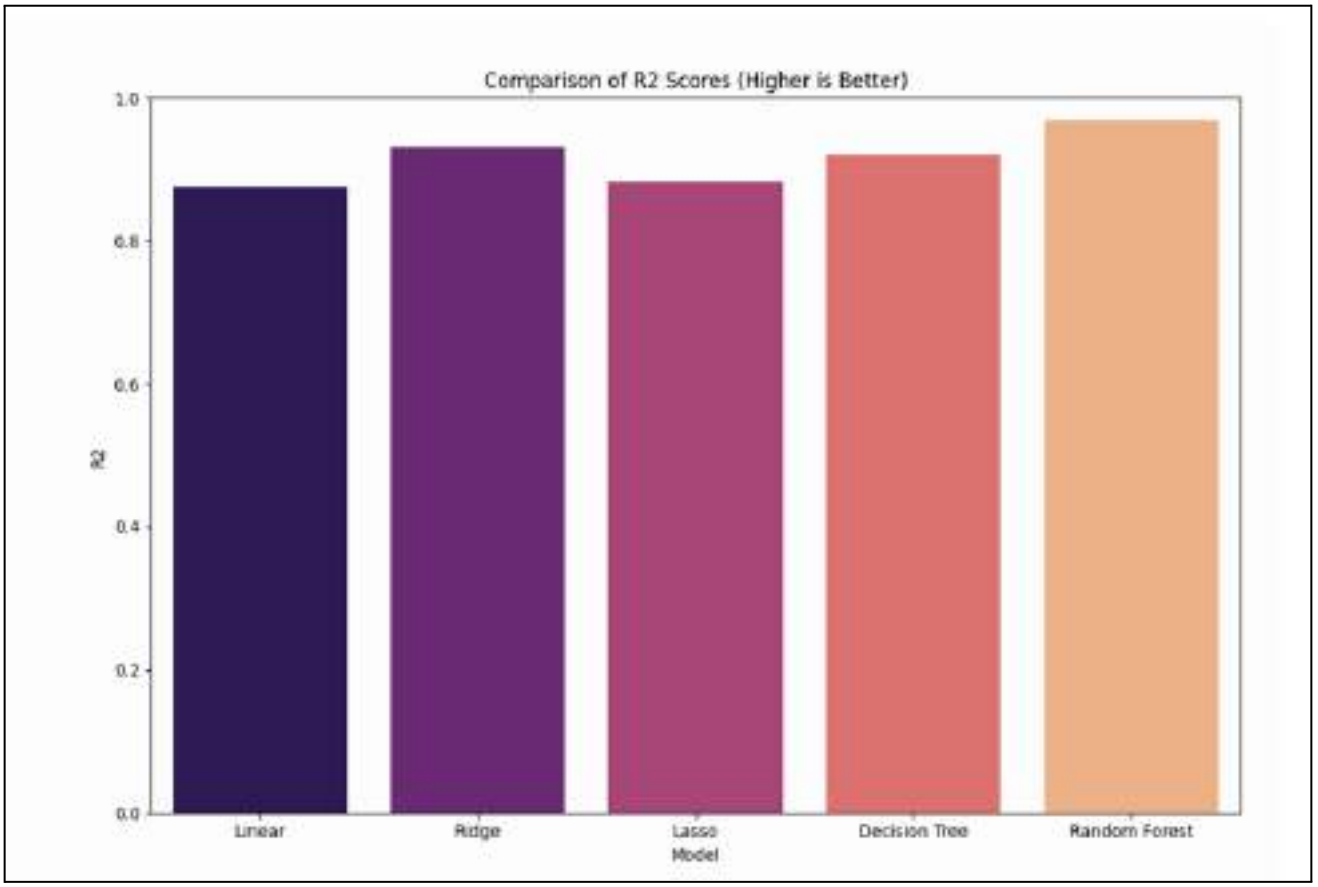
Evaluation Metrics Explained:

Metric	Explanation
R^2 Score	A statistical measure that represents the proportion of the variance for a dependent variable that's explained by the independent variables in the model.
MSE (Mean Squared Error)	The average of the squared differences between the actual values and the predicted values.

Final Comparison :

Model	R2	MSE	RMSE
Linear regression	0.8750	14007858	3742
Ridge	0.9304	7794482	2791
Lasso	0.8831	13094871	3618
Decision tree	0.9217	8770303	2961
Random forest	0.9685	3533105	1879


Comparison of MSE (Lower is Better)
1e7
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
MSE
Linear
Ridge
Lasso
Decision Tree
Random Forest
Model


Comparison of R2 Scores (Higher is Better)
1.0
0.8
0.6
0.4
0.2
0.0
R2
Linear
Ridge
Lasso
Decision Tree
Random Forest
Model

7. Conclusion

- Data preprocessing was the most critical step , The dataset contained '?' as missing values and mixed data types across 26 columns, requiring type conversion, imputation, and label encoding before any model could be trained.

- Random Forest significantly outperformed all other models, achieving an R^2 of 0.9685 and an RMSE of ~1880 — far better than plain Linear Regression (R^2 = 0.875, RMSE ~3743), confirming that ensemble methods capture the complex, non-linear interactions in car pricing data.

- Regularization improved over basic linear regression but has limits. Ridge (R^2 = 0.930) outperformed both Linear and Lasso, showing that L2 regularization helps control overfitting on this high-dimensional data; however, neither could rival tree-based models.

- Feature scaling is essential for regularized models. Ridge and Lasso depend on uniform penalization across features, making StandardScaler a prerequisite; tree-based models are inherently scale-invariant yet still benefited from a consistent pipeline.

In [ ]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet, S
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import mean_squared_error, r2_score, classification_repor
from sklearn.datasets import load_wine
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
```

In [ ]:
```
df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/Project/automobiles.csv
```

In [ ]:
```
df.head()
```

Out[ ]:

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels
0	3	?	alfa-romero	gas	std	two	convertible	rwd
1	3	?	alfa-romero	gas	std	two	convertible	rwd
2	1	?	alfa-romero	gas	std	two	hatchback	rwd
3	2	164	audi	gas	std	four	sedan	fwd
4	2	164	audi	gas	std	four	sedan	4wd

5 rows × 26 columns

In [ ]:
```
df = df.replace('?', np.nan)
```

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 26 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   symboling          205 non-null    int64  
 1   normalized-losses  164 non-null    object 
 2   make               205 non-null    object 
 3   fuel-type          205 non-null    object 
 4   aspiration         205 non-null    object 
 5   num-of-doors       203 non-null    object 
 6   body-style         205 non-null    object 
 7   drive-wheels       205 non-null    object 
 8   engine-location    205 non-null    object 
 9   wheel-base         205 non-null    float64
 10  length             205 non-null    float64
 11  width              205 non-null    float64
 12  height             205 non-null    float64
 13  curb-weight        205 non-null    int64  
 14  engine-type        205 non-null    object 
 15  num-of-cylinders   205 non-null    object 
 16  engine-size        205 non-null    int64  
 17  fuel-system        205 non-null    object 
 18  bore               201 non-null    object 
 19  stroke             201 non-null    object 
 20  compression-ratio  205 non-null    float64
 21  horsepower         203 non-null    object 
 22  peak-rpm           203 non-null    object 
 23  city-mpg           205 non-null    int64  
 24  highway-mpg        205 non-null    int64  
 25  price              201 non-null    object 
dtypes: float64(5), int64(5), object(16)
memory usage: 41.8+ KB
```

In [ ]:
```
df.shape
```

Out[ ]:
```
(205, 26)
```

In [ ]:
```
df = df.dropna(subset=['price'])
df['price'] = pd.to_numeric(df['price'])
```

In [ ]:
```
test_case_row = df.iloc[[0]].copy()
df = df.iloc[1:].copy()
```

In [ ]:
```
real_num_cols = ['normalized-losses', 'wheel-base', 'length', 'width', 'height
                 'engine-size', 'bore', 'stroke', 'compression-ratio', 'horsep
                 'peak-rpm', 'city-mpg', 'highway-mpg']
```

In [ ]:
```
for col in real_num_cols:
    df[col] = pd.to_numeric(df[col])
```

In [ ]:
```
df.info()
```

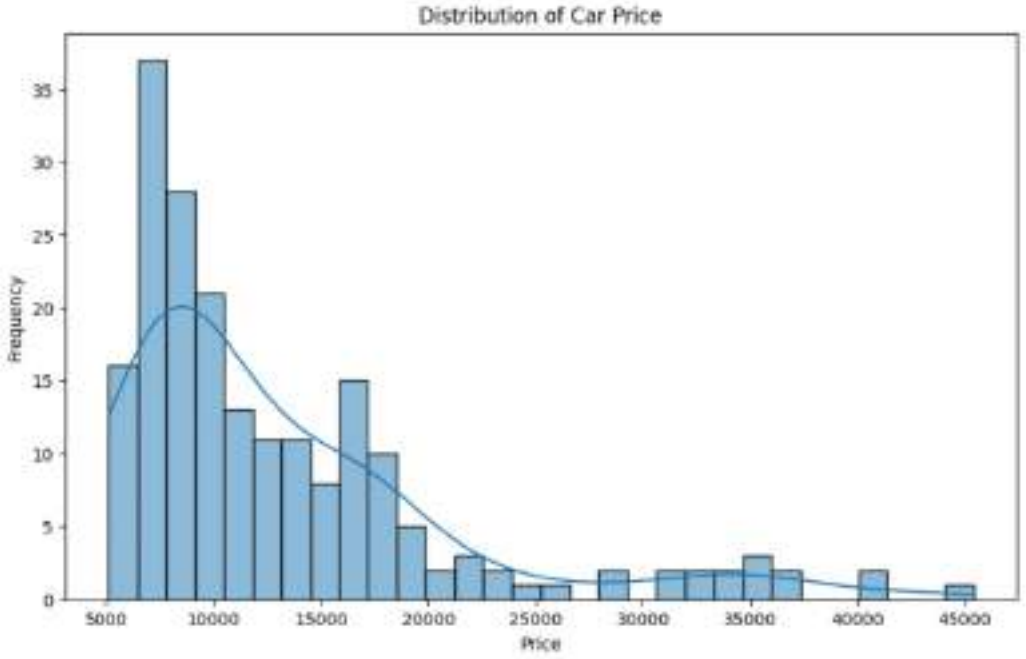
```
<class 'pandas.core.frame.DataFrame'>
Index: 200 entries, 1 to 204
Data columns (total 26 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   symboling          200 non-null    int64  
 1   normalized-losses  164 non-null    float64
 2   make               200 non-null    object 
 3   fuel-type          200 non-null    object 
 4   aspiration         200 non-null    object 
 5   num-of-doors       198 non-null    object 
 6   body-style         200 non-null    object 
 7   drive-wheels       200 non-null    object 
 8   engine-location    200 non-null    object 
 9   wheel-base         200 non-null    float64
 10  length             200 non-null    float64
 11  width              200 non-null    float64
 12  height             200 non-null    float64
 13  curb-weight        200 non-null    int64  
 14  engine-type        200 non-null    object 
 15  num-of-cylinders   200 non-null    object 
 16  engine-size        200 non-null    int64  
 17  fuel-system        200 non-null    object 
 18  bore               196 non-null    float64
 19  stroke             196 non-null    float64
 20  compression-ratio  200 non-null    float64
 21  horsepower         198 non-null    float64
 22  peak-rpm           198 non-null    float64
 23  city-mpg           200 non-null    int64  
 24  highway-mpg        200 non-null    int64  
 25  price              200 non-null    int64  
dtypes: float64(10), int64(6), object(10)
memory usage: 42.2+ KB
```

In [ ]:
```
imputer = SimpleImputer(strategy='median')
df[real_num_cols] = imputer.fit_transform(df[real_num_cols])
```

Exploratory Data Analysis (EDA)

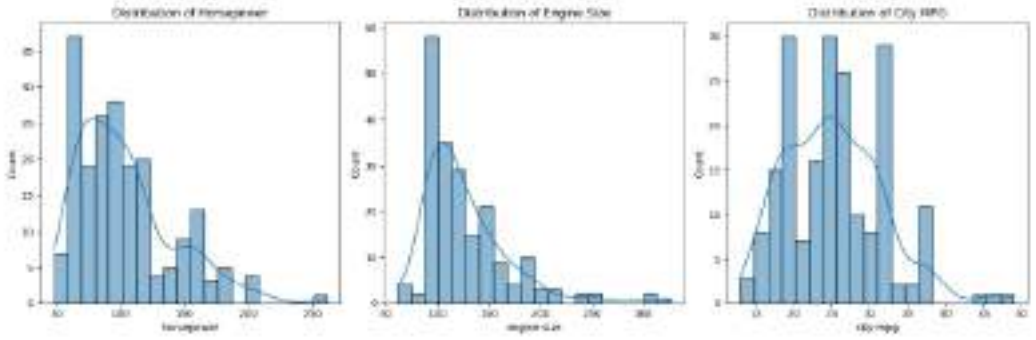
Let's start by visualizing the distribution of the target variable, `price`.

In [ ]:
```
plt.figure(figsize=(10, 6))
sns.histplot(df['price'], bins=30, kde=True)
plt.title('Distribution of Car Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```

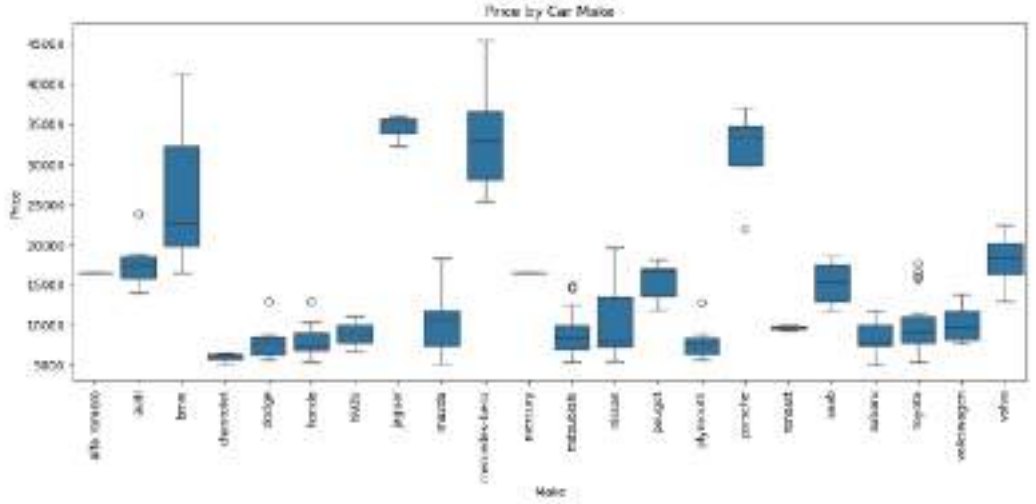


Now, let's look at the distributions of some other key numerical features like 'horsepower', 'engine-size', and 'city-mpg'.

In [ ]:
```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.histplot(df['horsepower'], bins=20, kde=True)
plt.title('Distribution of Horsepower')

plt.subplot(1, 3, 2)
sns.histplot(df['engine-size'], bins=20, kde=True)
plt.title('Distribution of Engine Size')

plt.subplot(1, 3, 3)
sns.histplot(df['city-mpg'], bins=20, kde=True)
plt.title('Distribution of City MPG')

plt.tight_layout()
plt.show()
```



In [ ]:
```
plt.figure(figsize=(12, 6))
sns.boxplot(x='make', y='price', data=df)
plt.xticks(rotation=90)
plt.title('Price by Car Make')
plt.xlabel('Make')
plt.ylabel('Price')
plt.tight_layout()
plt.show()
```



In [ ]:
```
df_encoded = pd.get_dummies(df, drop_first=True)
```

In [ ]:
```
X = df_encoded.drop('price', axis=1)
y = df_encoded['price']
```

In [ ]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2
```

In [ ]:
```
results = []
```

```
print("Preprocessing complete. Training set size:", X_train.shape[0])
```

```
Preprocessing complete. Training set size: 160
```

Linear regression

In [ ]:
```
model_lr = LinearRegression()
```

In [ ]:
```
model_lr.fit(X_train, y_train)
```

Out[ ]:
```
▾ LinearRegression
LinearRegression()
```

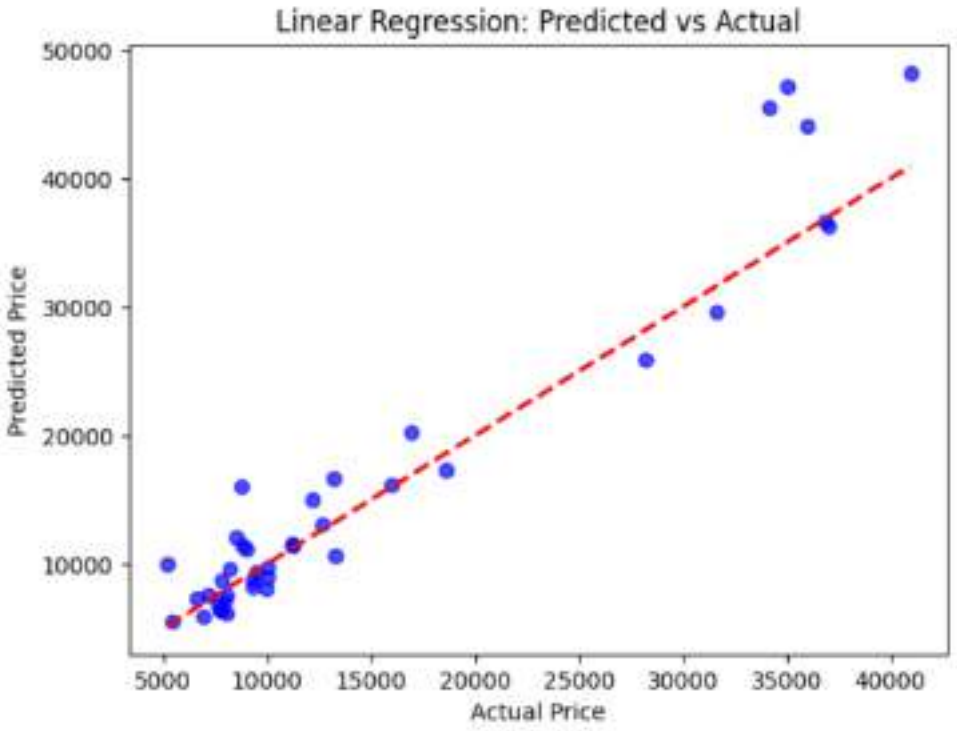
In [ ]:
```
preds_lr = model_lr.predict(X_test)
```

In [ ]:
```
mse_lr = mean_squared_error(y_test, preds_lr)
r2_lr = r2_score(y_test, preds_lr)
rmse_lr = np.sqrt(mse_lr)
```

In [ ]:
```
results.append({"Model": "Linear", "MSE": mse_lr, "R2": r2_lr, "RMSE": rmse_lr

print(f"Linear Regression ->\n R2: {r2_lr:.4f},\n MSE: {mse_lr:.2f},\n RMSE: {
```

```
Linear Regression ->
 R2: 0.8750,
 MSE: 14007858.67,
 RMSE: 3742.71
```

In [ ]:
```
plt.scatter(y_test, preds_lr, alpha=0.7, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.title("Linear Regression: Predicted vs Actual")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.show()
```



In [ ]:
```
model_ridge = Ridge(alpha=1.0)
model_ridge.fit(X_train, y_train)
preds_ridge = model_ridge.predict(X_test)
```

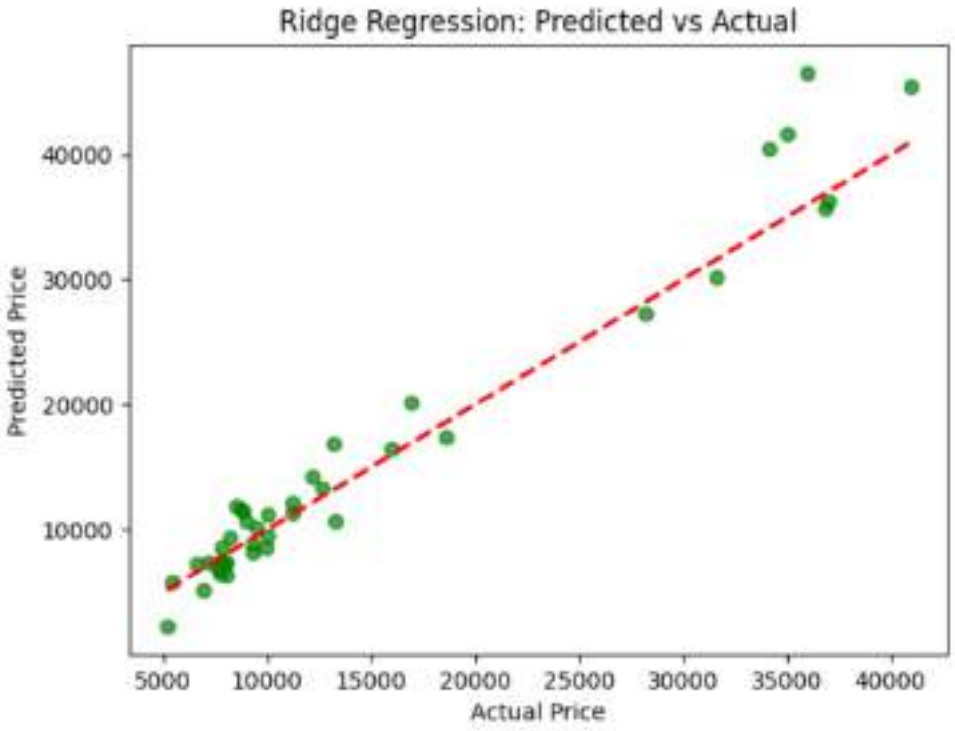
In [ ]:
```
mse_ridge = mean_squared_error(y_test, preds_ridge)
r2_ridge = r2_score(y_test, preds_ridge)
rmse_ridge = np.sqrt(mse_ridge)
```

In [ ]:
```
results.append({"Model": "Ridge", "MSE": mse_ridge, "R2": r2_ridge, "RMSE": rm
```

In [ ]:
```
print(f"Ridge Regression -> R2: {r2_ridge:.4f}, MSE: {mse_ridge:.2f}, RMSE: {r
```

```
Ridge Regression -> R2: 0.9304, MSE: 7794482.02, RMSE: 2791.86
```

In [ ]:
```
plt.scatter(y_test, preds_ridge, alpha=0.7, color='green')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.title("Ridge Regression: Predicted vs Actual")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.show()
```



In [ ]:
```
model_lasso = Lasso(alpha=1.0, max_iter=10000)
model_lasso.fit(X_train, y_train)
preds_lasso = model_lasso.predict(X_test)
```

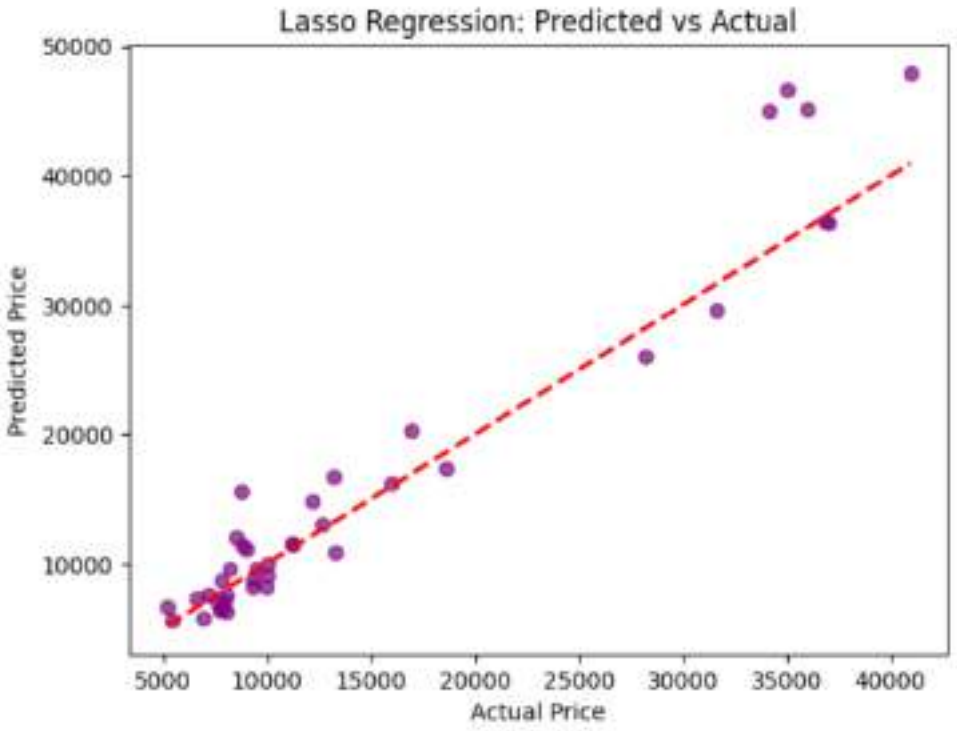
In [ ]:
```
mse_lasso = mean_squared_error(y_test, preds_lasso)
r2_lasso = r2_score(y_test, preds_lasso)
rmse_lasso = np.sqrt(mse_lasso)
```

In [ ]:
```
results.append({"Model": "Lasso", "MSE": mse_lasso, "R2": r2_lasso, "RMSE": rm
```

In [ ]:
```
print(f"Lasso Regression -> R2: {r2_lasso:.4f}, MSE: {mse_lasso:.2f}, RMSE: {r
```

```
Lasso Regression -> R2: 0.8831, MSE: 13094871.04, RMSE: 3618.68
```

In [ ]:
```
plt.scatter(y_test, preds_lasso, alpha=0.7, color="purple")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.title("Lasso Regression: Predicted vs Actual")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.show()
```



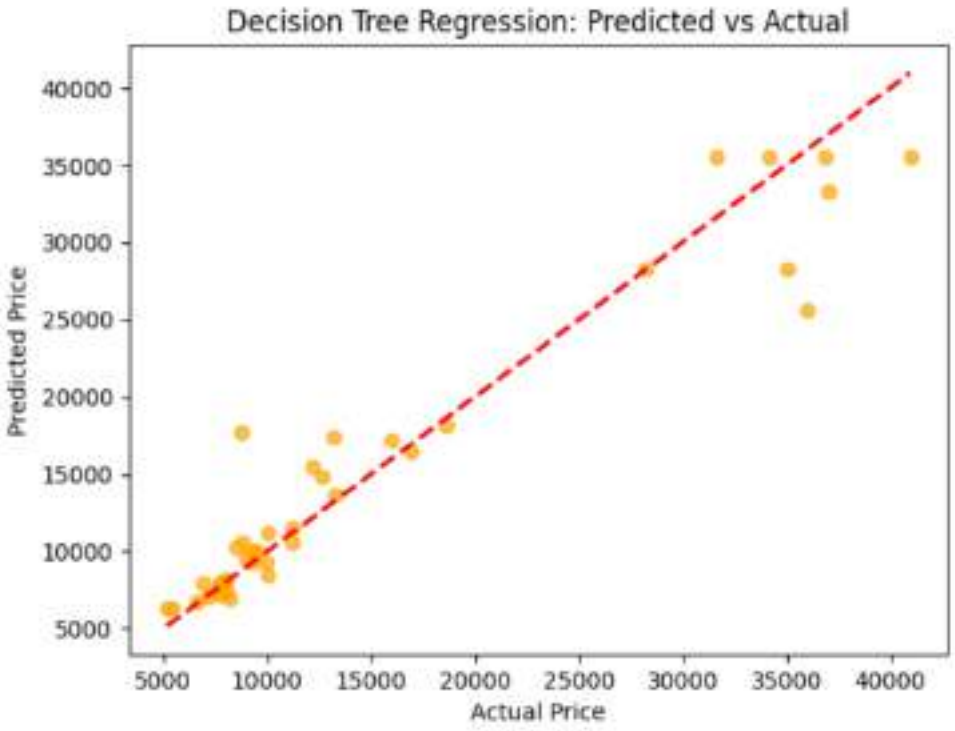
Decision Tree Regressor

In [ ]:
```
model_dt = DecisionTreeRegressor(random_state=42)
model_dt.fit(X_train, y_train)
preds_dt = model_dt.predict(X_test)
```

In [ ]:
```
mse_dt = mean_squared_error(y_test, preds_dt)
r2_dt = r2_score(y_test, preds_dt)
rmse_dt = np.sqrt(mse_dt)
results.append({"Model": "Decision Tree", "MSE": mse_dt, "R2": r2_dt, "RMSE":
print(f"Decision Tree Regressor -> R2: {r2_dt:.4f}, MSE: {mse_dt:.2f}, RMSE: {
```

```
Decision Tree Regressor -> R2: 0.9217, MSE: 8770303.55, RMSE: 2961.47
```

In [ ]:
```
plt.scatter(y_test, preds_dt, alpha=0.7, color='orange')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.title("Decision Tree Regression: Predicted vs Actual")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.show()
```



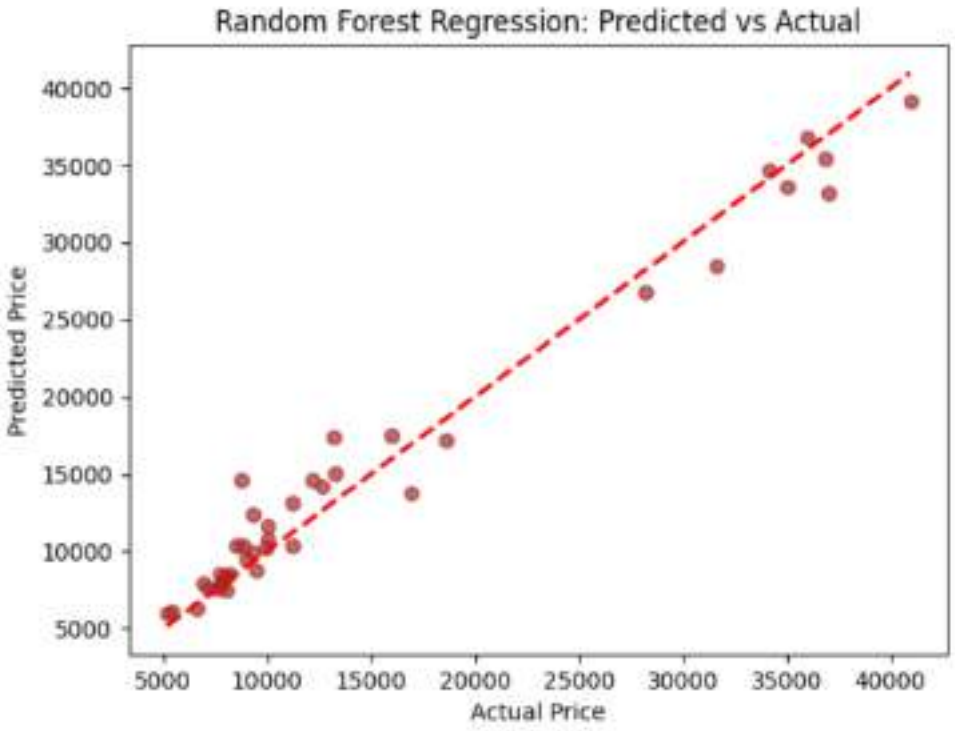
Random Forest Regressor

In [ ]:
```
model_rf = RandomForestRegressor(random_state=42)
model_rf.fit(X_train, y_train)
preds_rf = model_rf.predict(X_test)
```

In [ ]:
```
mse_rf = mean_squared_error(y_test, preds_rf)
r2_rf = r2_score(y_test, preds_rf)
rmse_rf = np.sqrt(mse_rf)
results.append({"Model": "Random Forest", "MSE": mse_rf, "R2": r2_rf, "RMSE":
print(f"Random Forest Regressor -> R2: {r2_rf:.4f}, MSE: {mse_rf:.2f}, RMSE: {
```

```
Random Forest Regressor -> R2: 0.9685, MSE: 3533105.77, RMSE: 1879.66
```

In [ ]:
```
plt.scatter(y_test, preds_rf, alpha=0.7, color='brown')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw
plt.title("Random Forest Regression: Predicted vs Actual")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.show()
```



Final Model Comparison

In [ ]:
```
print("\nFinal Model Comparison Summary:")
display(comparison_df)
```

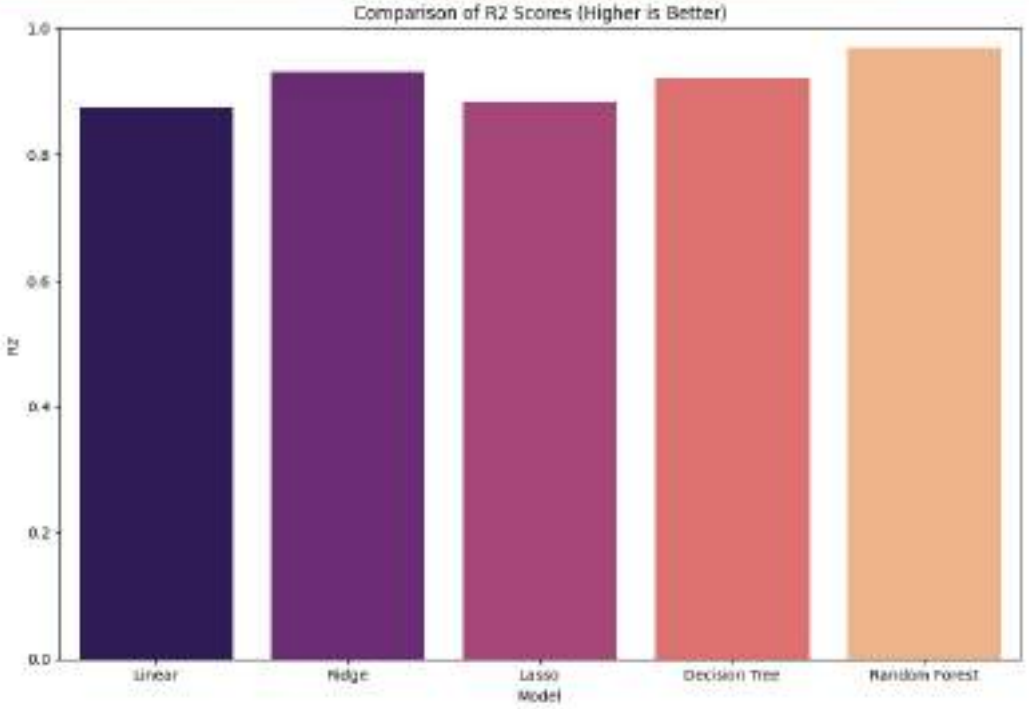
```
Final Model Comparison Summary:
```

	Model	MSE	R2	RMSE
0	Linear	1.400786e+07	0.874984	3742.707399
1	Ridge	7.794482e+06	0.930436	2791.859958
2	Lasso	1.309487e+07	0.883132	3618.683605
3	Decision Tree	8.770304e+06	0.921727	2961.469829
4	Random Forest	3.533106e+06	0.968468	1879.655759

In [ ]:
```
comparison_df = pd.DataFrame(results)

plt.figure(figsize=(10, 7))
sns.barplot(x='Model', y='R2', data=comparison_df, palette='magma')
plt.title("Comparison of R2 Scores (Higher is Better)")
plt.ylim(0, 1)
plt.tight_layout()
```

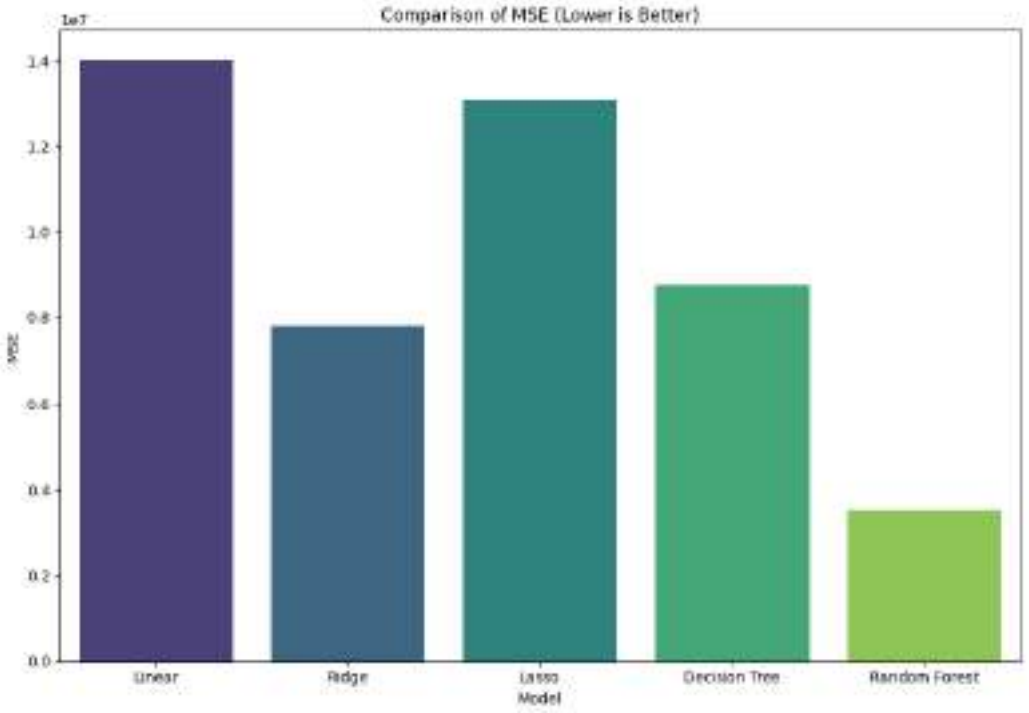
```
plt.show()
```

```
/tmp/ipykernel_6578/3386305981.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same e
ffect.

  sns.barplot(x='Model', y='R2', data=comparison_df, palette='magma')
```



[In [ ]]:
```
plt.figure(figsize=(10, 7))
sns.barplot(x='Model', y='MSE', data=comparison_df, palette='viridis')
plt.title("Comparison of MSE (Lower is Better)")
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_6578/3137539501.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same e
ffect.

  sns.barplot(x='Model', y='MSE', data=comparison_df, palette='viridis')
```


Comparison of MSE (Lower is Better)
1e7
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
MSE
Linear
Ridge
Lasso
Decision Tree
Random Forest
Model

Health Risk Level Prediction

Dataset	Health_Risk_Dataset.csv
Target Variable	Risk_Level (4 classes: Low, Medium, High, Critical)

1. Problem Definition

The goal of this project is to build a machine learning model that can classify a patient's health risk level based on their vital signs and clinical indicators. The output is one of four discrete categories Low, Medium, High, or Critical making this a multi-class classification problem, not a regression problem.

Parameter	Detail
Target Variable (y)	Risk_Level — 4 classes: Low / Medium / High / Critical
Nature of Target	Categorical, ordinal; encoded as integers via LabelEncoder (0, 1, 2, 3)

2. Data Understanding

The dataset is loaded from Health_Risk_Dataset.csv. Each row represents one patient record. The dataset includes a non-informative ID column, one mixed-type categorical column (Consciousness), several numerical vital sign measurements, and the target column Risk_Level. A correlation matrix is computed to understand feature relationships before modelling begins.

Feature Inventory (Input Features used in X):

Numerical Features (X):

- Respiratory_Rate
- Oxygen_Saturation
- O2_Scale
- Systolic_BP
- Heart_Rate
- Temperature
- On_Oxygen

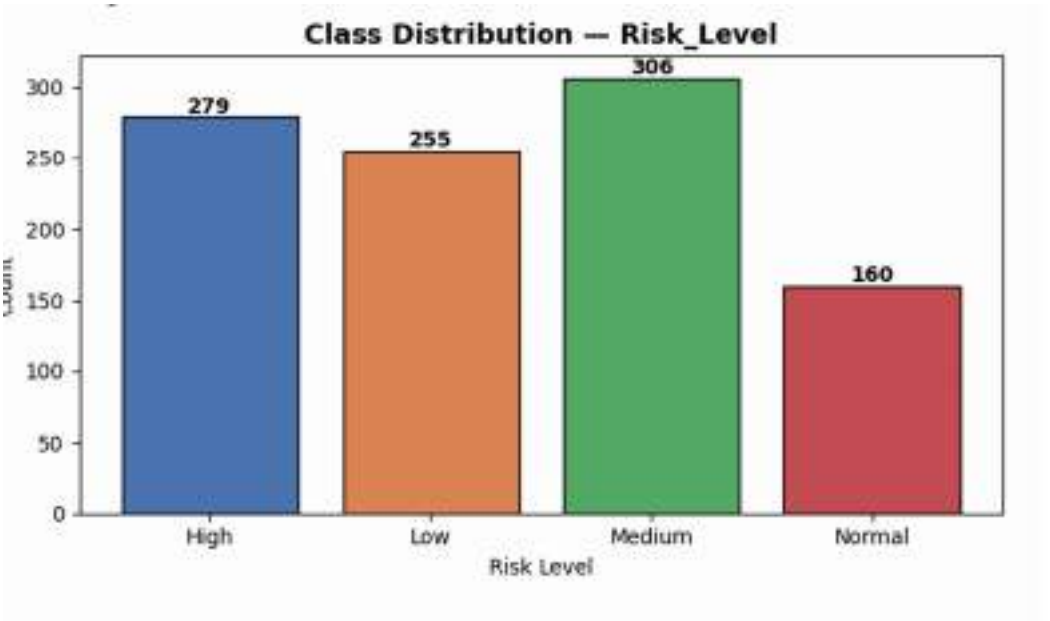
Categorical Features (X):

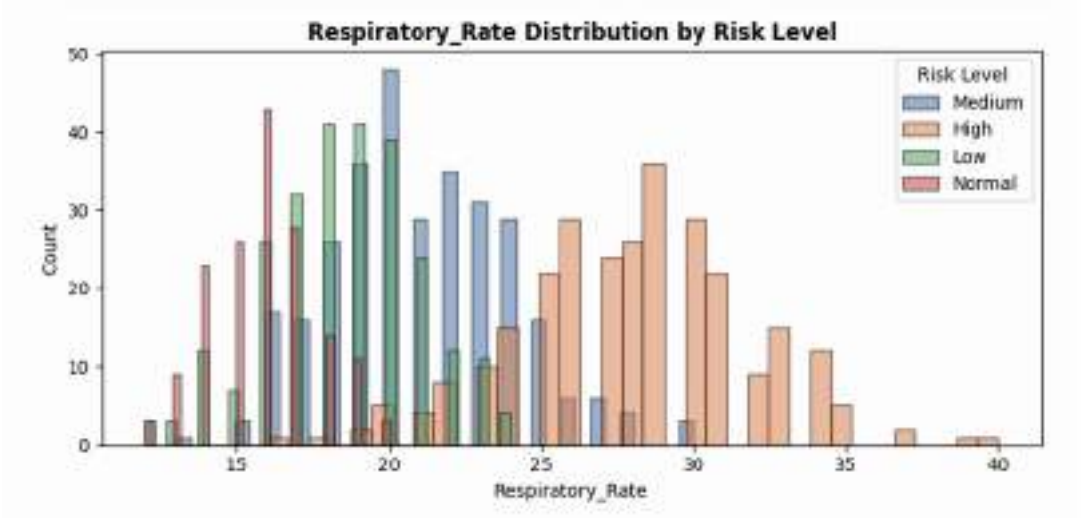
- Consciousness — Alert (A) / Responds to Pain (P) / Unresponsive (U)

Target Variable (y):

- Risk_Level — Normal/Low / Medium / High

Class Distribution :





3. Data Pre-Processing

The raw dataset requires three targeted preprocessing steps before any model can be trained.

Step 3a: Drop Patient_ID Column

Patient_ID is a unique row identifier with no predictive meaning. Including it would cause the model to memorise IDs instead of learning clinical patterns a classic form of data leakage.

df.drop(columns=['Patient_ID'])

Step 3b: Label Encode 'Consciousness'

Consciousness has 3 ordinal states (Alert < Confused < Unresponsive). LabelEncoder maps these to 0, 1, 2, preserving the inherent clinical order. One-Hot Encoding is avoided here because the ordinal relationship is medically meaningful.

```
le_cons = LabelEncoder()df['Consciousness'] = le_cons.fit_transform(df['Consciousness'])
```

Step 3c: Label Encode Target

The 4-class target (Low, Medium, High, Critical) must be integer-encoded for sklearn classifiers. le_target.classes_ is saved so original class names can be restored in classification reports and confusion matrix labels.

```
le_target = LabelEncoder()df['Risk_Level'] = le_target.fit_transform(df['Risk_Level'])
```

Step 3d: Feature Scaling

Vital signs vary in magnitude; without standardization, larger values dominate Logistic Regression. Fit your scaler only on X_train to prevent data leakage and ensure model integrity.

```
scaler = StandardScaler()X_train_scaled = scaler.fit_transform(X_train)X_test_scaled  = scaler.transform(X_test)
```

4. Splitting Point

The preprocessed feature matrix X and target vector y are split before scaling is applied, ensuring the scaler learns only from training statistics.

Split Parameter	Value	Reasoning
test_size	0.2 (20%)	Standard 80/20 split; enough test samples for reliable metric estimation across all 4 risk classes.
Train size	800 rows	Used to fit all 5 models. Scaler is also fitted only on this subset.
Test size	200 rows	Completely held-out. Used only for predict() and metric calculation.

5. Model Training

Five classification models are trained, spanning from simple linear classifiers to ensemble methods. Each was chosen for a specific reason tied to the multi-class nature of health risk prediction:

1. Logistic Regression

This unregularized multinomial baseline uses Softmax to model four classes simultaneously. It serves as a benchmark; if regularization provides no improvement, the features are already well-separated.

```
m1 = LogisticRegression(  penalty=None,  multi_class='multinomial',  max_iter=10000)m1.fit(X_train_scaled, y_train)
```

2. Logistic Regression(L10

Using a SAGA solver, this L1-regularized model (C=0.1) performs feature selection by zeroing out redundant coefficients. This is ideal for health data where overlapping vital signs often provide duplicate information. .

```
m2 = LogisticRegression(  penalty='l1', solver='saga',  multi_class='multinomial',  max_iter=5000, C=0.1)
```

3. Logistic Regression (L2)

This **L2-regularized** model (C=0.01) shrinks coefficients toward zero without eliminating them. It is used to determine if keeping all features at small weights outperforms the selection-heavy L1 approach.

```
m3 = LogisticRegression(  penalty='l2', solver='saga',  multi_class='multinomial',  max_iter=1000, C=0.01)
```

4. Decision Tree Classifier

This rule-based classifier uses a depth of five to ensure medical interpretability and prevent overfitting, mirroring clinical diagnostic logic.

```
m4 = DecisionTreeClassifier(  max_depth=5, random_state=42)
```

5. Bagging Classifier

By averaging 50 diverse trees, this ensemble reduces variance and stabilizes predictions, balancing computational speed with robust, reliable accuracy.

```
m5 = BaggingClassifier(  estimator=DecisionTreeClassifier(max_depth=5),  n_estimators=50, random_state=42)
```

6. XG Boost

This gradient boosting model builds trees sequentially, with each new tree correcting errors from previous ones. It offers high performance and efficiency through advanced regularization and optimized computation.

```
XGBClassifier(random_state=RANDOM_STATE, use_label_encoder=False')
```

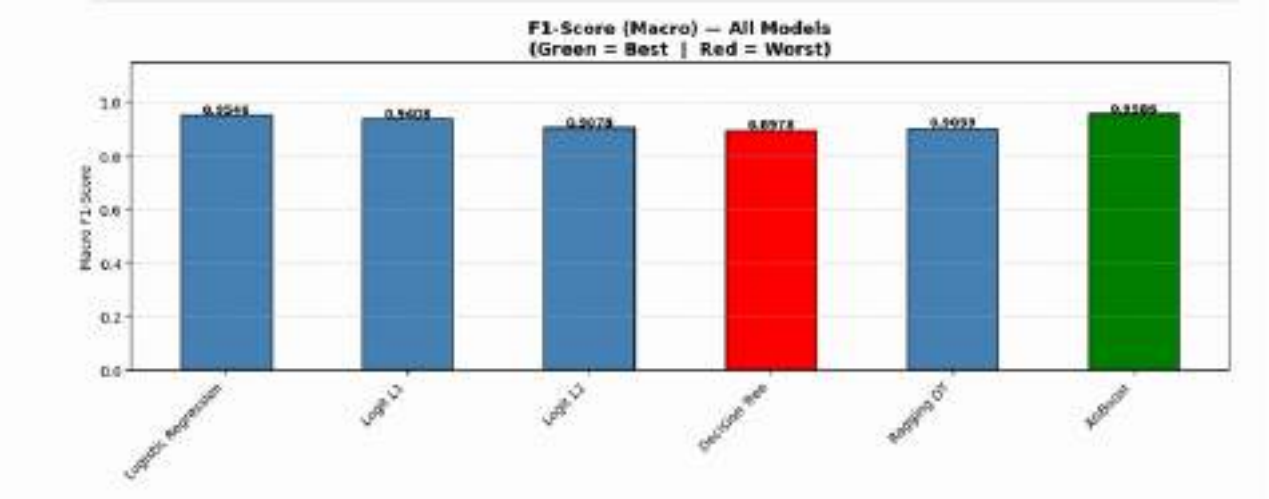
6. Test Model

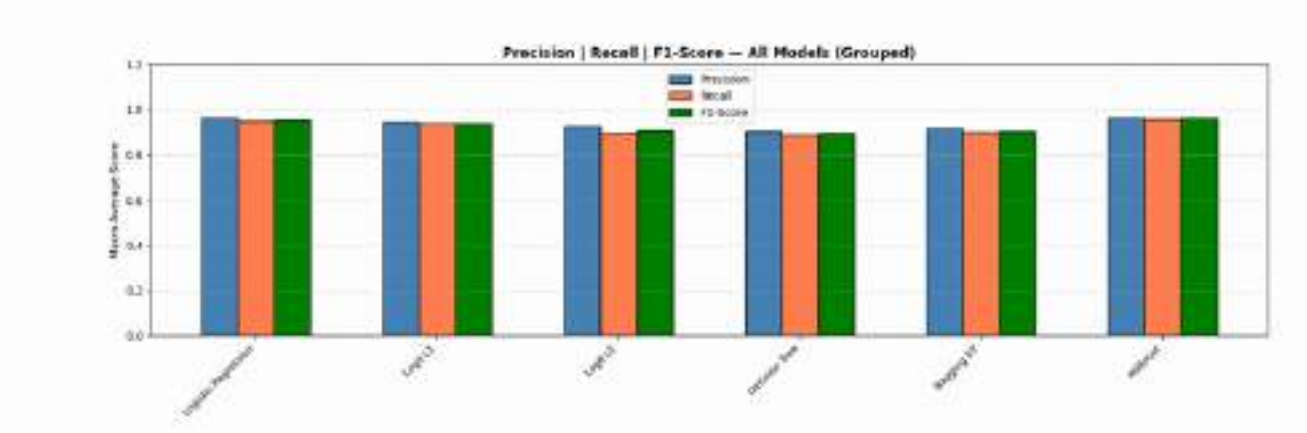
Each model is evaluated on the held-out test set using four complementary methods: a full classification report, per-class confusion matrices, a comparative bar chart, and macro-averaged ROC curves with AUC scores.

Evaluation Metrics Explained:

Metric	Explanation
Precision (macro avg)	Macro average precision measures correct class predictions, emphasizing low false positives.
Recall (macro avg)	Recall measures the fraction of true cases correctly identified, which is vital for catching all critical patients.
F1-Score (macro avg)	The harmonic mean of precision and recall, balancing both metrics. It is used here to ensure the model doesn't ignore minority risk classes.
ROC + AUC	evaluates each class binary, averaging the results. Using probability scores, it compares all models to determine discrimination excellence.

	Model	Precision	Recall	F1-Score	AUC	Accuracy
1	XGBoost	0.961230	0.956258	0.958598	0.998877	0.955
2	Logistic Regression	0.961400	0.950071	0.954576	0.996385	0.955
3	Logit L1	0.945012	0.937838	0.940833	0.995876	0.940
4	Logit L2	0.925307	0.897222	0.907773	0.980754	0.910
5	Bagging DT	0.914488	0.899364	0.905934	0.987020	0.900
6	Decision Tree	0.906018	0.891605	0.897761	0.961525	0.890





---

7. Conclusion

1. Stratified splitting ensured fair class representation. With 4 imbalanced classes , Medium (306), High (279), Low (255), and Normal (160) , using stratify=y preserved each class's proportion in the test set, making macro F1 and AUC scores reliable across all risk groups, especially the minority Normal class.

2. XGBoost emerged as the best overall model, achieving the highest macro F1 (0.959) and AUC (0.999), along with 95.5% accuracy , demonstrating that gradient boosting captures complex, non-linear interactions between vital signs far better than linear or shallow tree-based models.

3. Logistic Regression variants revealed clear regularisation tradeoffs. Vanilla LR (F1 = 0.955) outperformed L1 (0.941) and L2 (0.908), showing that aggressive regularisation hurt performance here ,L2 at C=0.01 notably degraded recall for Low and Normal classes by over-shrinking informative feature weights.

4. Bagging improved on a single Decision Tree but couldn't close the gap with linear models. The single Decision Tree (F1 = 0.898) was the weakest model; 50 bootstrapped trees in the BaggingClassifier raised the macro F1 to 0.906, confirming variance reduction , but both lagged behind even L2 Logistic Regression.

5. High AUC across all models confirmed strong probabilistic separation of risk levels. Every model scored above 0.96 AUC, meaning all models rank patients reliably by risk , making threshold tuning at deployment a viable strategy to further optimise recall for the Critical and High risk groups in a hospital triage context.

In [257…
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder, label_binarize
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, classif
                             f1_score, precision_score, recall_score,
                             roc_curve, auc)
from xgboost import XGBClassifier
```

In [258…
```
df = pd.read_csv('/content/drive/MyDrive/AIML_lab_sem4/Project/Health_Risk_Dat
```

In [259…
```
df.head()
```

Out[259…

	Patient_ID	Respiratory_Rate	Oxygen_Saturation	O2_Scale	Systolic_BP	Hear
0	P0522	25	96	1	97	
1	P0738	28	92	2	116	
2	P0741	29	91	1	79	
3	P0661	24	96	1	95	
4	P0412	20	96	1	97	

In [260…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   Patient_ID         1000 non-null   object 
 1   Respiratory_Rate   1000 non-null   int64  
 2   Oxygen_Saturation  1000 non-null   int64  
 3   O2_Scale           1000 non-null   int64  
 4   Systolic_BP        1000 non-null   int64  
 5   Heart_Rate         1000 non-null   int64  
 6   Temperature        1000 non-null   float64
 7   Consciousness      1000 non-null   object 
 8   On_Oxygen          1000 non-null   int64  
 9   Risk_Level         1000 non-null   object 
dtypes: float64(1), int64(6), object(3)
memory usage: 78.3+ KB
```

In [261…
```
df.isnull().sum()
```

Out[261…

	0
Patient_ID	0
Respiratory_Rate	0
Oxygen_Saturation	0
O2_Scale	0
Systolic_BP	0
Heart_Rate	0
Temperature	0
Consciousness	0
On_Oxygen	0
Risk_Level	0

dtype: int64

In [262…
```
df.describe()
```

Out[262…

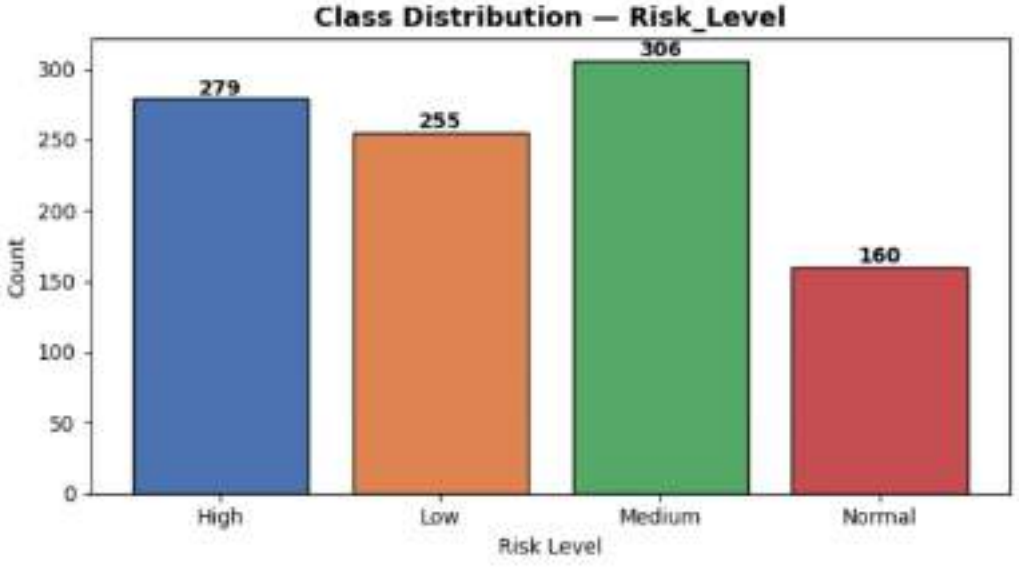
	Respiratory_Rate	Oxygen_Saturation	O2_Scale	Systolic_BP	Heart_R
count	1000.000000	1000.00000	1000.000000	1000.000000	1000.000
mean	21.511000	92.59000	1.124000	106.160000	98.460
std	5.287517	4.47302	0.329746	17.897562	19.694
min	12.000000	74.00000	1.000000	50.000000	60.000
25%	17.000000	90.00000	1.000000	94.000000	84.000
50%	20.000000	94.00000	1.000000	109.000000	95.500
75%	25.000000	96.00000	1.000000	119.000000	109.000
max	40.000000	100.00000	2.000000	146.000000	163.000

In [263…
```
TARGET = 'Risk_Level'

print(f"Target column : '{TARGET}'")
print('\nClass Distribution (before encoding):')
print(df[TARGET].value_counts())
print(f'\nTotal classes : {df[TARGET].nunique()}')
```

```
Target column : 'Risk_Level'

Class Distribution (before encoding):
Risk_Level
Medium    306
High      279
Low       255
Normal    160
Name: count, dtype: int64

Total classes : 4
```

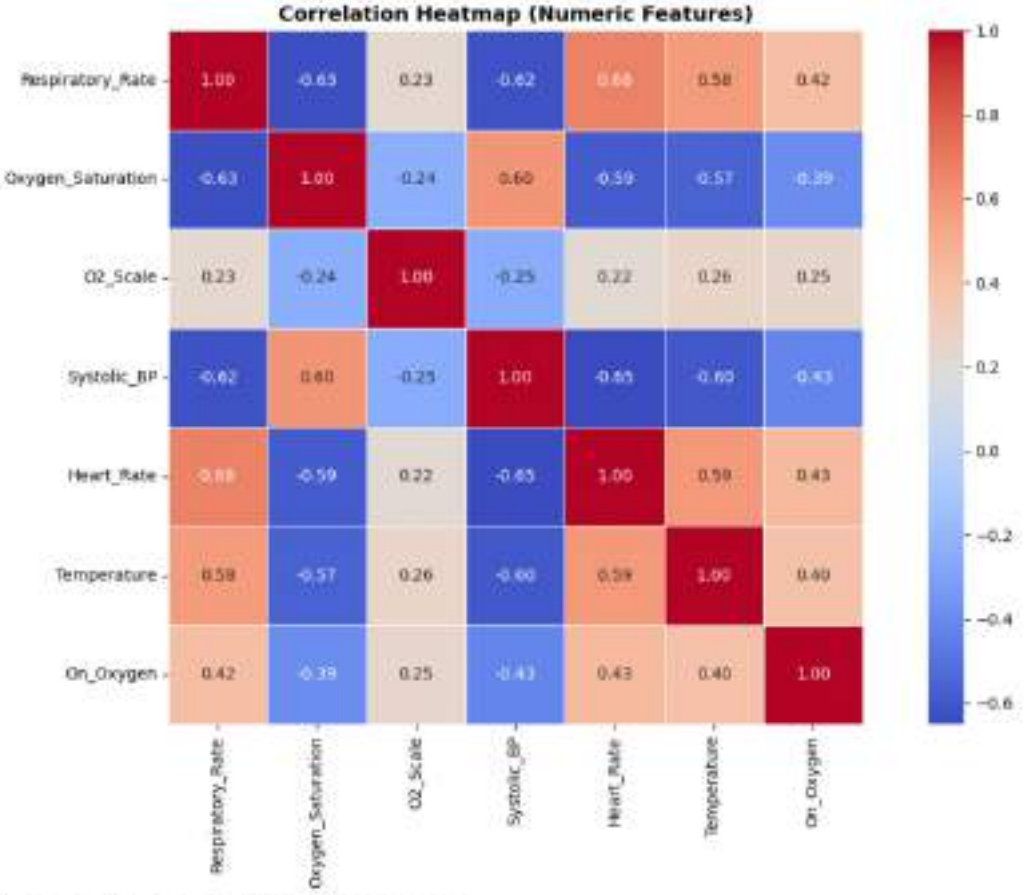
In [264…
```python
print('Generating EDA Plots...')

counts = df[TARGET].value_counts().sort_index()
colors_cls = ['#4C72B0', '#DD8452', '#55A868', '#C44E52', '#8172B3']

plt.figure(figsize=(7, 4))
bars = plt.bar(counts.index.astype(str), counts.values,
               color=colors_cls[:len(counts)], edgecolor='black')
for bar in bars:
    plt.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + max(counts.values) * 0.01,
             str(bar.get_height()), ha='center', fontweight='bold', fontsize=1
plt.title(f'Class Distribution — {TARGET}', fontsize=13, fontweight='bold')
plt.xlabel('Risk Level')
plt.ylabel('Count')
plt.tight_layout()
plt.savefig('plot1_class_distribution.png', dpi=150)
plt.show()
print('Saved: plot1_class_distribution.png')
```

```
Generating EDA Plots...
```



```
Saved: plot1_class_distribution.png
```

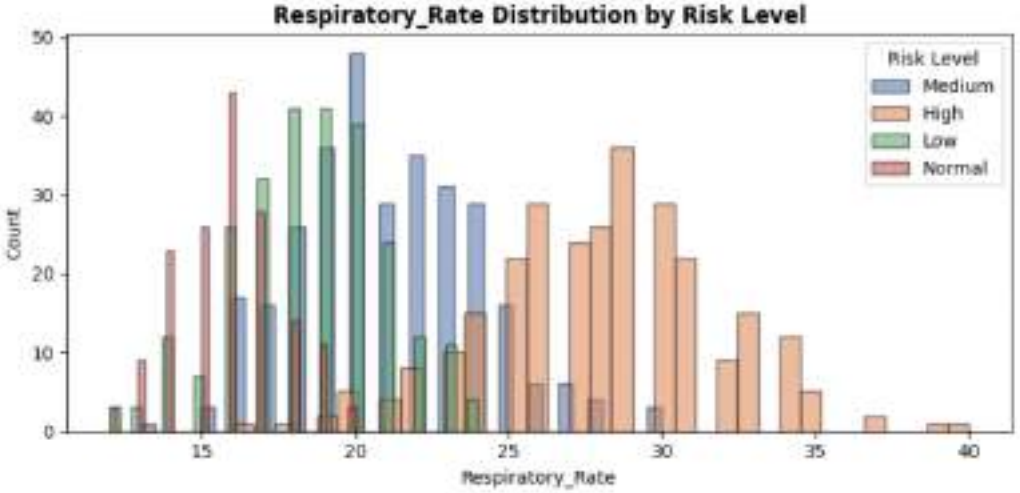
In [265…
```
numeric_df = df.select_dtypes(include=[np.number])

plt.figure(figsize=(12, 8))
sns.heatmap(numeric_df.corr(), annot=True, fmt='.2f',
            cmap='coolwarm', linewidths=0.5, square=True)
plt.title('Correlation Heatmap (Numeric Features)', fontsize=13, fontweight='b
plt.tight_layout()
plt.savefig('plot3_correlation_heatmap.png', dpi=150)
plt.show()
print('Saved: plot3_correlation_heatmap.png')
```




```
Saved: plot3_correlation_heatmap.png
```

In [266…
```
KEY_FEATURE = numeric_df.columns[0]   # change to preferred column if desired

plt.figure(figsize=(8, 4))
for cls, clr in zip(df[TARGET].unique(), colors_cls):
    subset = df[df[TARGET] == cls][KEY_FEATURE].dropna()
    plt.hist(subset, bins=35, alpha=0.55, color=clr,
```

```
            label=str(cls), edgecolor='black')
plt.title(f'{KEY_FEATURE} Distribution by Risk Level', fontsize=12, fontweight
plt.xlabel(KEY_FEATURE)
plt.ylabel('Count')
plt.legend(title='Risk Level')
plt.tight_layout()
plt.savefig('plot4_feature_by_class.png', dpi=150)
plt.show()
print('Saved: plot4_feature_by_class.png')
```




```
Saved: plot4_feature_by_class.png
```

Preprocessing

In [267…
```
df = df.drop(columns=['Patient_ID'], errors='ignore')
```

In [268…
```
le_cons = LabelEncoder()
df['Consciousness'] = le_cons.fit_transform(df['Consciousness'])
print('Encoded: Consciousness')
```

```
Encoded: Consciousness
```

In [269…
```
le_target = LabelEncoder()
df[TARGET]  = le_target.fit_transform(df[TARGET])
```

In [270…
```
X = df.drop(columns=[TARGET])
y = df[TARGET]
```

In [271…
```
RANDOM_STATE = 42

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
```

In [272…
```
scaler         = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

In [273…
```
m1 = LogisticRegression(penalty=None, multi_class='multinomial',
                        solver='lbfgs', max_iter=10000, random_state=RANDOM_ST
m1.fit(X_train_scaled, y_train)
p1 = m1.predict(X_test_scaled)

print('--- Model 1: Logistic Regression ---')
print(classification_report(y_test, p1, target_names=le_target.classes_))
```

```
--- Model 1: Logistic Regression ---
              precision    recall  f1-score   support

        High       1.00      0.95      0.97        56
         Low       0.91      0.98      0.94        51
      Medium       0.94      0.97      0.95        61
      Normal       1.00      0.91      0.95        32

    accuracy                           0.95       200
   macro avg       0.96      0.95      0.95       200
weighted avg       0.96      0.95      0.96       200
```

In [274…
```
print('Manually tuning L1 Logistic Regression...')
m2 = LogisticRegression(penalty='l1', solver='saga', multi_class='multinomial'
                        max_iter=5000, C=0.1, random_state=RANDOM_STATE)
m2.fit(X_train_scaled, y_train)
p2 = m2.predict(X_test_scaled)
print('L1 Logistic Regression (C=0.1) fitted.')

print('\n--- Model 2: Logistic Regression L1 ---')
print(classification_report(y_test, p2, target_names=le_target.classes_))
```

```
Manually tuning L1 Logistic Regression...
L1 Logistic Regression (C=0.1) fitted.

--- Model 2: Logistic Regression L1 ---
              precision    recall  f1-score   support

        High       1.00      0.96      0.98        56
         Low       0.92      0.88      0.90        51
      Medium       0.89      0.97      0.93        61
      Normal       0.97      0.94      0.95        32

    accuracy                           0.94       200
   macro avg       0.95      0.94      0.94       200
weighted avg       0.94      0.94      0.94       200
```

In [275…
```
print('Manually tuning L2 Logistic Regression...')
m3 = LogisticRegression(penalty='l2', solver='lbfgs', multi_class='multinomial
                        max_iter=1000, C=0.01, random_state=RANDOM_STATE)
```

```
m3.fit(X_train_scaled, y_train)
p3 = m3.predict(X_test_scaled)
print('L2 Logistic Regression (C=0.01) fitted.')

print('\n--- Model 3: Logistic Regression L2 ---')
print(classification_report(y_test, p3, target_names=le_target.classes_))
```

```
Manually tuning L2 Logistic Regression...
L2 Logistic Regression (C=0.01) fitted.

--- Model 3: Logistic Regression L2 ---
              precision    recall  f1-score   support

        High       1.00      0.95      0.97        56
         Low       0.85      0.86      0.85        51
      Medium       0.86      0.97      0.91        61
      Normal       1.00      0.81      0.90        32

    accuracy                           0.91       200
   macro avg       0.93      0.90      0.91       200
weighted avg       0.92      0.91      0.91       200
```

In [276…
```
m4 = DecisionTreeClassifier(max_depth=5, random_state=RANDOM_STATE)
m4.fit(X_train_scaled, y_train)
p4 = m4.predict(X_test_scaled)

print('--- Model 4: Decision Tree ---')
print(classification_report(y_test, p4, target_names=le_target.classes_))
```

```
--- Model 4: Decision Tree ---
              precision    recall  f1-score   support

        High       0.98      0.93      0.95        56
         Low       0.81      0.86      0.84        51
      Medium       0.83      0.87      0.85        61
      Normal       1.00      0.91      0.95        32

    accuracy                           0.89       200
   macro avg       0.91      0.89      0.90       200
weighted avg       0.90      0.89      0.89       200
```

In [277…
```
m5 = BaggingClassifier(estimator=DecisionTreeClassifier(max_depth=5),
                       n_estimators=50, random_state=RANDOM_STATE)
m5.fit(X_train_scaled, y_train)
p5 = m5.predict(X_test_scaled)

print('--- Model 5: Bagging Classifier ---')
print(classification_report(y_test, p5, target_names=le_target.classes_))
```

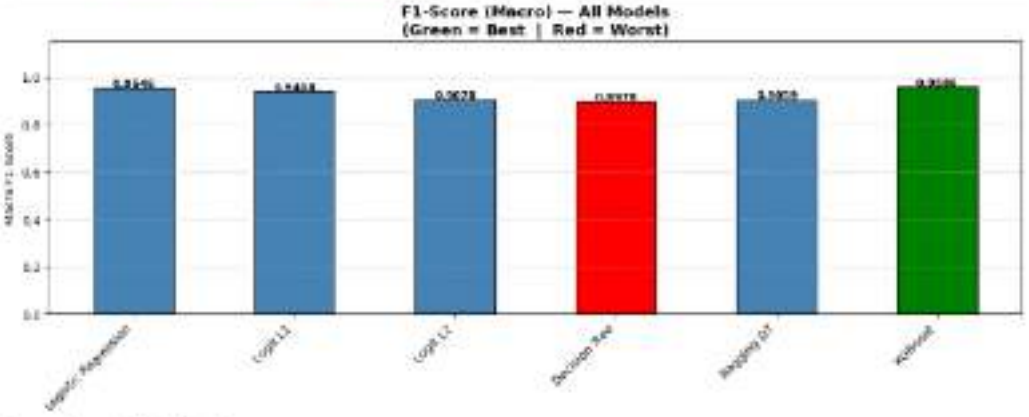
```
--- Model 5: Bagging Classifier ---
              precision    recall  f1-score   support

        High       0.98      0.95      0.96        56
         Low       0.84      0.84      0.84        51
      Medium       0.83      0.90      0.87        61
      Normal       1.00      0.91      0.95        32

    accuracy                           0.90       200
   macro avg       0.91      0.90      0.91       200
weighted avg       0.90      0.90      0.90       200
```

In [278…
```
m_xgb = XGBClassifier(random_state=RANDOM_STATE, use_label_encoder=False, eval
m_xgb.fit(X_train_scaled, y_train)
p_xgb = m_xgb.predict(X_test_scaled)

print('\n--- Model 6: XGBoost Classifier ---')
print(classification_report(y_test, p_xgb, target_names=le_target.classes_))
```

```
--- Model 6: XGBoost Classifier ---
              precision    recall  f1-score   support

        High       0.96      0.96      0.96        56
         Low       0.96      0.94      0.95        51
      Medium       0.92      0.95      0.94        61
      Normal       1.00      0.97      0.98        32

    accuracy                           0.95       200
   macro avg       0.96      0.96      0.96       200
weighted avg       0.96      0.95      0.96       200
```

In [280…
```
bar_colors_f1 = [
    'green'     if v == max(f1_macro_scores) else
    'red'       if v == min(f1_macro_scores) else
    'steelblue'
    for v in f1_macro_scores
]

plt.figure(figsize=(12, 5))
bars = plt.bar(model_names, f1_macro_scores,
               color=bar_colors_f1, edgecolor='black', width=0.5)
for bar in bars:
    plt.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + 0.005,
             f'{bar.get_height():.4f}',
             ha='center', fontsize=9, fontweight='bold')
plt.title('F1-Score (Macro) — All Models\n(Green = Best  |  Red = Worst)',
          fontsize=13, fontweight='bold')
plt.ylabel('Macro F1-Score')
plt.ylim(0, 1.15)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.6)
```

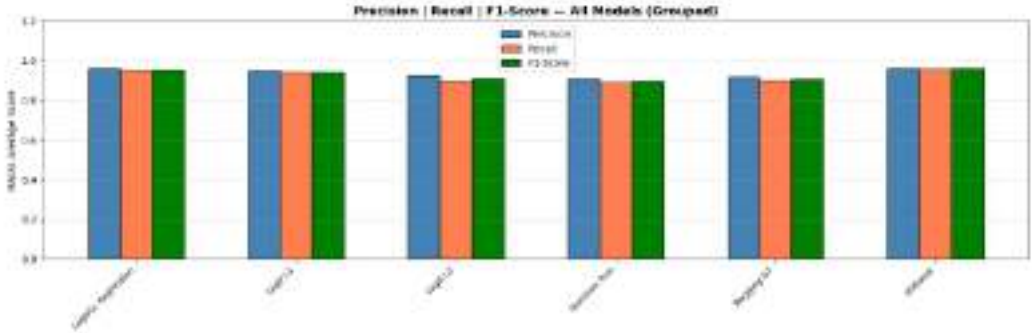
```
plt.tight_layout()
plt.savefig('plot5_f1_bar.png', dpi=150)
plt.show()
print('Saved: plot5_f1_bar.png')
```



Saved: plot5_f1_bar.png

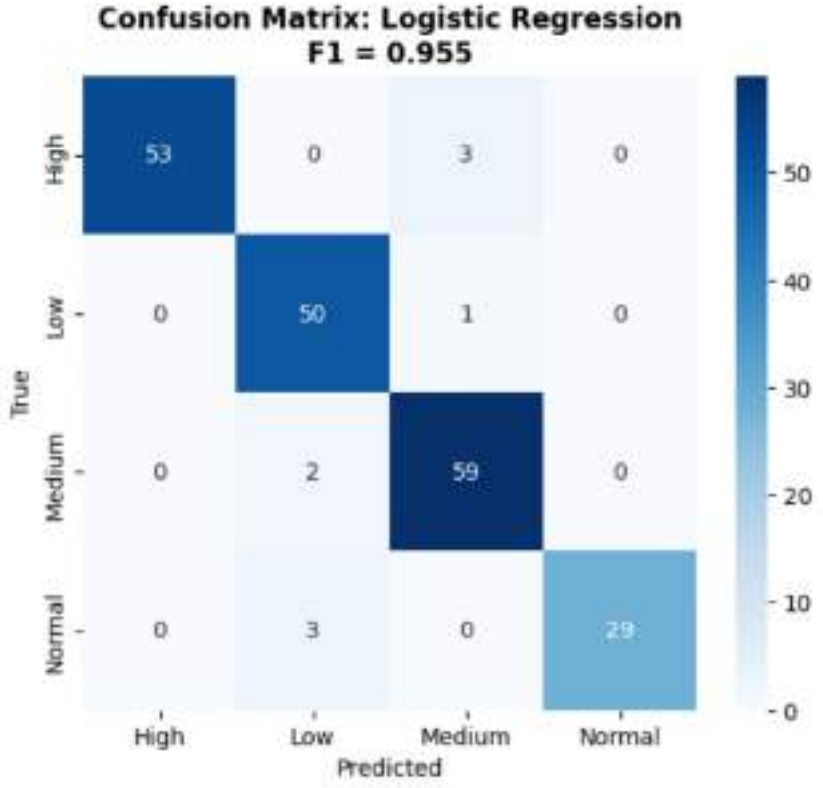
In [281…

```
x = np.arange(len(model_names))
w = 0.2

plt.figure(figsize=(15, 5))
plt.bar(x - w, precision_macro_scores, w,
        label='Precision', color='steelblue', edgecolor='black')
plt.bar(x,     recall_macro_scores,    w,
        label='Recall',    color='coral',     edgecolor='black')
plt.bar(x + w, f1_macro_scores,        w,
        label='F1-Score',  color='green',     edgecolor='black')

plt.xticks(x, model_names, rotation=45, ha='right', fontsize=9)
plt.ylabel('Macro Average Score')
plt.ylim(0, 1.2)
plt.title('Precision | Recall | F1-Score — All Models (Grouped)',
          fontsize=13, fontweight='bold')
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('plot6_grouped_bar.png', dpi=150)
plt.show()
print('Saved: plot6_grouped_bar.png')
```



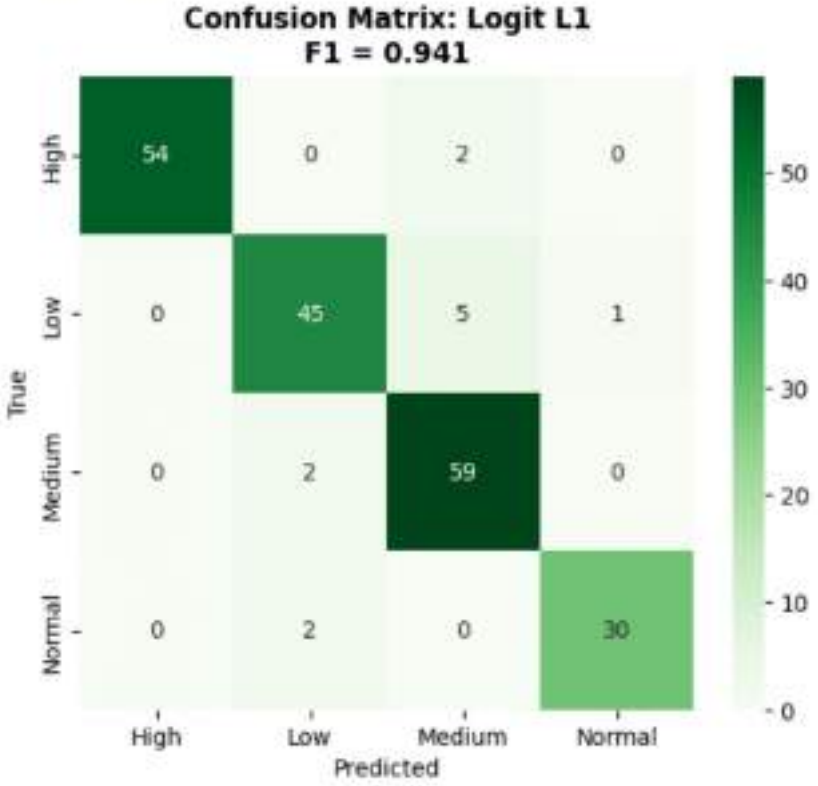
Saved: plot6_grouped_bar.png

In [282…
```
# Model 1: Baseline
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, p1), annot=True, fmt='d', cmap='Blues',
            xticklabels=le_target.classes_, yticklabels=le_target.classes_)
plt.title(f'Confusion Matrix: {model_names[0]}\nF1 = {f1_macro_scores[0]:.3f}')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```



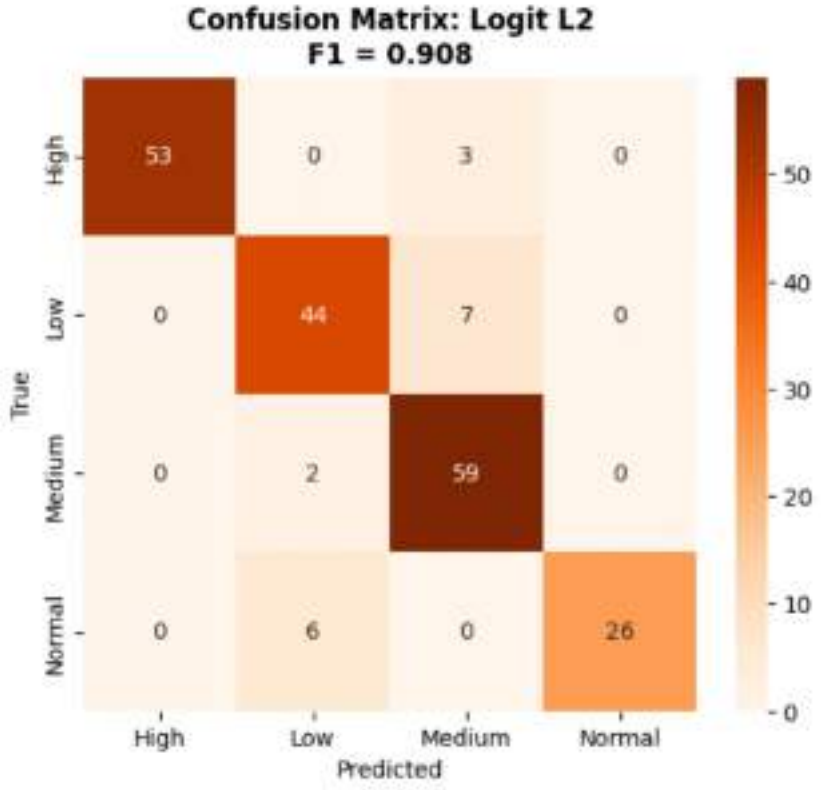
In [283…
```
# Model 2: Lasso (L1)
```

```
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, p2), annot=True, fmt='d', cmap='Greens',
            xticklabels=le_target.classes_, yticklabels=le_target.classes_)
plt.title(f'Confusion Matrix: {model_names[1]}\nF1 = {f1_macro_scores[1]:.3f}')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```



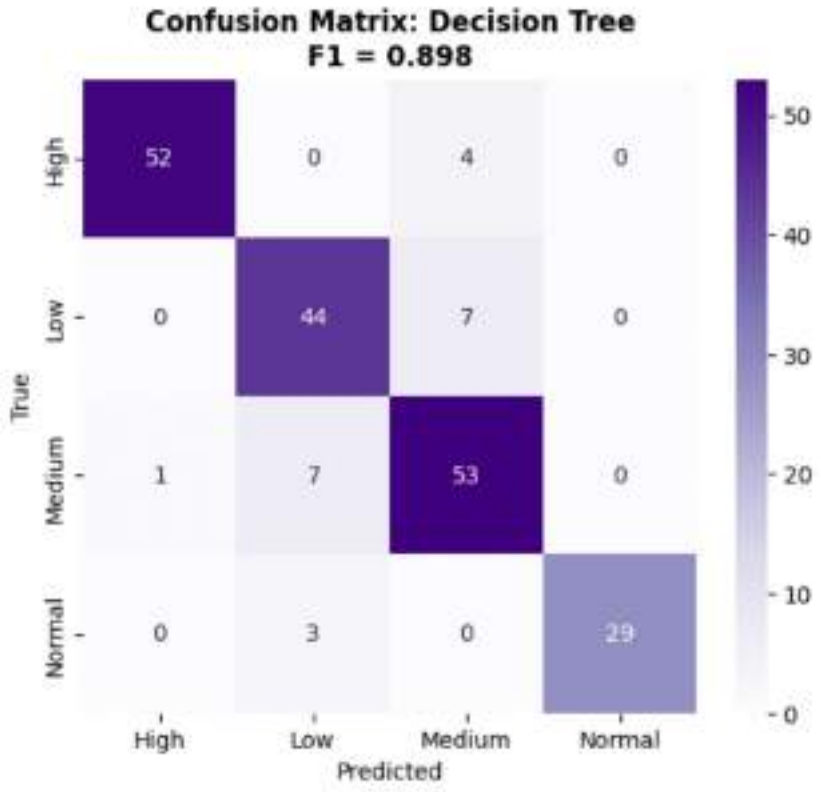
In [284…

```
# Model 3: Ridge (L2)
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, p3), annot=True, fmt='d', cmap='Oranges',
            xticklabels=le_target.classes_, yticklabels=le_target.classes_)
plt.title(f'Confusion Matrix: {model_names[2]}\nF1 = {f1_macro_scores[2]:.3f}')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```



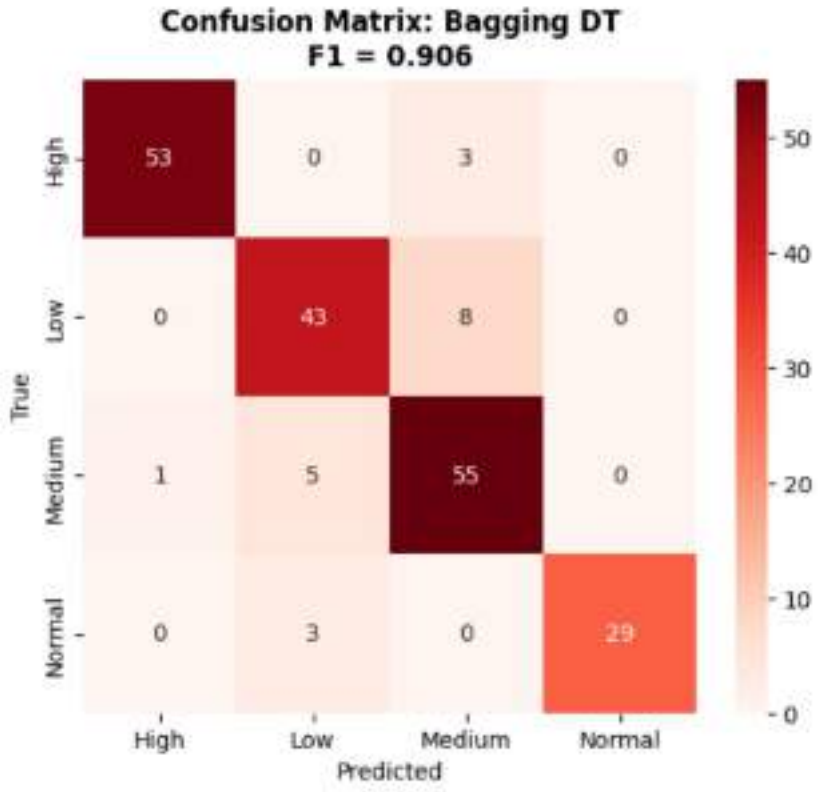
In [285…

```
# Model 4: Decision Tree
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, p4), annot=True, fmt='d', cmap='Purples',
            xticklabels=le_target.classes_, yticklabels=le_target.classes_)
plt.title(f'Confusion Matrix: {model_names[3]}\nF1 = {f1_macro_scores[3]:.3f}')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```



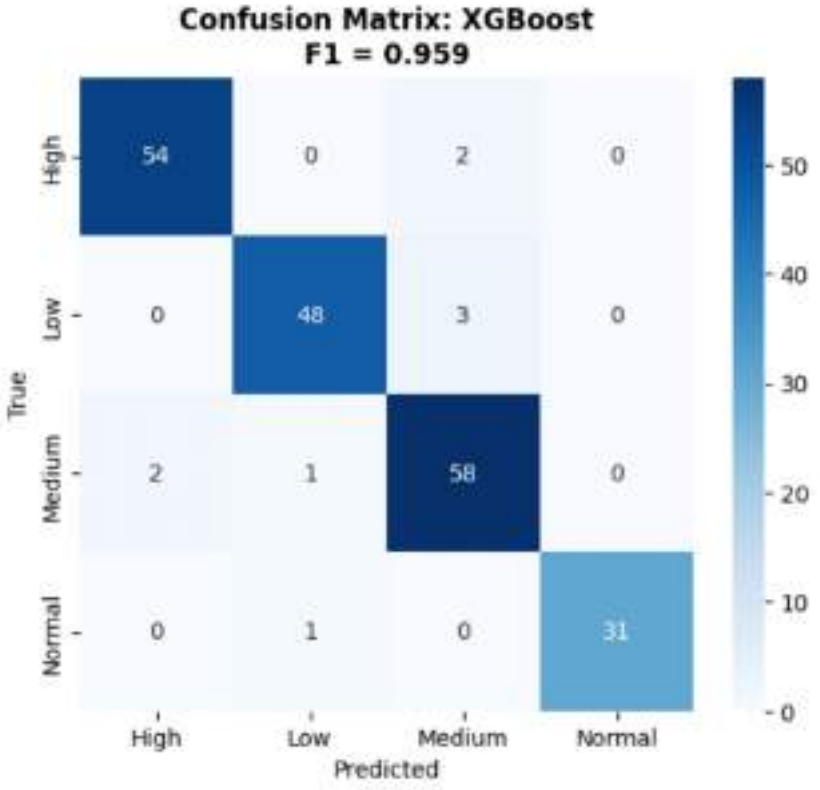
In [286…

```
# Model 5: Random Forest
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, p5), annot=True, fmt='d', cmap='Reds',
            xticklabels=le_target.classes_, yticklabels=le_target.classes_)
plt.title(f'Confusion Matrix: {model_names[4]}\nF1 = {f1_macro_scores[4]:.3f}')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```



In [287…

```
# Model 6: XGBoost
plt.figure(figsize=(6, 5))
sns.heatmap(confusion_matrix(y_test, p_xgb), annot=True, fmt='d', cmap='Blues'
            xticklabels=le_target.classes_, yticklabels=le_target.classes_)
plt.title(f'Confusion Matrix: {model_names[5]}\nF1 = {f1_macro_scores[5]:.3f}'
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

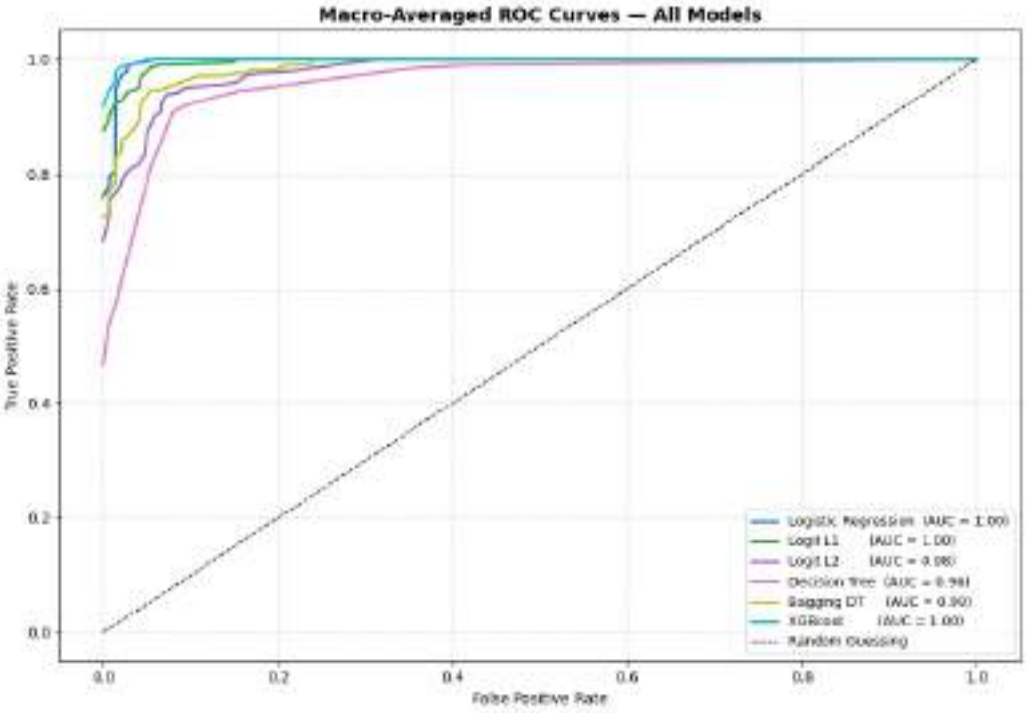


In [288…

```
y_test_bin = label_binarize(y_test, classes=np.unique(y_train))
n_classes  = y_test_bin.shape[1]

models_roc = [
    ('Logistic Regression', m1),
    ('Logit L1',        m2),
    ('Logit L2',        m3),
    ('Decision Tree',  m4),
    ('Bagging DT',     m5),
    ('XGBoost',         m_xgb)
]

roc_colors = plt.cm.tab10(np.linspace(0, 1, len(models_roc)))
auc_scores_list = []

plt.figure(figsize=(10, 7))

y_score_m1 = m1.predict_proba(X_test_scaled)
fpr_m1, tpr_m1, roc_auc_m1 = {}, {}, {}
for i in range(n_classes):
    fpr_m1[i], tpr_m1[i], _ = roc_curve(y_test_bin[:, i], y_score_m1[:, i])
    roc_auc_m1[i] = auc(fpr_m1[i], tpr_m1[i])
all_fpr_m1  = np.unique(np.concatenate([fpr_m1[i] for i in range(n_classes)]))
mean_tpr_m1 = np.zeros_like(all_fpr_m1)
```

```
for i in range(n_classes):
    mean_tpr_m1 += np.interp(all_fpr_m1, fpr_m1[i], tpr_m1[i])
mean_tpr_m1 /= n_classes
macro_auc_m1 = auc(all_fpr_m1, mean_tpr_m1)
plt.plot(all_fpr_m1, mean_tpr_m1, color=roc_colors[0], linewidth=2,
         label=f'Logistic Regression  (AUC = {macro_auc_m1:.2f})')
auc_scores_list.append({'Model': 'Logistic Regression', 'AUC': macro_auc_m1})

# --- Logit L1 ---
y_score_m2 = m2.predict_proba(X_test_scaled)
fpr_m2, tpr_m2, roc_auc_m2 = {}, {}, {}
for i in range(n_classes):
    fpr_m2[i], tpr_m2[i], _ = roc_curve(y_test_bin[:, i], y_score_m2[:, i])
    roc_auc_m2[i] = auc(fpr_m2[i], tpr_m2[i])
all_fpr_m2  = np.unique(np.concatenate([fpr_m2[i] for i in range(n_classes)]))
mean_tpr_m2 = np.zeros_like(all_fpr_m2)
for i in range(n_classes):
    mean_tpr_m2 += np.interp(all_fpr_m2, fpr_m2[i], tpr_m2[i])
mean_tpr_m2 /= n_classes
macro_auc_m2 = auc(all_fpr_m2, mean_tpr_m2)
plt.plot(all_fpr_m2, mean_tpr_m2, color=roc_colors[1], linewidth=2,
         label=f'Logit L1        (AUC = {macro_auc_m2:.2f})')
auc_scores_list.append({'Model': 'Logit L1', 'AUC': macro_auc_m2})

# --- Logit L2 ---
y_score_m3 = m3.predict_proba(X_test_scaled)
fpr_m3, tpr_m3, roc_auc_m3 = {}, {}, {}
for i in range(n_classes):
    fpr_m3[i], tpr_m3[i], _ = roc_curve(y_test_bin[:, i], y_score_m3[:, i])
    roc_auc_m3[i] = auc(fpr_m3[i], tpr_m3[i])
all_fpr_m3  = np.unique(np.concatenate([fpr_m3[i] for i in range(n_classes)]))
mean_tpr_m3 = np.zeros_like(all_fpr_m3)
for i in range(n_classes):
    mean_tpr_m3 += np.interp(all_fpr_m3, fpr_m3[i], tpr_m3[i])
mean_tpr_m3 /= n_classes
macro_auc_m3 = auc(all_fpr_m3, mean_tpr_m3)
plt.plot(all_fpr_m3, mean_tpr_m3, color=roc_colors[2], linewidth=2,
         label=f'Logit L2        (AUC = {macro_auc_m3:.2f})')
auc_scores_list.append({'Model': 'Logit L2', 'AUC': macro_auc_m3})

# --- Decision Tree ---
y_score_m4 = m4.predict_proba(X_test_scaled)
fpr_m4, tpr_m4, roc_auc_m4 = {}, {}, {}
for i in range(n_classes):
    fpr_m4[i], tpr_m4[i], _ = roc_curve(y_test_bin[:, i], y_score_m4[:, i])
    roc_auc_m4[i] = auc(fpr_m4[i], tpr_m4[i])
all_fpr_m4  = np.unique(np.concatenate([fpr_m4[i] for i in range(n_classes)]))
mean_tpr_m4 = np.zeros_like(all_fpr_m4)
for i in range(n_classes):
    mean_tpr_m4 += np.interp(all_fpr_m4, fpr_m4[i], tpr_m4[i])
mean_tpr_m4 /= n_classes
macro_auc_m4 = auc(all_fpr_m4, mean_tpr_m4)
plt.plot(all_fpr_m4, mean_tpr_m4, color=roc_colors[3], linewidth=2,
```

```
        label=f'Decision Tree  (AUC = {macro_auc_m4:.2f})')
auc_scores_list.append({'Model': 'Decision Tree', 'AUC': macro_auc_m4})

# --- Bagging DT ---
y_score_m5 = m5.predict_proba(X_test_scaled)
fpr_m5, tpr_m5, roc_auc_m5 = {}, {}, {}
for i in range(n_classes):
    fpr_m5[i], tpr_m5[i], _ = roc_curve(y_test_bin[:, i], y_score_m5[:, i])
    roc_auc_m5[i] = auc(fpr_m5[i], tpr_m5[i])
all_fpr_m5  = np.unique(np.concatenate([fpr_m5[i] for i in range(n_classes)]))
mean_tpr_m5 = np.zeros_like(all_fpr_m5)
for i in range(n_classes):
    mean_tpr_m5 += np.interp(all_fpr_m5, fpr_m5[i], tpr_m5[i])
mean_tpr_m5 /= n_classes
macro_auc_m5 = auc(all_fpr_m5, mean_tpr_m5)
plt.plot(all_fpr_m5, mean_tpr_m5, color=roc_colors[4], linewidth=2,
        label=f'Bagging DT     (AUC = {macro_auc_m5:.2f})')
auc_scores_list.append({'Model': 'Bagging DT', 'AUC': macro_auc_m5})

# --- XGBoost ---
y_score_xgb = m_xgb.predict_proba(X_test_scaled) # Predictions for XGBoost
fpr_xgb, tpr_xgb, roc_auc_xgb = {}, {}, {}
for i in range(n_classes):
    fpr_xgb[i], tpr_xgb[i], _ = roc_curve(y_test_bin[:, i], y_score_xgb[:, i])
    roc_auc_xgb[i] = auc(fpr_xgb[i], tpr_xgb[i])
all_fpr_xgb  = np.unique(np.concatenate([fpr_xgb[i] for i in range(n_classes)]))
mean_tpr_xgb = np.zeros_like(all_fpr_xgb)
for i in range(n_classes):
    mean_tpr_xgb += np.interp(all_fpr_xgb, fpr_xgb[i], tpr_xgb[i])
mean_tpr_xgb /= n_classes
macro_auc_xgb = auc(all_fpr_xgb, mean_tpr_xgb)
plt.plot(all_fpr_xgb, mean_tpr_xgb, color=roc_colors[5], linewidth=2,
        label=f'XGBoost        (AUC = {macro_auc_xgb:.2f})')
auc_scores_list.append({'Model': 'XGBoost', 'AUC': macro_auc_xgb})


plt.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random Guessing')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Macro-Averaged ROC Curves – All Models',
          fontsize=13, fontweight='bold')
plt.legend(loc='lower right', fontsize=9)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('plot8_roc_curves.png', dpi=150)
plt.show()
print('Saved: plot8_roc_curves.png')
```



Saved: plot8_roc_curves.png

In [289…

```
auc_vals_dict = {
    'Logistic Regression': macro_auc_m1,
    'Logit L1'           : macro_auc_m2,
    'Logit L2'           : macro_auc_m3,
    'Decision Tree'      : macro_auc_m4,
    'Bagging DT'         : macro_auc_m5,
    'XGBoost'            : macro_auc_xgb # Added XGBoost AUC
}

summary_df = pd.DataFrame({
    'Model'    : model_names,
    'Precision': precision_macro_scores,
    'Recall'   : recall_macro_scores,
    'F1-Score' : f1_macro_scores,
    'AUC'      : [macro_auc_m1, macro_auc_m2, macro_auc_m3, macro_auc_m4, macr
    'Accuracy' : accuracy_scores
})

summary_df = summary_df.sort_values('F1-Score', ascending=False).reset_index(d
summary_df.index += 1

print('=' * 65)
print('   FINAL RESULTS - Ranked by Macro F1-Score')
print('=' * 65)
display(summary_df)
```

```
============================================================
  FINAL RESULTS — Ranked by Macro F1-Score
============================================================
```

	Model	Precision	Recall	F1-Score	AUC	Accuracy
1	XGBoost	0.961230	0.956258	0.958598	0.998877	0.955
2	Logistic Regression	0.961400	0.950071	0.954576	0.996385	0.955
3	Logit L1	0.945012	0.937838	0.940833	0.995876	0.940
4	Logit L2	0.925307	0.897222	0.907773	0.980754	0.910
5	Bagging DT	0.914488	0.899364	0.905934	0.987020	0.900
6	Decision Tree	0.906018	0.891605	0.897761	0.961525	0.890